nexign

Java Fundamentals

Write once, run anywhere!



Александр Митясов,

специалист по технологическому обучению

Nexign Team



Александр Митясов

Старший специалист по технологическому обучению

Технологии JVM: Профиль Java/Kotlin
Курсы Spring framework, groovy, логирование и мониторинг высоконагруженных приложений

nexign

Давайте познакомимся



nexign

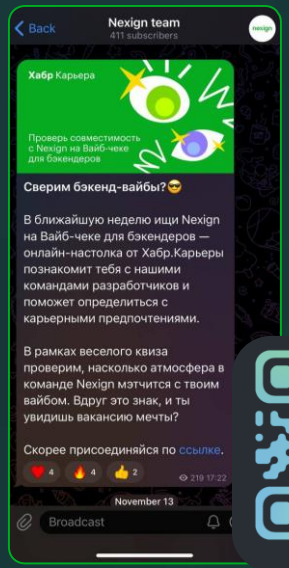
Разработчик софта B2B

2000 сотрудников	11 филиалов
25+ технологических продуктов	150+ реализованных проектов

nexign

Где можно ставить лайки?

Телеграм-канал



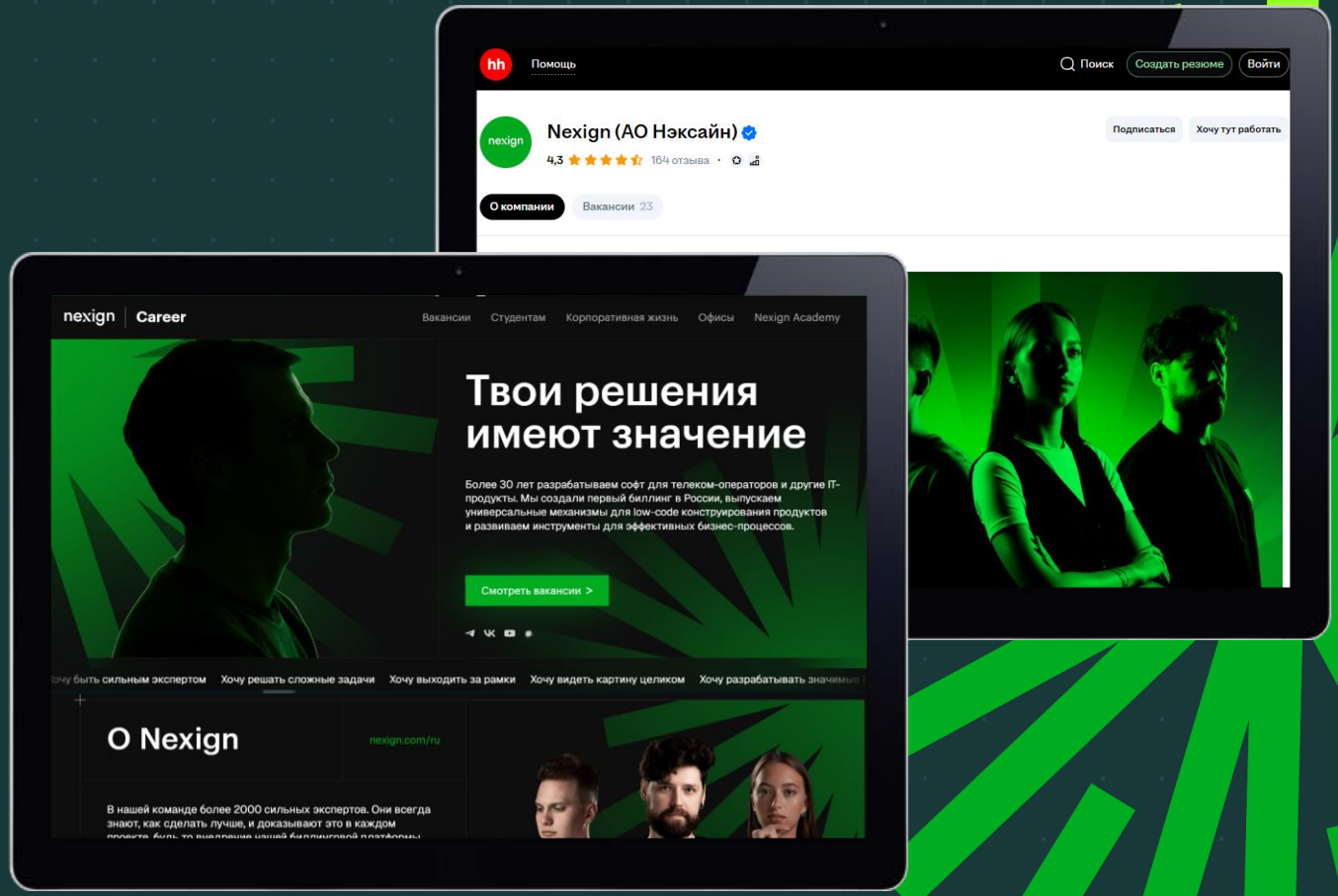


Подкаст «Кто за кодом?»



Где искать вакансии

или карьерные возможности



О чем этот курс

Курс предназначен для ознакомления с языком Java и концепциями ООП.

БЕЗ ПРАКТИКИ

01

Общее представление о программировании на Java и синтаксисе языка.

С ПРАКТИКОЙ

02

Умение создавать и запускать простые приложения на Java (Core). Понимание принципов ООП.

План курса

7 занятий в семестре

ЗАНЯТИЯ

01–06

Теория + практика со сдачей сразу на паре.

- теоретическая часть
- практические задания
- сдача на паре

ЗАНЯТИЕ

07

Защита индивидуального проекта.

- индивидуальный проект
- устная защита

Требования

1. **AI только для пояснения —** не для полной генерации кода. За генерацию — бан.
2. **Дедлайн PR — конец пары.** После — штраф за каждую неделю просрочки.
3. **Болезни и уважительные причины пропуска —** писать в личные сообщения преподавателю.
4. **Защита только очно.** Онлайн-защита проектов не предусмотрена.

Тест

Отсканируйте QR-код для прохождения входного теста.

О ТЕСТЕ

Тема: основы Java и ООП

Время: 10 минут

Формат: онлайн



Зачем учить язык (Java / C# / C++ / Python / SQL и т.п.) в эру AI-агентов?

Язык программирования — это не про синтаксис. Это про способ мышления и ответственность за результат.

01 **AI не понимает бизнес-контекст**

Агенты генерируют код по промпту, но не знают предметную область, требования заказчика и архитектурные ограничения вашего проекта. Решение остаётся за инженером.

02 **Code review и аудит**

Без знания языка невозможно оценить качество сгенерированного кода: найти баги, уязвимости, проблемы производительности и архитектуры.

03 **Декомпозиция задач**

Программирование учит разбивать сложную задачу на атомарные шаги. Этот навык нужен и для написания промптов, и для отладки AI-выдачи.

04 **Ответственность за результат**

AI ускоряет рутину, но ответственность за работающий код в продакшене несёт разработчик, а не модель. Нужно уметь читать и править код.

05 **Системное мышление**

Понимание типов данных, памяти, потоков и API формирует инженерное мышление, которое не заменяется ни одним AI-инструментом.

06 **Язык = коммуникация**

Код — это формальная спецификация требований, читаемая и командой, и AI. Без общего языка разработчик становится пассажиром, а не автором.

nexign

Основы технологии Java

Понятия и термины



nexign

План урока

- Java – что это? История появления
- Особенности языка
- Инструменты для работы
- Настройка и установка необходимого ПО

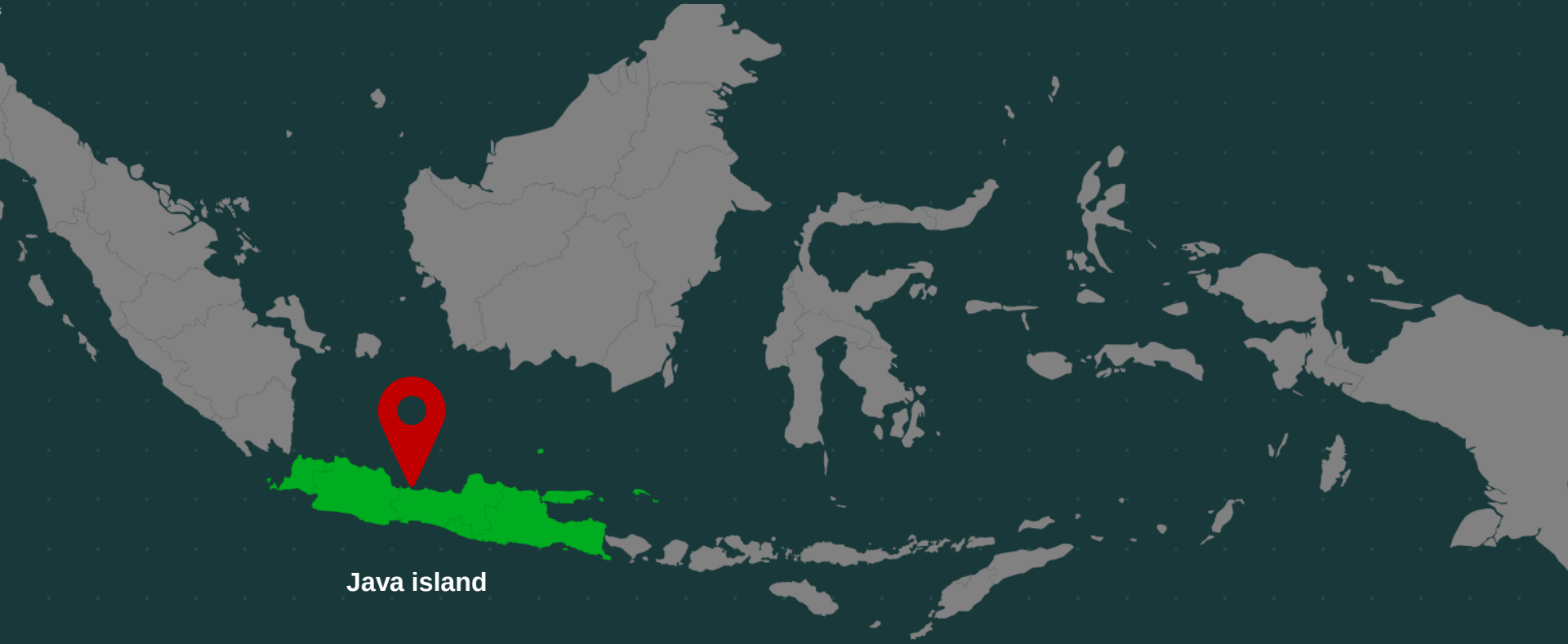
nexign

nexign

Java. История и основные понятия



JAVA – что это?



nexign

Java

мультипарадигменный (преимущественно объектно - ориентированный) статически типизированный кроссплатформенный язык программирования общего назначения.



nexign

История появления

1991
внутренний проект Sun Microsystems по созданию платформы для разработки встраиваемых систем — Green Project;

1992
первое демонстрационное устройство на новой платформе — PDA Star7

1994
фокус на разработке интерактивных приложений (апплетов) для веб-страниц;
Язык переименован в Java

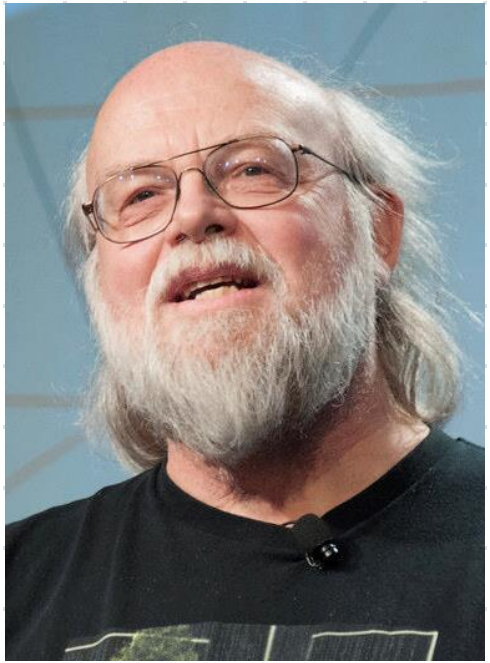
1996
Java Development Kit 1.0



Маскот языка Duke

nexign

Джеймс Гослинг



- Переносимость
- Синтаксис близкий к C++
- Язык для бекенда и встаиваемых систем

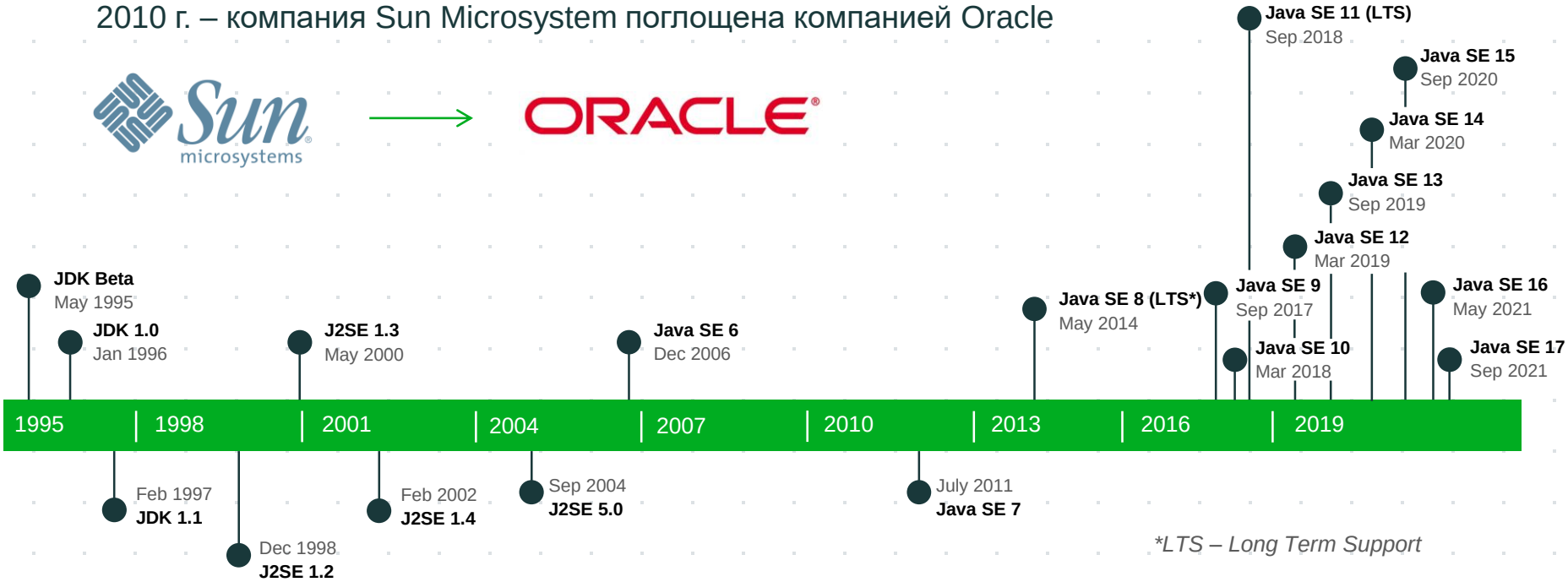
Q&A with James Gosling, June 2020:

https://www.youtube.com/watch?v=XThhlihqTXI

nexign

История версий

2010 г. – компания Sun Microsystem поглощена компанией Oracle



LTS – Long Term Support

nexign

Редакции Java

- Standard Edition (SE)
- Enterprise Edition (EE)

  SE + дополнительные библиотеки и возможности
- Micro Edition (ME)

  подмножество SE + специфические библиотеки
- Java Card

  урезанная версия SE, изменения в виртуально машине

nexign

nexign

Особенности языка Java



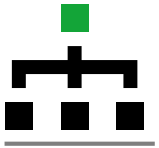
Особенности языка



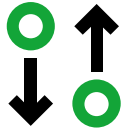
Переносимый



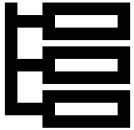
Безопасный



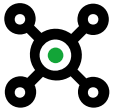
Объектно-ориентированный



С автоматическим управлением памятью



Многопоточный

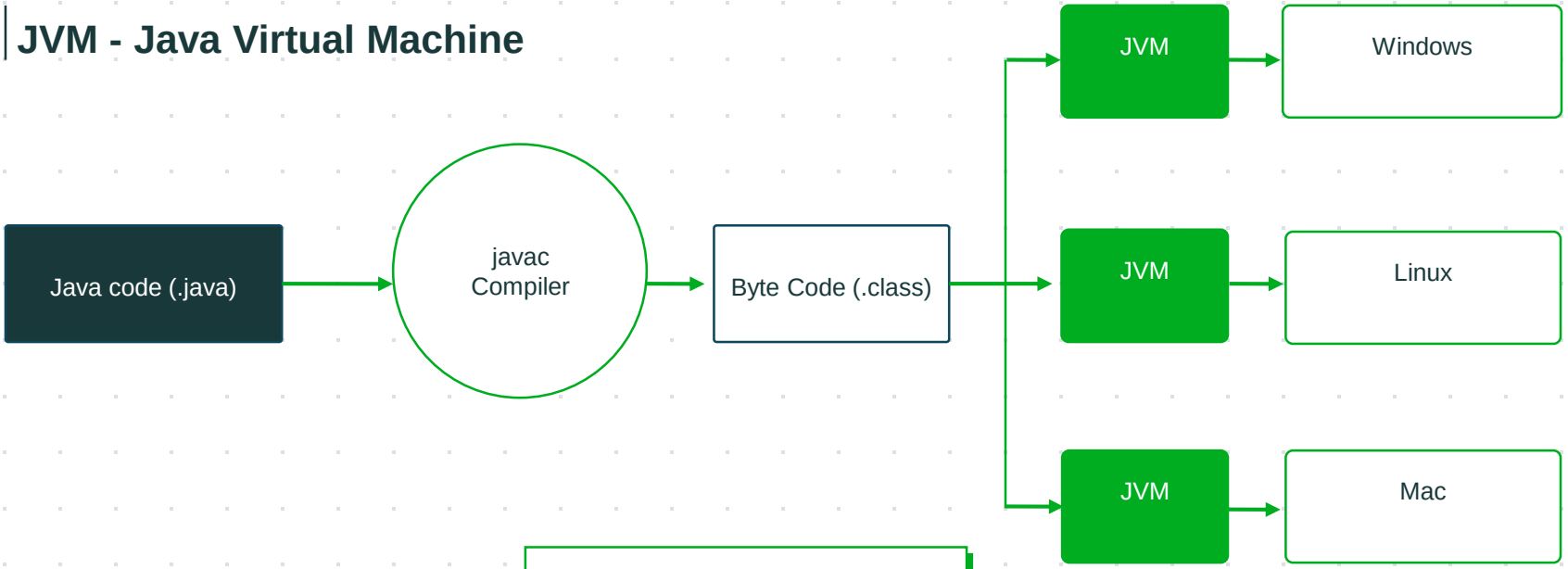


Распределённый

nexign

Переносимость

JVM - Java Virtual Machine



Write Once, Run Anywhere

nexign

JIT

JIT – Just In Time compiler

Участки кода, которые выполняются **часто**

компилятор может оптимизировать, «на лету»

преобразовывая фрагмент сразу в машинный код.



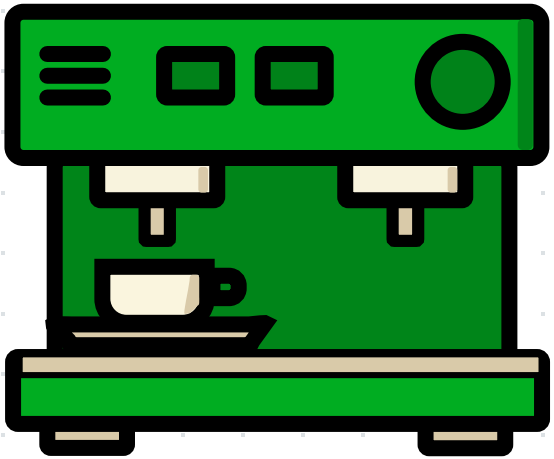
nexign

JVM

- **HotSpot (Reference Implementation) - Oracle Java**
- Azul Zing
- IBM I9 (Open J9)
- Amazon Coretto
- SAP JVM
- Liberica JDK
- …

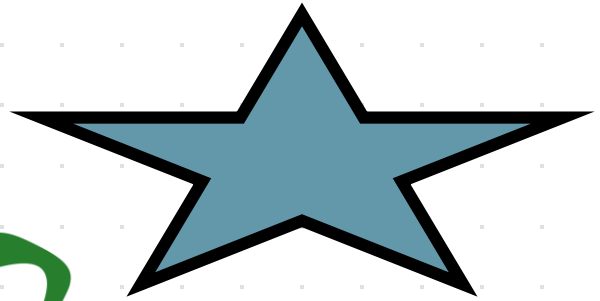
20+

Реализаций



nexign

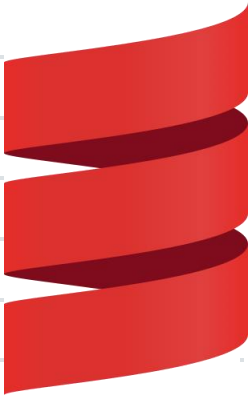
JVM языки



Groovy



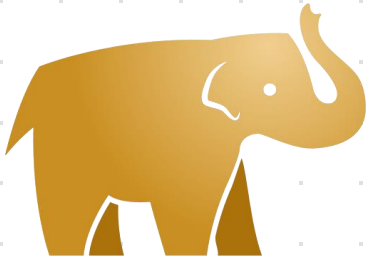
Kotlin



Scala



Jython



Ceylon



Clojure

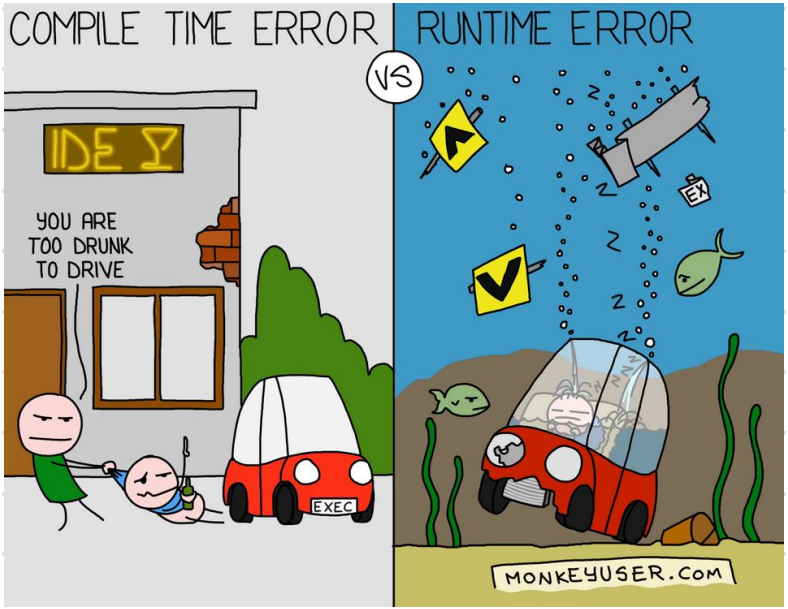


JRuby

nexign

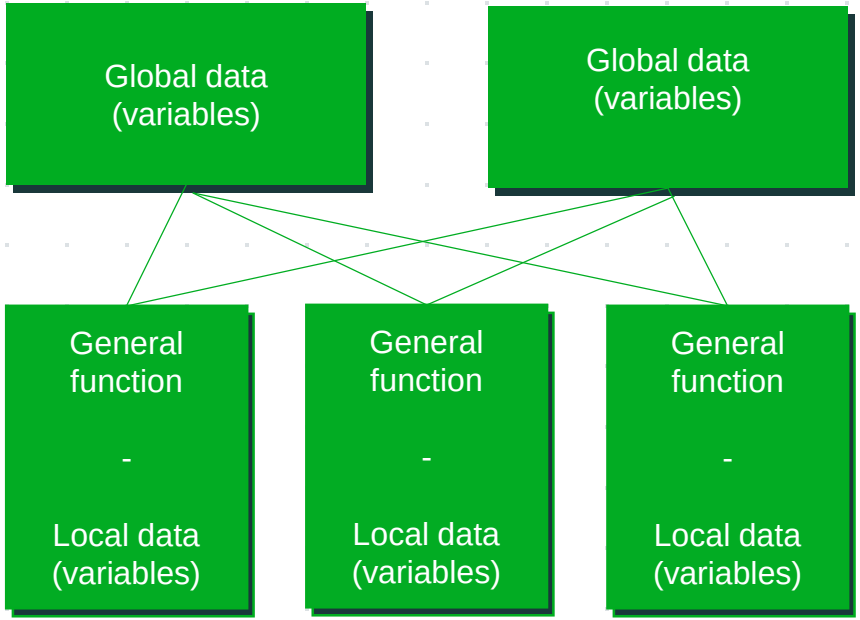
Безопасность

- Верификация байткода
- Автоматическое управление памятью
- Встроенный механизм управления правами

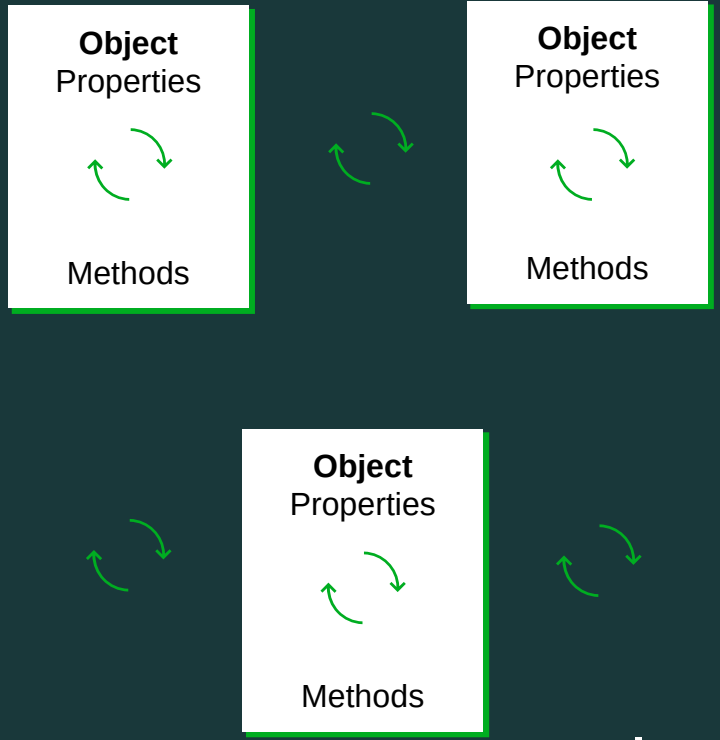


nexign

Процедурное



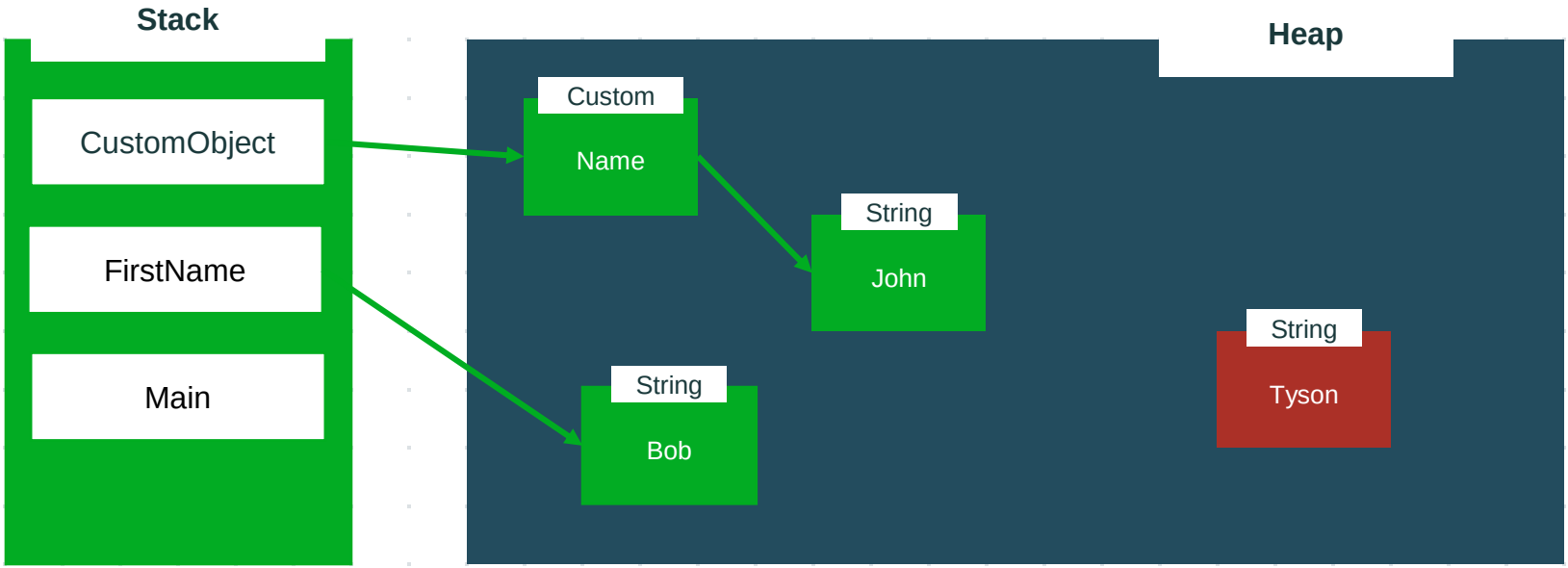
ООП



nexign

Управление памятью

GC – Garbage Collector



nexign

Garbage Collector

Достоинства:

- Меньше кода
- Не бывает некорректных ссылок (на разрушенный объект)
- Меньше утечек памяти
- Разработчику не нужно заботиться о высвобождении памяти

Недостатки:

- Не контролируем время работы GC
- STW-паузы (бывает, в неподходящий момент)



nexign

Многопоточность и распределенность

Многопоточность

- встроенная поддержка потоков
- богатая библиотека примитивов синхронизации

Распределенность

- встроенные сетевые возможности
- пересылка данных и объектов по сети
- работа с удаленными объектами (RMI)

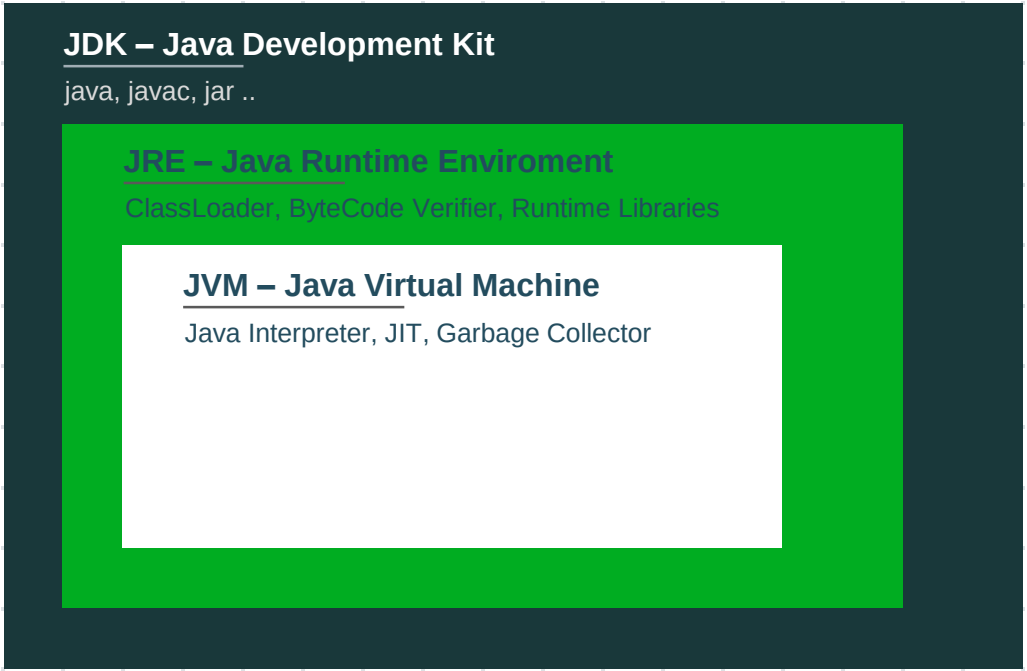
nexign

nexign

Основные инструменты для работы



Инструменты



JDK

- **Oracle JDK**
- **OpenJDK Build от Oracle**
- AdoptOpenJDK
- Red Hat OpenJDK
- Azul Zulu
- Amazon Corretto

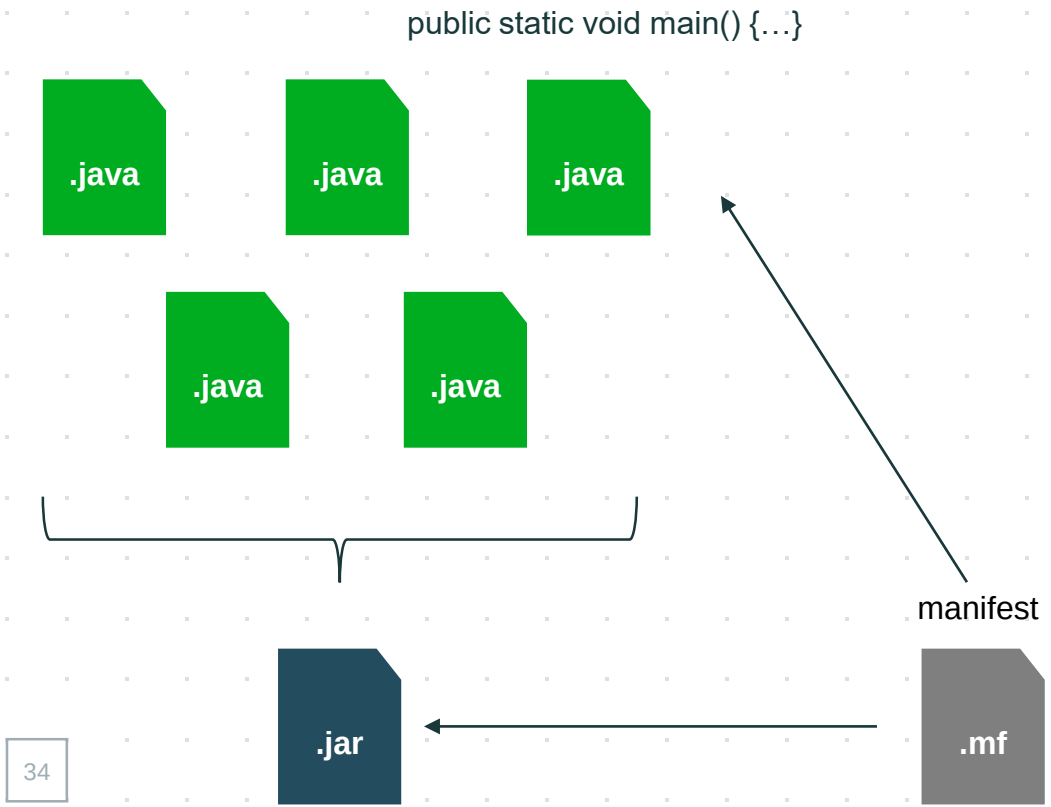
nexign

Утилиты JDK

- **javac** – компилирует исходный код (*.java) в байткод (*.class)
- **java** – Исполняет байткод
- **jdb** – отладчик
- **javadoc** – генератор документации
- **jar** – создание и управление архивами jar
- **javah** – генератор h-файлов C/C++ интерфейса JNI
- **javap** – дизассемблер классов
- **extcheck** – обнаружение конфликтов между файлами архивов jar

nexign

JAR



Программа состоит из множества файлов - классов.

Чтобы не путаться в том, какой именно файл запускать, классы объединяют в архивы с расширением .jar, с добавлением специального файла manifest, в котором есть указатель на файл, содержащий “главный” метод **main()**

nexign

Важнейшие элементы экосистемы

- Бибилиотеки и фреймворки (в т. ч. для тестирования)
- Среды разработки
- Системы сборки
- CI



Jenkins

nexign

Q&A



Ещё по этой теме



Книги:

- Cay S. Horstmann. Core Java vol 1 & 2
- Raoul-Gabriel Urma et al. Modern Java in Action
- Joshua Bloch Effective Java
- Brian Goetz et al. Java Concurrency in Practice
- Head First Java, 3nd Edition by Kathy Sierra, Bert Bates

nexign

Структура программы
Типы данных



nexign

План урока

- Структура программы
- Примитивные типы данных
- Ссылочные типы

nexign

nexign

Структура программы



Синтаксис

You **vs** **The guy she tells you not to worry about**

```java
public class Main {
    public static String reverseString(String str) {
        StringBuilder reverse = new StringBuilder();
        for (int idx = hello.length() - 1; idx >= 0; idx--) {
            reverse.append(hello.charAt(idx));
        }
        return reverse.toString();
    }

    public static void main(String[] args) {
        String hello = "Hello world!";
        System.out.println(reverseString(hello));
    }
}
```

```python
hello = 'Hello world!'
print(hello[::-1])
```

https://www.reddit.com/r/ProgrammerHumor/comments/66jj7f/java_vs_python/

Структура программы

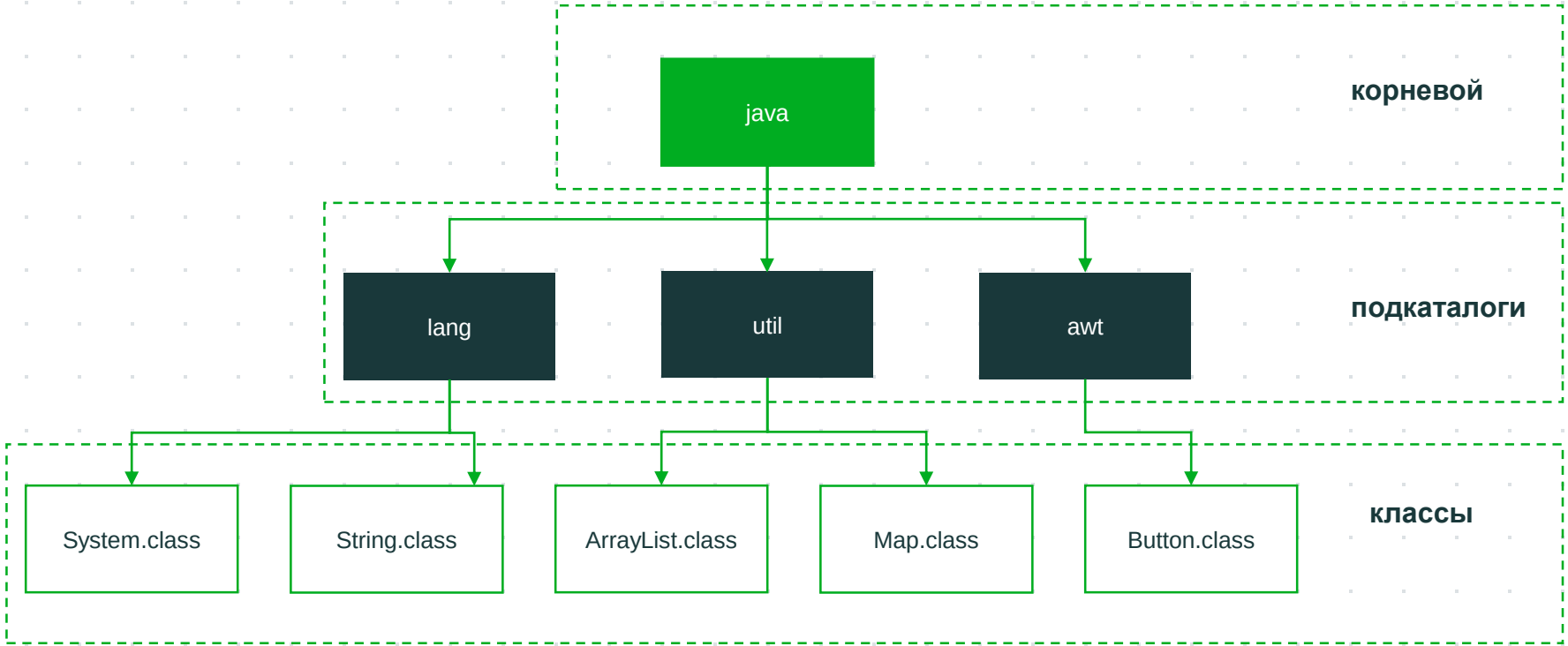
- Package statement
- Import statements
- Class declaration{
  - Variables
  - Constructors
  - Methods
  - Nested classes
  - Nested interfaces
  - Enum

  }

```java
package test;
import java.util.Scanner;
public class Test {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Введите число:");
        int number = sc.nextInt();
        System.out.println("Вы ввели " + number);
    }
}
```

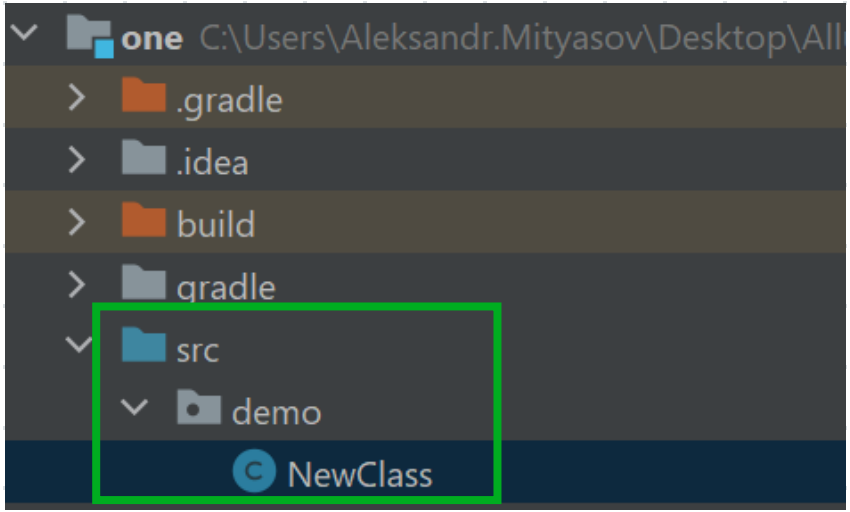

nexign

Packages



nexign

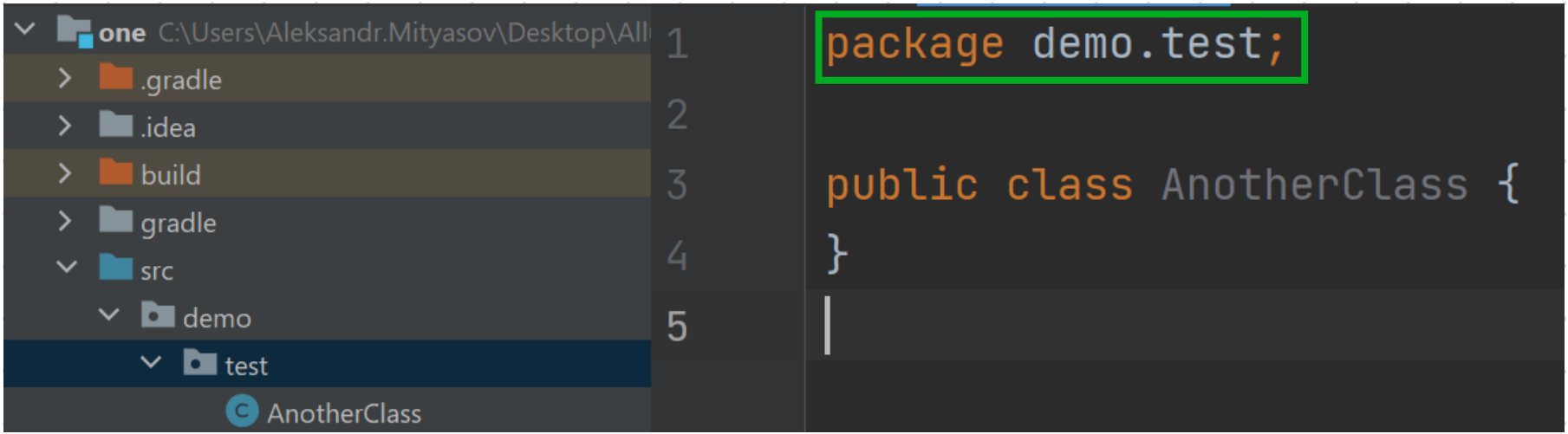
Packages




```
1  package demo;
2
3  public class NewClass {
4  }
5
```


nexign

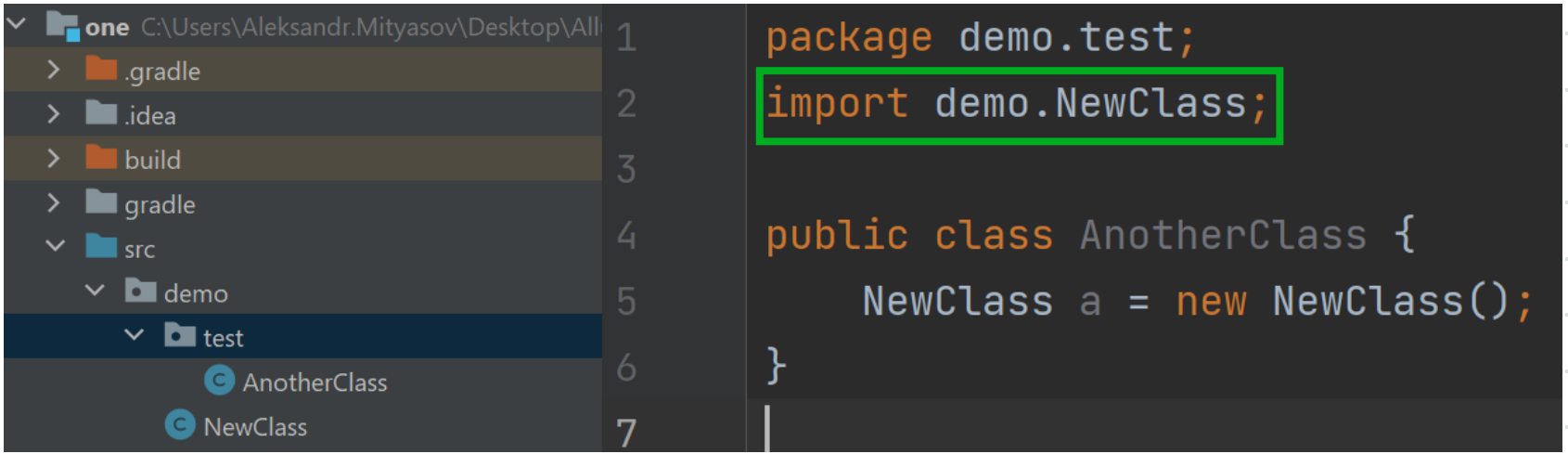
Packages



nexign

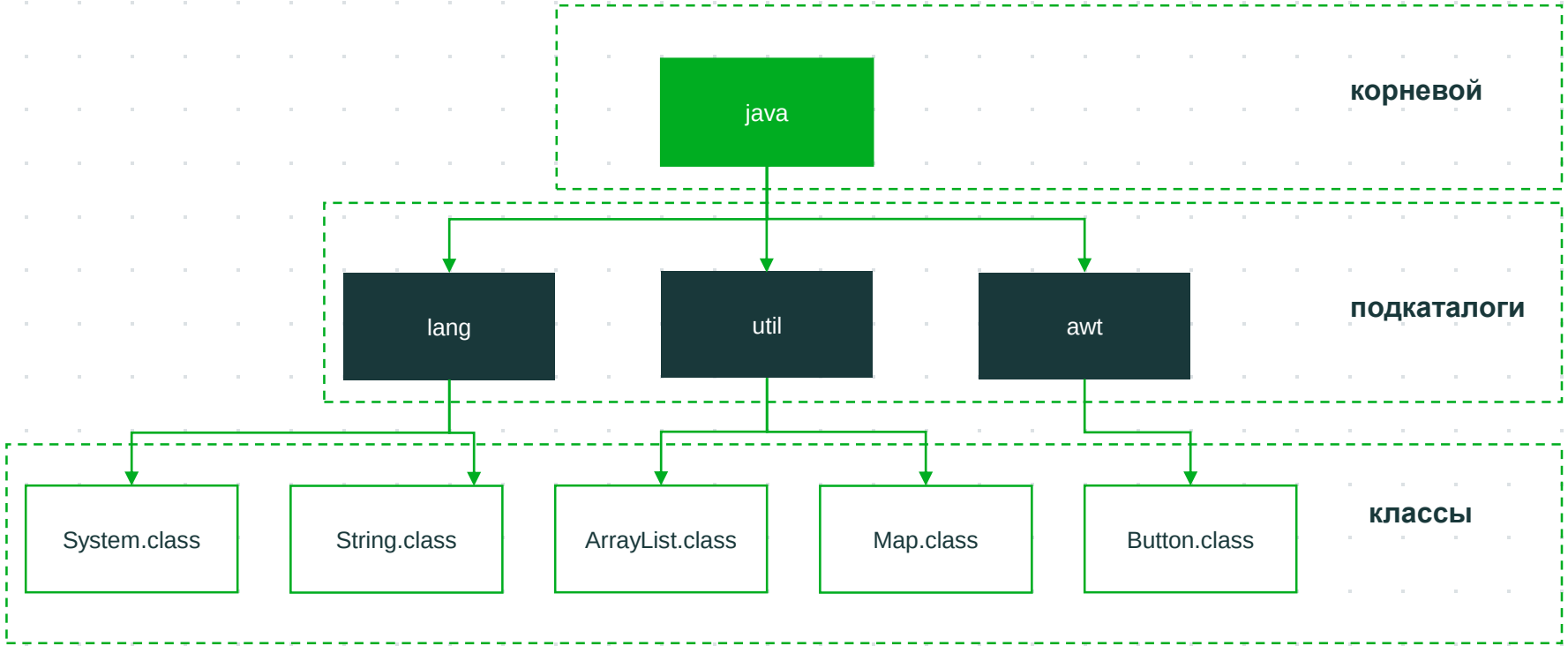
Import

Оператор **import** сообщает компилятору Java, где найти классы, на которые ссылается код.



nexign

Packages



nexign

Import

```
import java.util.*; //импорт всех классов из пакета
```

```
package demo.test;
import java.lang.Math;

public class AnotherClass {
   double a = Math.sqrt(4);
}
```



```
package demo.test;
import static java.lang.Math.sqrt;

public class AnotherClass {
   double a = sqrt(4);
}
```


nexign

Import

Оператор **import** сообщает компилятору Java, где найти классы, на которые ссылается код.

```
import java.util.*; // импорт всех классов из пакета java.util
import java.util.Scanner;
import static java.lang.Math.sqrt;
```


nexign

Class

Программа, которая печатает в консоль фразу «Hello World»

```
package test;
public class Test {
    public static void main(String[] args) {
        System.out.println("Hello World");
    }
}
```


nexign

Class

```
package test;
public class Test {
    public static void main(String[] args) {
        System.out.println("Hello World");
    }
}
```

Имя класса = имя файла

Регистр важен

nexign

Class

```
package test;
public class Test {
    public static void main(String[] args) {
        System.out.println("Hello World");
    }
}
```

Модификатор уровня доступа

***public** – нет ограничений*

на доступ к классу

nexign

Class

```
package test;
public class Test {
    public static void main(String[] args) {
        System.out.println("Hello World");
    }
}
```

static *– позволяет вызывать метод* ***main()*** *без получения экземпляра класса*

nexign

Class

```
package test;
public class Test {
    public static void main(String[] args) {
        System.out.println("Hello World");
    }
}
```

main() *– имя метода класса (член класса), ключевое слово* ***void*** *указывает на то, что метод ничего не возвращает*

nexign

Class

```
package test;
public class Test {
    public static void main(String[] args) {
        System.out.println("Hello World");
    }
}
```

args *– имя массива экземпляров класса* ***String*** *и принимает любые аргументы командной строки, присутствующие во время выполнения программы*

Тип_данных имяПеременной

nexign

Class

```
package test;
public class Test {
    public static void main(String[] args) {
        System.out.println("Hello World");
    }
}
```

*В IntelliJ Idea,можно быстро создать метод при помощи сниппета **«psvm»***

nexign

Class

```
package test;
public class Test {
    public static void main(String[] args) {
        System.out.println("Hello World");
    }
}
```

println() *отображает переданную ему символьную строку с переводом на новую строку,* ***System*** *обозначает предопределенный класс, предоставляющий доступ к системе, а* ***out*** *- поток вывода, связанный с консолью*

nexign

Class

```
package test;
public class Test {
    public static void main(String[] args) {
        System.out.println("Hello World");
    }
}
```

*Сниппет **«sout»***

nexign

nexign

Рекомендации по стилю кода

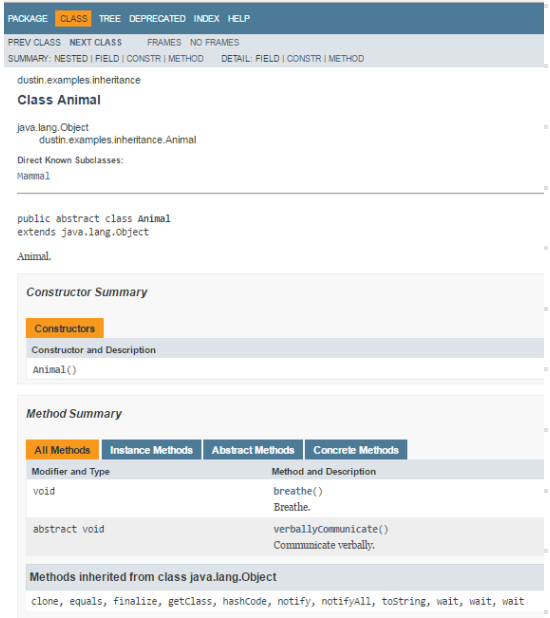


Комментарии

- Однострочный // …
- Многострочный /* … */
- Многострочный комментарий документации (javadoc) /** … */

Для вывода документации на русском,
использовать команды в качестве аргументов при сборке:

-encoding UTF-8 -docencoding UTF-8 -charset UTF-8



JavaDoc File

nexign

Именование объектов

- Классы
- Переменные
- Константы
- Методы

С большой буквы

Стиль CamelCase

Пример: MyFirstClass

С маленькой буквы

Использовать понятные по смыслу значения

Пример: userId, todayTemperature

Заглавные бкувы. Разделители «_»

Пример: MAX_VALUE

С маленькой буквы

Использовать глагольные выражения

Пример: getCount(), setTemperature()

nexign

Рекомендации по стилю кода

- В промышленной разработке важно, когда все файлы имеют согласованный стиль!
- Code Conventions for the Java ™ Programming Language (https://www.oracle.com/technetwork/java/codeconvtoc-136057.html)
- **Правила:**

    1. Комментарии/Javadoc: пишите их; используйте стандартный стиль.
    2. Короткие методы: не пишите объёмных (по коду) методов.
    3. Поля: должны быть вверху файла, или прямо перед методом, который их использует.

    ….

nexign

nexign

Типы данных.
Определение переменных



Определение переменных

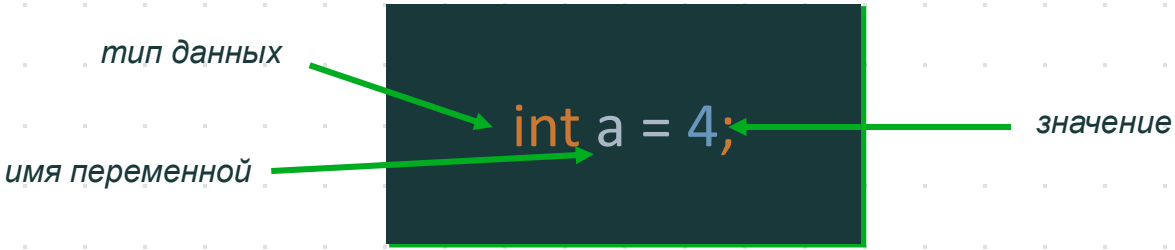
Декларация

```
int a;
```

Инициализация

```
a = 4;
```

Декларация + инициализация



nexign

Статическая типизация

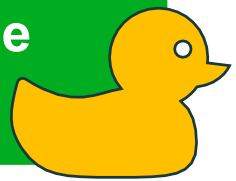
```
int a = 4;
a = "строка"; //ошибка нессответствия типов
//или
String a = "строка"; // тип изменить нельзя
```

```
int a;
System.out.println(a);
//ошибка - переменная не инициализированна
```


nexign

Типы данных

Примитивные



- Логическое : boolean
- Целочисленные: byte, short, int, long
- С плавающей точкой: float, double
- Символьные: char

Ссылочные

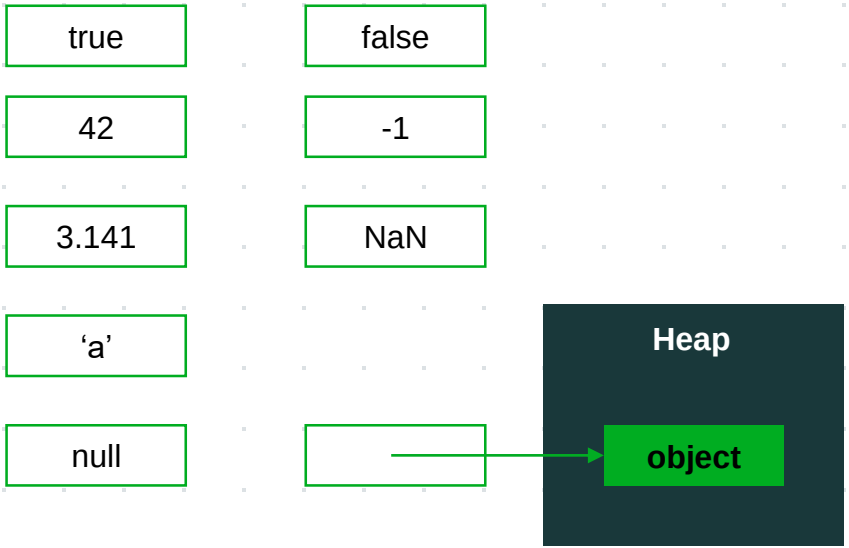


- Все остальные

nexign

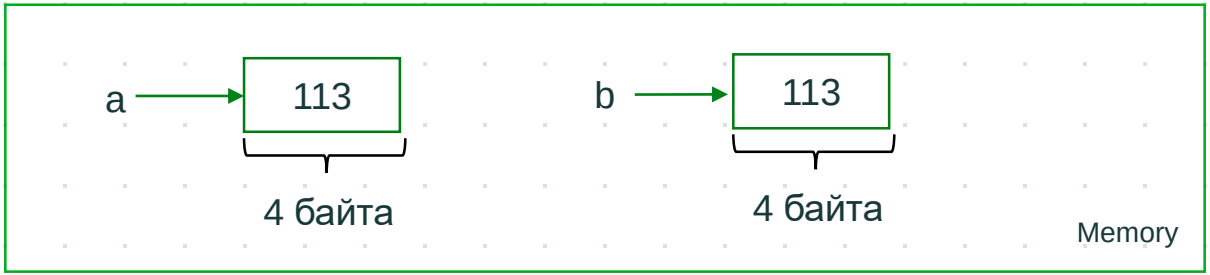
Типы данных

- Логическое : boolean
- Целочисленные: byte, short, int, long
- С плавающей точкой: float, double
- Символьные: char
- Ссылочные



nexign

Устройство примитивных типов




```
int a = 113;
int b = a;   // копирование по значению
```


nexign

Константы (final)

final применительно к переменным, указывает на запрет изменения значений после инициализации

```
final int a;
...
a = 42; // инициализация

...
a = 43; // compilation error:
// variable a might already have been initialized
```


nexign

nexign

Типы данных. Примитивы



Типы данных

- Логическое : boolean
- Целочисленные: byte, short, int, long
- С плавающей точкой: float, double
- Символьные: char

nexign

Типы данных

- **Логическое : boolean**
- Целочисленные: byte, short, int, long
- С плавающей точкой: float, double
- Символьные: char

nexign

Boolean

- Литералы: **false, true**
- Результат любого сравнения — **boolean**:

  < > == <= >= !=
- Нет преобразования между **boolean** и другими примитивными типами

```
boolean b = true;
//не сработает
if (b == 1) {...}
```


nexign

Boolean

Логические операторы

Оператор	Описание
&&	Логический оператор «И»
||	Логический оператор «ИЛИ»
!	Логический оператор «НЕ»
^	Логический оператор исключающее или (XOR)

- && и || — вычисление по сокращенной схеме
- & и | — вычисление по полной схеме

```
if (c instanceof Integer && c > 5) {...}
```

Операторы сравнения

Оператор	Описание
==	равны
!=	не равны
>	строго больше
<	строго меньше
>=	больше или равно
<=	меньше или равно

nexign

Типы данных

- Логическое : boolean
- **Целочисленные: byte, short, int, long**
- С плавающей точкой: float, double
- Символьные: char

nexign

Целочисленные

Type	Storage	Range
byte	1 byte	–128 to 127
short	2 bytes	–32,768 to 32,767
int	4 bytes	–2,147,483,648 to 2,147,483, 647
long	8 bytes	–9,223,372,036,854,775,808 to 9,223,372,036,854,775,807

- Размер фиксирован, одинаков для всех платформ
- Все типы знаковые, беззнаковых вариантов нет

nexign

Целочисленные литералы

- Long 100L
- Underscores 1_000_000
- Hexadecimal 0xCAFEBABE
- Binary 0b0101
- Octal 010

```
long a = 100L;

int b = 1_000_000;

int c = 0x2D;

int d =0b0101;

int x = 010;
```


nexign

Арифметические операции

Оператор	Описание
+	Складывает значения по обе стороны от оператора
-	Вычитает правый операнд из левого операнда
*	Умножает значения по обе стороны от оператора
/	Оператор деления делит левый операнд на правый операнд
%	Делит левый операнд на правый операнд и возвращает остаток
++	Инкремент - увеличивает значение операнда на 1
--	Декремент - уменьшает значение операнда на 1

nexign

Арифметические операции



- a++, ++a
- a--, --a
- +=, -=, *=, /=, %=, &=, |=, ^=, <<=, >>=, >>>=

```
y= 10;
z = ++y * y--;
```

```
y = y + 1;
z = 11 * 11;
y = y - 1;
```


nexign

Приоритет операторов

Категория	Оператор	Ассоциативность
Постфикс	() [] . (точка)	Слева направо
Унарный	++ - - ! ~	**Справа налево**
Мультипликативный	* / %	Слева направо
Аддитивный	+ -	Слева направо
Сдвиг	>> >>> <<	Слева направо
Реляционный	> >= < <=	Слева направо
Равенство	== !=	Слева направо
Побитовое «И» («AND»)	&	Слева направо
Побитовое исключающее «ИЛИ» («XOR»)	^	Слева направо
Побитовое «ИЛИ» («OR»)	|	Слева направо
Логическое «И» («AND»)	&&	Слева направо
Логическое «ИЛИ» («OR»)	||	Слева направо
Условный	?:	**Справа налево**
Присваивание	= += -= *= /= %= >>= <<= &= ^= |=	**Справа налево**
Запятая	,	Слева направо

nexign

Типы данных

- Логическое : boolean
- Целочисленные: byte, short, int, long
- **С плавающей точкой: float, double**
- Символьные: char

nexign

Типы с плавающей точкой

Type	Storage	Range
float	4 bytes	Approximately ±3.40282347E+38F (6–7 significant decimal digits)
double	8 bytes	Approximately ±1.79769313486231570E+308 (15 significant decimal digits)

По умолчанию: **double**

nexign

Литералы для типов с плавающей точкой

- 2.998e8
- 1.0 (1.)
- 3.14F
- 0x1.0p-3 (0.125, 2 в минус третьей степени)
- Double.POSITIVE_INFINITY
- Double.NEGATIVE_INFINITY
- Double.NaN

nexign

Проблемы float и double

- Побитовые операции не поддерживаются
- System.out.println(2.0 - 1.1) выводит 0.8999999999999999
- System.out.println(2.0 / 0.0) выводит 'Infinity` (но 2 / 0 — divizion by zero)
- System.out.println(0.0 / 0.0) выводит 'NaN`
- Сравнение с бесконечностью и NaN не работает, надо проверять

  с помощью Double.isNaN, Double.isInfinite.
- strictfp – вычисления соответствующие стандартам double и float

nexign

BigInteger, BigDecimal

- Лучше когда арифметика и сравнения просто работают
- Корректность гарантирована, корректность почти всегда важнее скорости
- Нет предыдущих проблем с double и float
- Очень большой диапазон

```
BigDecimal num1 = BigDecimal.ONE;
BigDecimal num3 = new BigDecimal(3);
System.out.println(num1.divide(num3, 3, RoundingMode.CEILING));
//0.334
System.out.println(num1.divide(num3, 3,RoundingMode.HALF_UP));
//0.333
```


nexign

Типы данных

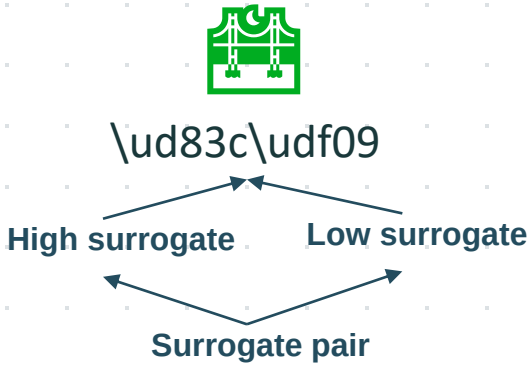
- Логическое : boolean
- Целочисленные: byte, short, int, long
- С плавающей точкой: float, double
- **Символьные: char**



nexign

Char

- char — 16 бит, беззнаковый (0 .. 2 16 − 1)
- Представляет номер символа в кодировке Unicode
- Литералы: символ в одинарных кавычках: ’a’
- Шестнадцатеричный код символа: ’\u78bc’
- Спецпоследовательности: ’\t’, ’\n’, ’\r’, ’\”, ’\\’
- Свободно конвертируется в числовые типы и обратно




```
ch1 = 67; // код переменной
ch2 = 'a'; // сам символ
ch3 = 116; // код переменной
System.out.println("Кот по-английски: " + ch1 + ch2 + ch3);
```


nexign

nexign

Типы данных. Преобразование типов



Расширяющее

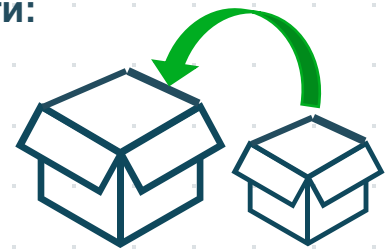
byte → short → int → long → float → double

char

```
short s = b;
int i = s;
long l = i;
float f = l;
double d = f;
```

Потеря точности:

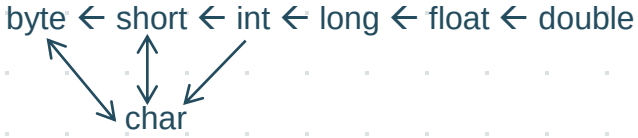
- int → float
- long → float
- long → double



Примитивный тип	Размер в памяти
byte	8 бит
short	16 бит
char	16 бит
int	32 бита
long	64 бита
float	32 бита
double	64 бита
boolean	8 (при использовании в массивах), 32 (при использовании не в массивах)

nexign

Сжимающее




```
a = (int) b;
```

(целевой_тип) значение

Потеря точности:

Всегда

- Оператор приведения типа: (typename)
- При приведении более емкого целого типа к менее емкому старшие биты просто отбрасываются
- При приведении типа с плавающей точкой к целому типу дробная часть отбрасывается (никакого округления)
- Слишком большое дробное число при приведении к целому превращается в MAX_VALUE или MIN_VALUE
- Слишком большой double при приведении к float превращается в Float.POSITIVE_INFINITY или Float.NEGATIVE_INFINITY

nexign

Автоматическое расширение



При вычислении выражения (a @ b) аргументы a и b преобразовываются в числа, имеющие одинаковый тип:

- если одно из чисел double, то в double;
- иначе, если одно из чисел float, то в float;
- иначе, если одно из чисел long, то в long;
- иначе оба числа преобразуются в int

```
byte b = 1;
byte c = b + 1; // compilation error
```

```
double a = Long.MAX_VALUE;
long b = Long.MAX_VALUE;
int c = 1;
System.out.println(a+b+c);
System.out.println(c+b+a);
```

nexign

Неявное приведение



- Сокращенная запись

  var @= expr раскрывается в

  var = (typename) (var @ (expr))

- Неявно срабатывает приведение типа,

  в том числе с потерей данных

```
byte b = 1;
b += 1; // OK
```

nexign

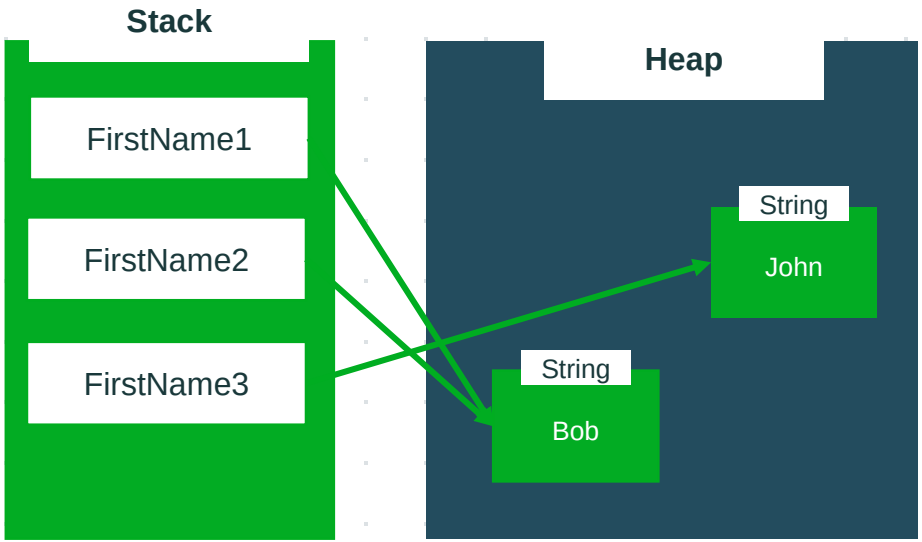
nexign

Типы данных. Ссылочные типы



Ссылочные типы

- Все остальные, кроме примитивных
- Передаются по ссылке
- Являются объектами
- Имеют поля и методы
- Ссылка может принимать значение null



nexign

Создание объекта

Для создания объекта в памяти используется оператор **new**

```
BigInteger number;
number = new BigInteger("12123");
...
BigInteger number2 = new BigInteger("123123");
```



nexign

Классы обёртки (Wrapper Classes)

Примитивные типы	Wrapper Classes
int	Integer
short	Short
long	Long
byte	Byte
float	Float
double	Double
char	Character
boolean	Boolean

nexign

Wrapper Classes



Объекты классов обёрток *кэшируются*, поэтому лучше их не создавать через **new**, а через **valueOf**.

Кэширование

Integer	Byte	Short/Long	Character	Boolean	Float/Double
-128…127	-128…127	-128…127	0…127	TRUE/FALSE	нет

Можно поднять верхнюю границу через параметр

-XX:AutoBoxCacheMax=<size>

Сравнение ссылок

- Оператор == сравнивает ссылки на объекты и выдает true тогда и только тогда, когда они указывают на один и тот же объект в памяти
- Для сравнения объектов по содержанию применяется метод equals(…), доступный всем классам в Java

```
String s1 = "abc";
String s2 = "ab" + "c";
System.out.println(s1 == s2);
System.out.println(s1.equals(s2));
```


nexign

Boxing/unboxing

- Autoboxing: примитивное значение → объект-обертка
- Autounboxing: объект-обертка → примитивное значение

```
Integer i = 1;
Integer j = i + 1;
int k = i + j;
```


nexign

nexign

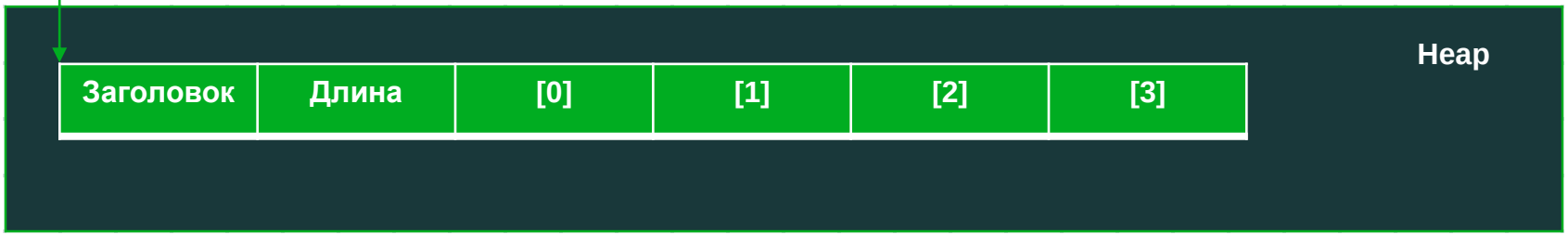
Типы данных. Ссылочные типы. Массивы



Массивы

- Из любого типа можно построить массив данного типа.
- Длина массива может быть определена в runtime, но после создания не может быть изменена.
- Массив аллоцируется в куче и передаётся по ссылке.
- Массивы проверяют тип данных (ArrayStoreException) и границы (ArrayIndexOutOfBoundsException) в run-time.

```
int[] ints;
```



nexign

Инициализация массивов

Шаблон

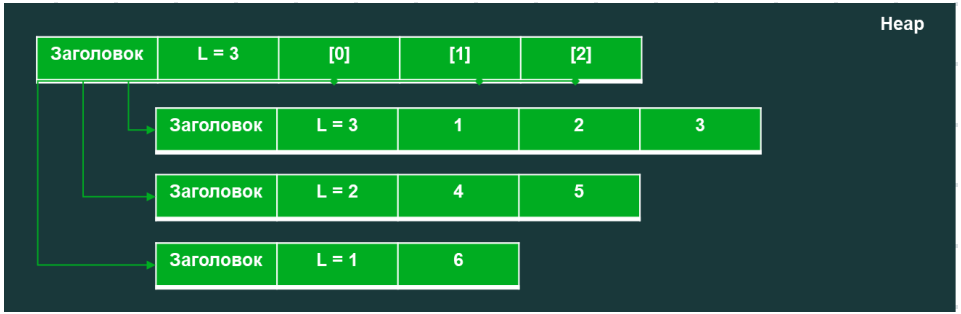
тип имя_переменной [ ] =
new тип [размер] {список_значений};

```
int  week_days[] = new int []
{1, 2, 3, 4, 5, 6, 7};
```

```
int numbers[] = {1, 2, 3, 4, 5};
int arrayLength = numbers.length ;
int firstNumber = numbers[0];
```

Многомерные массивы

```
int[][] initTable = {{1,2,3}, {4,5}, {6}};
```



nexign

Массивы передаются по ссылке



```
int[] smallPrimes = {0, 1, 2, 3, 4, 5, 6}
int[] luckyNumbers = smallPrimes;
luckyNumbers[5] = 12; // теперь smallPrimes[5] тоже 12

luckyNumbers = Arrays.copyOf(luckyNumbers, 2 * luckyNumbers.length);
//теперь luckyNumbers это отдельный массив
//и он стал в два раза длиннее
```

nexign

Операции с массивами



```
int [] ints = {1,2,3,2,1};                          //5
System.out.println(ints.length);                    //1
System.out.println(ints[0]);                        //ints[1] = 3
ints[1] +=1;                                        //ArrayIndexOutOfBoundsException
ints[10] =10;
int[] copy = ints.clone();                          //false
System.out.println(ints == copy);                   //false
System.out.println(ints.equals(copy));              //true
System.out.println(Arrays.equals(ints, copy));      /[I@54bedef2
System.out.println(ints);                           //[1, 3, 3, 2, 1]
System.out.println(Arrays.toString(ints));          //[1,1,2,3,3,]
Arrays.sort(ints);
```

nexign

nexign

Типы данных. Ссылочные типы. Строки



Строки

- Для работы с текстовыми данными в **Java** чаще всего используется класс **String**.
- Класс **String** имеет две фундаментальные особенности:
  - **immutable** (неизменный) класс;
  - **final** класс.
- Класс **String** поддерживается на уровне виртуальной машины.
- Перегружен оператор **"+".**
- Неизменность класса **String**, порождает новые экземпляры строк, а старые отбрасываются, создавая большое количество мусора.

nexign

Работа со строками

```
String s1 = "abc";              // Упрощенная инициализация строки s2:
String s2 = "abc";              // Инициализация сроки s3
String s3 = "ab" + "c";         // конкатенация
String s4 = new String("abc"); // Классическая инициализация строки
```

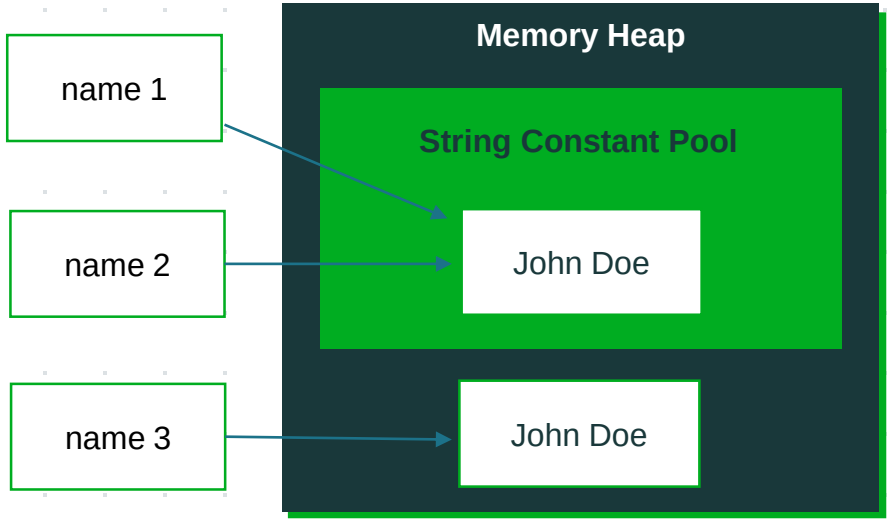

nexign

Внутреннее представление

- До Java 9 — char[]
- После Java 9 — byte[] и byte coder
- UTF-16 / Latin1 ('Compact Strings')

```
String name1 = "John Doe";
String name2 = "John Doe";

String name3
        = new String("John Doe");
```



Возврат в String Pool

```
name3 = name3.intern();
```


nexign

Методы String

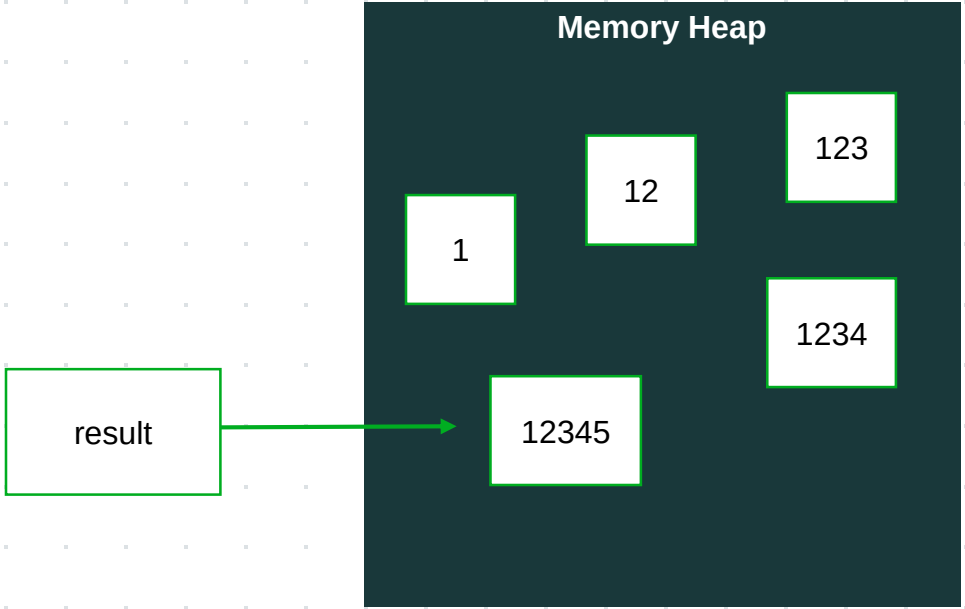


```
String text = "Some Text";
int length() //9
char charAt(int index) //text.charAt(0) - S
int compareTo(String other) //"роза".compareTo("азор":)/ -16
boolean equals(Object other) //text.equals("text") /false
boolean  equalsIgnoreCase(Object other) //text.equals("some text") /true
boolean startWith(String prefix) //text.startWith("S") /true
boolean endsWith(String prefix) //text.endsWith("t") /true
String toLowerCase() / String toUpperCase() //text.toLowerCase()/SOME TEXT
String trim() //"   some text    ".trim()/ "sometext"
String join(CharSequence delimiter, CharSequence... elements)
//String.join("$","Demo","Text"); / Demo$Text
```

Проблемы неизменяемости



```
String result = "";
for (int i = 0; i < 5; i++)
        result += i;
        return result;
```



nexign

StringBuffer & StringBuilder



- Класс **StringBuffer** рекомендуется использовать в тех случаях, когда используются потоки (threads), так как он, в отличие от классов **String** и **StringBuilder**, обеспечивает синхронизацию строк.
- Класс **StringBuilder**, введённый начиная с **JDK 1.5**, полностью ему подобен, но синхронизации не поддерживает. Зато обеспечивает большую скорость работы со строками (что обычно бывает важно только в лексических анализаторах).

String vs StringBuffer vs StringBuilder

Особенности	String	StringBuffer	StringBuilder
Изменяемость	Immutable (нет)	mutable (да)	mutable (да)
Расширяемость	final (нет)	final (нет)	final (нет)
Потокобезопасность	Да, за счет неизменяемости	Да, за счет синхронизации	Нет
Когда использовать	Редкая модификация строк	Частая модификация строк в многопоточной среде	Частая модификация строк в однопоточной среде

nexign

Q&A



nexign

Управление выполнением программы



nexign

nexign

Условные выражения



Условный оператор if

Шаблон

```
if (условие) оператор;
else оператор;
```

```
if (условие)
    оператор;
else if (условие)
    оператор;
else if (условие)
    оператор;
…
else
    оператор;
```

Фигурные скобки рекомендуется ставить, даже когда они необязательны

```
int a, b;
...
if (a < b)
    a = 0;
else
    b = 0;
```

```
int a, b;
...
if (a < b) {
    a = 0;
    ...
} else {
    b = 0;
    ...
}
```


nexign

switch-case



Шаблон

```
switch (выражение) {
   case значение:
        оператор;
        break;
   case значение:
        оператор;
        break;
                  …
   default:
     оператор;
}
```

- Можно писать произвольное число операторов для каждого варианта **case**
- Выход из выполнения последовательности операторов осуществляется с помощью оператора **break**
- Если **break** отсутствует, происходит переход в следующий блок операторов
- указанное выражение должно иметь тип **byte, short, int, char** или **перечислимый тип + String (Java 7+)**

Java 14+ Switch Expression

Expression	Statement

int numLetters = switch(day) {
case MONDAY, FRIDAY, SUNDAY -> 6;
case TUESDAY -> 7;
case THURSDAY, SATURDAY -> 8;
default -> 9;
}
	switch(day) {
	case MONDAY, FRIDAY, SUNDAY:
	numLetters = 6;
	break;
	case TUESDAY:
	logger.info("Tuesday");
	numLetters = 7;
	break;
	case THURSDAY:
	logger.info("Thursday");
	case SATURDAY:
	numLetters = 8;
	break;
	default:
	numLetters = 9;
	}

nexign

Тернарный оператор




```
public static void main(String[] args) {
        Man man = new Man(22);
        String securityAnswer = (man.getAge() >= 18) ? "Проходите!" : "Вход запрещён!";
        System.out.println(securityAnswer);
    }
}
```


nexign

nexign

Циклы



Циклы с условием

Шаблон цикла while

```
while (условие) {
    тело цикла
}
```

```
int i = 10;
while (i > 0){
    i--;
    System.out.println(i);
}
```

Шаблон цикла do…while

```
do {
    тело цикла
} while (условие);
```

```
int i = 10;
do {
    i--;
    System.out.println(i);
} while (i > 0);
```


nexign

Цикл for

Шаблон

```
for (инициализация; условие; итерация) {
      тело цикла
}
```

```
for (int i = 1; i <= 10; i++) {
    System.out.println(i);
}
```

- Каждый из блоков оператора **for** является необязательным, но при этом разделительные **“;”** требуется писать
- **for (;;)** — бесконечный цикл
- Наиболее употребительное использование оператора **for** для перебора значений некоторой переменной, увеличивающихся или уменьшающихся на 1

nexign

Цикл for each

Шаблон

```
for (тип итерационная_переменная : коллекция) {
      тело цикла
}
```

```
int nums[] = {1, 2, 3};
int y = 0;
for (int x : nums){
  y+= x;
  }
```

- Предназначен для циклической обработки коллекций (массивов, списков, деревьев и т. д.)
- В каждом проходе тела цикла извлекается очередной элемент коллекции и запоминается в переменной цикла
- Цикл выполняется до тех пор, пока не будут извлечены все элементы коллекции
- Итерационная переменная доступна "только для чтения".

nexign

Оператор break



- Передает управление
  на следующий за циклом оператор
- Применим ко всем видам циклов

```
for (String s: haystack) {
    if (needle.equals (s)) {
        found = true ;
        break;
    }
}
```

nexign

Оператор continue



- Прерывает текущую итерацию цикла и начинает следующую
- Перед новой итерацией проверяется условие цикла
- Применим ко всем видам циклов

```
for (String s : haystack) {
    if (!needle.equals (s)) {
        continue;
    }
    count++;
}
```

nexign

Q&A



Итоги

- Java - наследник C++
- Есть автоматическое управление памятью
- Код выполняется в JVM, что даёт простор для кастомизации
- Заточен на ООП и работу с бэкендом
- Есть рудименты процедурных языков в виде примитивов
- Для большинства действий уже есть классы/методы
- Есть обратная совместимость, мешающая частому внедрению новых фич



Спасибо!


Aleksandr.Mityasov


@mityasov


Aleksandr.Mityasov@nexign.com

В настоящем документе содержится не предназначенная для публикации конфиденциальная информация и материалы, принадлежащие АО «Нэксайн» или его аффилированным лицам, охраняемые российским и международным законодательством об авторском, патентном праве, праве на секрет производства (ноу-хау), о товарных знаках и иных средствах индивидуализации.
Программное обеспечение АО «Нэксайн» и связанная с ним документация могут распространяться и использоваться только на основании отдельного лицензионного соглашения с АО «Нэксайн».
Любое использование, копирование, воспроизведение, публикация, обнародование, передача, изменение, переработка или перевод настоящего документа и содержащихся в нем информации и материалов, полностью или частично, в любой форме и любыми средствами без предварительного письменного разрешения АО «Нэксайн» строго запрещено.
Если настоящий документ предоставляется с программным обеспечением, лицензированным АО «Нэксайн», информация в нем предоставляется в соответствии с условиями гарантии, предоставляемой с лицензией на программное обеспечение.
Если настоящий документ не предоставляется с лицензионным программным обеспечением, информация в нем предоставляется «как есть» без каких-либо гарантий.
В обоих случаях АО «Нэксайн» не предоставляет никаких иных гарантий в отношении настоящего документа и содержащихся в нем информации и материалов, явных или подразумеваемых, включая, помимо прочего, подразумеваемые гарантии в отношении их качества, коммерческой пригодности, пригодности для определенной цели, ненарушения прав третьих лиц.
АО «Нэксайн» или его аффилированные лица ни при каких обстоятельствах не несут ответственность за любые прямые или косвенные убытки, случайные расходы или штрафы, возникшие в результате использования или невозможности использования материалов и/или информации, содержащихся в настоящем документе.
АО «Нэксайн» принимает надлежащие меры для обеспечения качества информации и материалов, содержащихся в настоящем документе. Однако документ может содержать технические неточности или опечатки, отсутствие ошибок не гарантируется.
Настоящий документ может быть изменен или дополнен без какого-либо уведомления. АО «Нэксайн» не обязуется обновлять настоящий документ на основе изменений в продуктах или услугах, как собственных, так и третьих лиц. АО «Нэксайн» может вносить улучшения или изменения в продукты или услуги, описанные в настоящем документе, в любое время без предварительного уведомления.

nexign

Классы
Объекты
Методы



nexign

План урока

- Общая информация об ООП
- Класс. Структура класса
- Поля
- Конструкторы
- Методы
- Наследование
- Методы класса Object

nexign

nexign

ООП. Общая информация



Парадигмы программирования

Императивная парадигма:

- исходный код программная инструкция;
- инструкции должны выполняться последовательно;
- данные результата выполнении предыдущих инструкций, могут читаться из памяти последующими инструкциями;
- данные, полученные при выполнении инструкции, могут записываться в память.



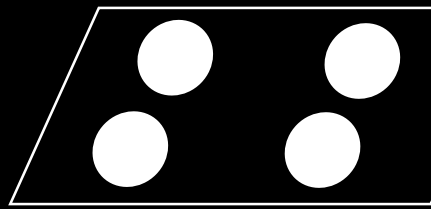
nexign

Объектно-ориентированное программирование — парадигма программирования, в которой программа строится из взаимодействующих объектов









Принципы ООП

- **Инкапсуляция:**

  объединение данных и методов их обработки в одну сущность, приводящее к сокрытию реализации класса и отделению его внутреннего представления от внешних пользователей

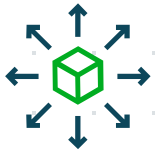
- **Наследование:**

  отношение между классами, при котором один класс использует структуру или поведение другого класса

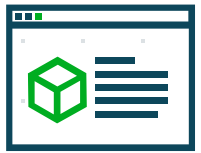
- **Полиморфизм:**

  способность объекта соответствовать во время выполнения двум или более возможным типам. С другой стороны, способность различных объектов предлагать свои реализации одного и того-же интерфейса

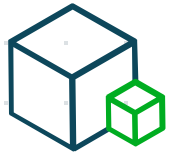
полиморфизм



инкапсуляция



наследование



nexign

nexign

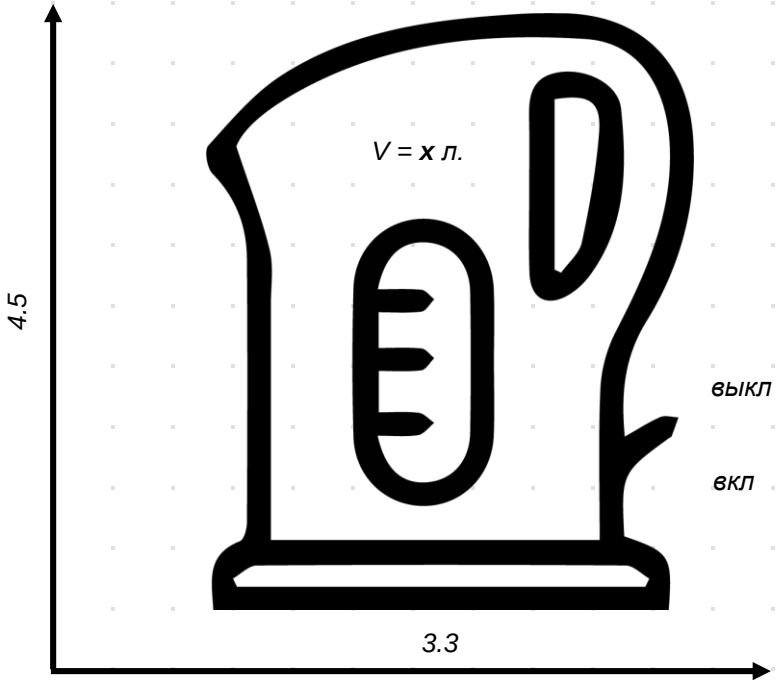
Класс. Структура класса



Класс

- Класс представляет собой абстрактное описание структуры реальных объектов
- Объекты создаются в оперативной памяти машины на основе такого описания, которое представляет собой перечисление свойств, имеющихся у объектов и методов, которые объектам доступны
- Объект представляет собой совокупность конкретных значений свойств и методов, работающих с этими свойствами
- Вся логика работы java-приложений построена на взаимоотношении классов и объектов





nexign

Структура класса

```
class ClassName
{
   field1
   field2
   . . .
   constructor1
   constructor2
   . . .
   method1
   method2
   . . .
}
```

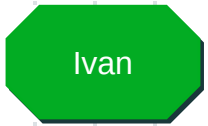
```
package org.megacompany.staff;
    class Employee {
        // instance fields
        private String name;
        private double salary;
        private LocalDate hireDay;
        // constructor
        public Employee(String n, double s, int year, int month, int day) {
            name = n;
            salary = s;
            hireDay = LocalDate.of(year, month, day);
        }
        // a method
        public String getName() {
            return name;
        }
        // more methods
         . . .
```


nexign

Рождение, жизнь и смерть объекта

Создание объекта

```
new Employee("Ivan");
```



nexign

Рождение, жизнь и смерть объекта

Присвоение ссылки

```
Employee designer = new Employee("Ivan");
```

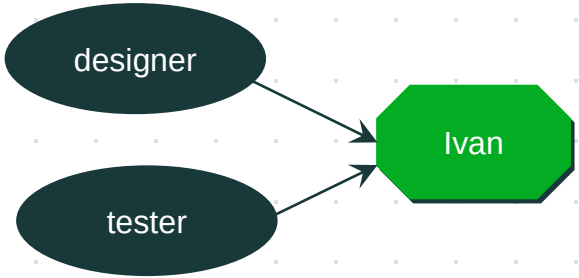


nexign

Рождение, жизнь и смерть объекта

Присвоение ссылки

```
Employee tester = designer;
```

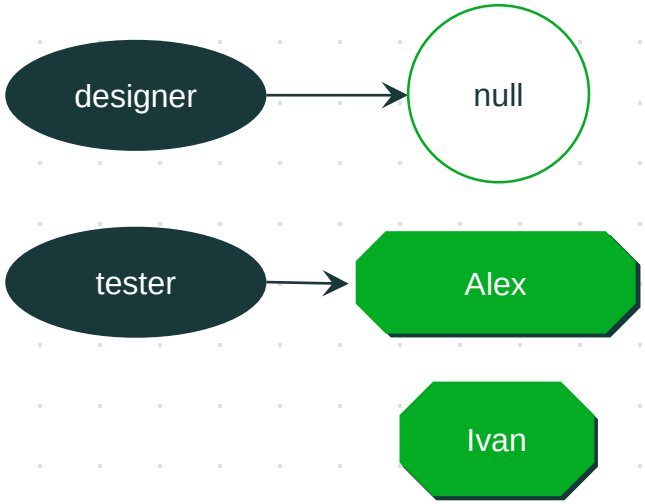


nexign

Рождение, жизнь и смерть объекта

Потеря ссылки

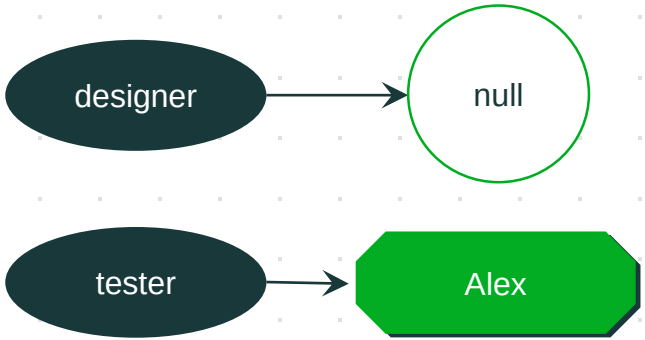
```
designer = null;
tester = new Employee("Alex");
```



nexign

Рождение, жизнь и смерть объекта

Сборка мусора



nexign

var Java 10+

local variable type inference



```
TheLongestNameYouCanEverImagine theLongestNameYouCanEverImagine = new TheLongestNameYouCanEverImagine();
```



```
var theLongestNameYouCanEverImagine = new TheLongestNameYouCanEverImagine();
```


nexign

var Java 10+

- Ключевое слово var можно использовать при объявлении локальных переменных в конструкторах, блоках инициализации, методах.
- var нужно инициализировать сразу после именования — в одном операторе. При этом можно переносить такое объявление переменной на разные строки.
- Объявлять сразу несколько переменных с помощью var в одном операторе нельзя.
- Инициализировать var-переменную значением null без явного указания типа тоже нельзя.
- Значение var-переменной в дальнейшем меняться может, а вот тип — нет.
- var допустимо использовать в качестве названия переменной, но нельзя так именовать тип: класс, интерфейс или перечисление.

nexign

nexign

Структура класса. Поля



Поля

- В отличие от локальных переменных, поля можно не инициализировать явно.
- В этом случае примитивные типы получают значение по умолчанию **(0, false),** а поля со ссылками — значение **null**.
- Проинициализировать поле по месту его определения не возбраняется: **int a = 42** или даже **int a = getValue().**

```
class Employee {
        // instance fields
        private String name;
        private double salary;
        private LocalDate hireDay;
}
```


nexign

Модификаторы доступа

private



доступен только методам данного класса

public



доступен всем желающим

protected



доступен наследникам и всем классам в пакете

default



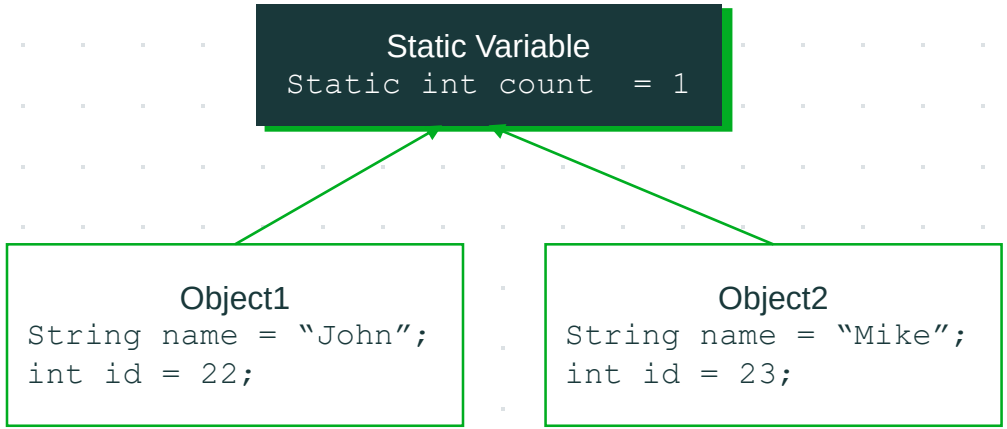
доступен всем классам в том же пакете

nexign

Статические поля в JVM

Модификатор **static** означает, что поле принадлежит классу, а не создаваемому от него экземпляру.

Иными словами, одна переменная связана со всеми объектами, созданными по этому классу.



nexign

Статические поля

- Статические переменные могут быть созданы только, как переменные класса. Они не могут быть локальными переменными

- К статическим полям класса можно получить доступ без создания объекта, используя имя класса (ссылка на объект не нужна)

```
public class Main {
    public static void main(String[] args) {
        static String wrongVar =
        "Modifier 'static' not allowed here";
        System.out.println(wrongVar);
    }
}

public static int counts = 100;
System.out.println("Результат: " + Car.counts);
```


nexign

Неизменяемые (inmutable) поля



```
static final double GRAVITATIONAL_CONSTANT = 9.81;

        double potentialEnergy(double mass, double height) {
        return mass * height * GRAVITATIONAL_CONSTANT;
        }
```

Константы

статические финальные поля, содержимое которых неизменно. Это относится к примитивам, String, неизменяемым типам и неизменяемым коллекциям неизменяемых типов. Если состояние объекта может измениться, он не является константой.

nexign

Структура класса. Конструкторы



Конструкторы

Конструктор класса

именованный блок инструкций в составе класса, вызываемый каждый раз при создании нового объекта

- Имя конструктора всегда совпадает с именем класса
- Класс может иметь несколько конструкторов
- Конструктор не имеет возвращаемого значения

```
public class Person {
//public-конструктор без аргументов
    public Person() {
    ....
    }
//package-private конструктор с
аргументом
    Person(String  name) {
    ....
    }
}
```


nexign

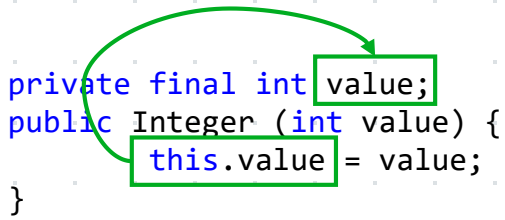
Конструкторы

- Если в класс явным образом не включен конструктор, он снабжается конструктором по умолчанию, не имеющим параметров
- Если конструктор явным образом включен в класс, то конструктор по умолчанию в него не добавляется
- Если в родительском классе нет конструктора без аргументов,конструктор по умолчанию не создаётся
- Если класс имеет несколько конструкторов, они должны явно отличаться по количеству и типу передаваемых параметров
- Конструктор не обязан быть публичным.

nexign

Поле this()

- Ключевое слово **this** применяется тогда, когда из метода экземпляра класса необходимо сослаться на поле или метод того же самого экземпляра
- Как правило, это слово используется для того, чтобы избежать конфликта при совпадении имен параметров метода и полей объекта




```
public class Integer {
    private final int value ;
    public Integer (int value) {
        this.value = value;
    }
    registerMe(Registrator r) {
        //нужна ссылка на себя
        r.register(this);
    }
}
```


nexign

Пример

```
public class Employee {
        public String name;
        public int age;
        public Employee(String name, int age) {
            this.name = name;
            this.age = age;
        }
            public Employee() {
            }
            public void sayMyName() {
                System.out.println(this.name);
            }
        public static void main(String[] args) {
            Employee e1 = new Employee();
            Employee e2 = new Employee("Heisenberg", 22);
            e1.sayMyName();
            e2.sayMyName();
        }
    }
```


nexign

Перегрузка конструкторов



```
class Person {
    Person(String  name){
    ...
    }

    Person(){
        this("unknown");
    }
}
```


nexign

Методы

Методы – это алгоритмы, обрабатывающие данные(состояния)

Шаблон

Модификаторы Тип Имя(*список параметров*){
Тело // return *выражение;*
}

- Если в методе в какой-либо из ветвей не использовать оператор return, будет выдана ошибка компиляции
- Если тип возвращаемого значения void, оператор return не нужен

```
public class Hello {
    private String s;

    public void getName(String s) {
        System.out.println("Hello" + s);
    }
}
```


nexign

nexign

Структура класса. Методы



Параметры методов

- Метод может иметь ноль или более параметров
- Каждый параметр имеет свой тип и имя
- Внутри тела метода параметры используются как локальные переменные
- Тип параметра может быть как примитивным, так и ссылочным

```
void increment(int tmpInt) {
    tmpInt++;
}

public static void main(String[] args)
{
    int myInt = 42;
    Hello obj = new Hello();
    obj.increment(myInt);
    System.out.println(myInt);
}
```


nexign

Параметры переменной длины

```
static void vaTest(int ... v){
    System.out.println("Number of args:" +v.length);
    System.out.println("Contents:");
    for (int x : v) {
        System.out.println(x + " ");
    }
}
```

Варианты вызова метода vaTest

```
vaTest(10);  vaTest(1,2,3);  vaTest();
```


nexign

Параметры переменной длины

- В список параметров наряду с параметрами переменной могут быть включены «обычные» параметры
- Такие параметры обязательно надо указывать при вызове метода
- Параметр, содержащий переменное число аргументов, должен быть последним в списке параметров метода

  (*int doIt(int a, int b, double c, int… vals) { … })*
- В списке параметров метода может быть только один параметр переменной длины

  (*int doIt(int a, int b, double c,* ***int… vals, double… moreVals****){ … }*)

nexign

Перегрузка методов (Overloading)

Сигнатура метода

Имя метода в сочетании с числом, типом и порядком следования его параметров

- Тип возвращаемого значения и имена параметров в сигнатуру не входят
- Методы, имеющие разные сигнатуры, считаются различными
- Если сигнатуры методов совпадают, перегрузка запрещена

```
static int addNumbers(int i1, int i2){
    return i1 + i2;
}

static int addNumbers(double d1, int i2){
    return (int)d1 + i2;
}

static int addNumbers(int i1,double d2){
    return i1 + (int)d2;
}

static int addNumbers(int i1, int i2, int i3){
    return i1 + i2 + i3;
}
```


nexign

Статические методы

Статическим методам доступны только статические переменные и вызовы других статических методов

```
class Employee {

    private static int nextId = 1;
    private int id;
    . . .
    public static int getNextId() {
        return nextId; // returns static field
    }
}


        . . .
   Employee.nextId() //имя класса вместо объекта
```


nexign

Причины использовать статические методы

- Для доступа / управления статическими переменными и другими статическими методами, которые не зависят от объектов
- Для служебных, вспомогательных классов и интерфейсов, поскольку не требуют создания объектов и соответственно, обеспечивают большую производительность
- Когда методу требуется доступ лишь к статическим полям класса

nexign

nexign

Наследование



Механизм наследования

- Наследование позволяет строить новые классы на основе одного первоначального
- В новые классы добавляются новые поля данных и методы
- Первоначальный класс называется родителем, новые классы – его потомками
- От потомков также можно наследовать, получая очередных потомков
- Набор классов, связанных отношением наследования, называется иерархией классов

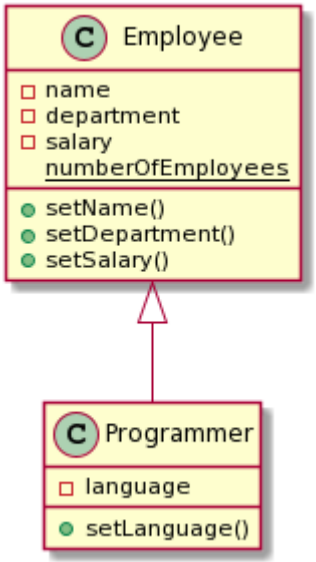
nexign

Пример наследования

Модификатор **extends** указывает на расширение класса

```
class Programmer extends Employee {
    //Переменные экземпляра класса Programmer
    private String language; // Язык программирования
}
…

        Programmer bill = new Programmer();
        bill.setName("Bill");
        bill.setDepartment("Development");
        bill.setSalary(new BigDecimal(55000));
        bill.setLanguage("Java");
```



nexign

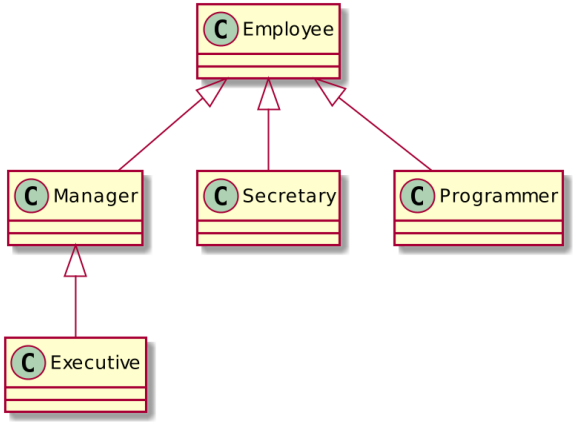
Наследование

- В языке **Java** поддерживается только простое наследование:

  класс может иметь одного и только одного предка

- Все классы в **Java** являются потомками класса **Object**

- Явным образом указывать наследование от **Object** не нужно



nexign

Тип ссылки и тип объекта



Проверить соответствие объекта какому-либо типу можно с помощью оператора **instanceof:**

if (e **instanceOf** Programmer){ … }

Или с помощью метода getClass()

nexign

Связывание

Раннее связывание (compile time)

- Overloading – поиск подходящей сигнатуры в зависимости от списка параметров

Позднее связывание (runtime)

- Overriding – поиск подходящей реализации по реальному типу объекта. При использовании рекомендуется прописывать соответствующую аннотацию **@Override**

```
class Super {
    int myInt = 42;

    String test() {
        return "Super";
    }
}
```

Класс-родитель

```
class Sub extends Super {
    int myInt = 11;

    String test(){
        return "Sub";
    }
}
```

Класс-потомок

nexign

Перекрытие методов и переменных

Класс-родитель

```
class Super {
    int myInt = 42;

    String test() {
        return "Super";
    }
}
```

Класс-потомок

```
class Sub extends Super {
    int myInt = 11;
    @Override
    String test(){
        return "Sub";
    }
}
```

```
Super mySuper = new Super();
Sub mySub = new Sub();
System.out.println(mySuper.test());  //   "Super"
System.out.println(mySuper.myInt); //    42
System.out.println(mySub.test());      //   "Sub"
System.out.println(mySub.myInt);     //    11

Super superSub = new Sub();
System.out.println(superSub.test()); //   "Sub"
System.out.println(superSub.myInt);//    42
```


nexign

Модификатор final

- **final** class MyClass {...} нельзя создать класс-наследник
- **final** void myMethod() {...} нельзя переопределить метод в классе-потомке

nexign

Ключевое слово “super”

- Для вызова конструктора супер класса:

```
super(Список аргументов);
```

- Для обращения к члену супер класса, скрываемому членом подкласса:

```
super.member;
```

```
private static class Executive {
    public Executive() {
        System.out.println("Empty");
    }
    ...
private static class Programmer extends Executive
{
    public Programmer() {
        super();
    }
    ...

private static class Employee extends Programmer
{
    public Employee() {
        super.print();
    }
```

nexign

sealed-типы (Java 15+)

Наследоваться можно, но только тем, кому разрешено:

```
public sealed class Pet
        permits
                //никакие другие не могут наследоваться от него
                Cat, Dog, Fish {
}

public final Cat {...}

public sealed Dog permits Hound, Terrier, Toy {...}

public non-sealed Fish {...}
```


nexign

this & super

- Вызов метода **super()** должен быть всегда в первом операторе, выполняемом в теле конструктора подкласса
- Если в конструкторе наследника нет явного вызова **super(), JVM** сама его подставляет первой строкой
- Из этого следует, что супер класс должен иметь конструктор без параметров, иначе возникнет ошибка компиляции
- переменная **this** ссылается на текущий экземпляр класса, в котором она используется, тогда как **super** — на текущий экземпляр родительского класса.

nexign

nexign

Методы класса Object



Методы класса Object



В Java все классы являются потомками класса Object. Из него любой java-класс наследует ряд полезных на практике методов:

- **public boolean equals(Object obj)** – возвращает true в случае, когда равны значения объекта, из которого вызывается метод, и объекта, передаваемого через ссылку obj в списке параметров. В классе **Object** равенство рассматривается как равенство ссылок и эквивалентно оператору сравнения “==”.
- **public int hashCode()** – выдаёт *хэш-код* объекта. В **Java** хэш-код заменяет адрес объекта в тех случаях, когда для каких-либо целей надо хранить таблицы адресов объектов.
- **protected Object clone() throws CloneNotSupportedException** – метод занимается копированием объекта и возвращает ссылку на созданный клон (дубликат) объекта.
- **public String toString() -** Возвращает строковое представление объекта

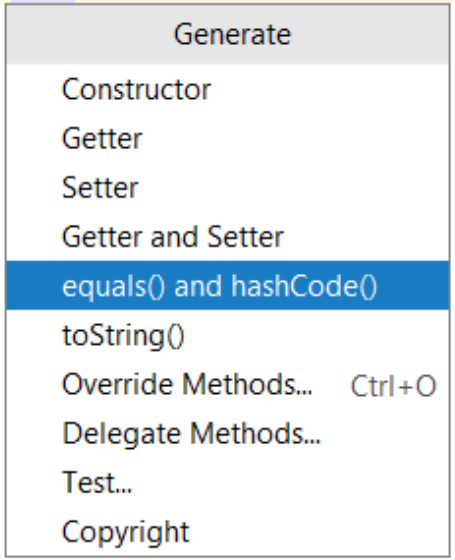
nexign

Генерация hashCode и equals

IntelliJ Idea:

Alt + Insert

```
public class Person {
    private String name;
    private int age;
}
```



nexign

“Джентельменский набор” для класса

```
public class Point {
    private final int x;
    private final int y;

    /*Подождите! Нам нужны:
     * конструктор
     * getX() и getY()
     * equals и hashCode
     * toString
     * 40 строчек кода из ничего!
     */

    public double distance(Point other) {
    ...
    }
}
```


nexign

Records (Java 14+)

```
record Person(String name, int age){

static int minAge;
static{
        minAge = 18;
        System.out.println("Static initializer");
      }
   }
}
```

- private final поля
- конструктор
- доступ через x() и y()
- equals, hashCode и toString
- невозможно наследование от класса и от рекорда
- возможна реализация интерфейсов

nexign

Практика

- Добавить классу **Person** методы **talk(), move(),** которые выводят соответствующие сообщения
- Удалить поле **age**, а вычисления перенести в статический метод, получающий в качестве параметра **Person**
- Создать класс **Worker**, наследующийся от **Person**
- Добавить ему поле **BigDecimal salary** и создать конструктор
- Добавить метод **work()**, который выводит информацию, о том,

что сотрудник с именем **%username%** работает + вызывает метод **talk()**



nexign

Q&A



nexign

Вложенные классы

Абстрактные классы

Интерфейсы

Перечисления



nexign

План урока

- Вложенные классы
- Абстрактные классы
- Интерфейсы
- Перечисления

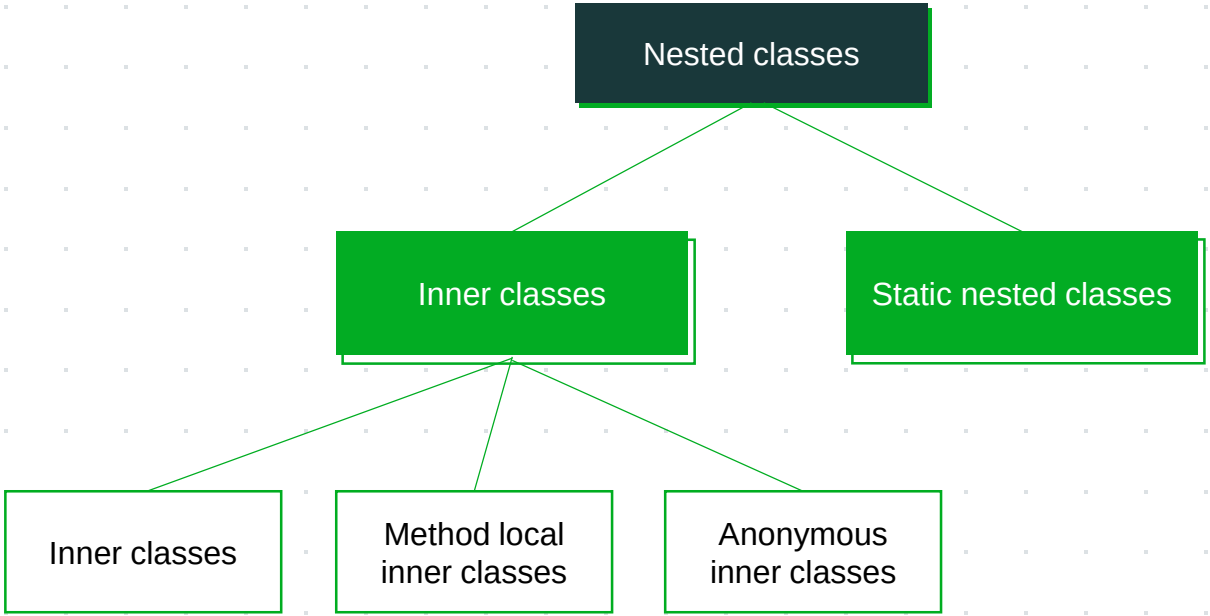
nexign

nexign

Вложенные классы



Вложенные классы (nested classes)



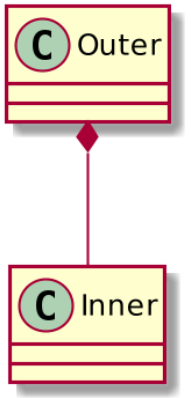
nexign

Вложенный класс (Inner class)

```
class Outer {
    int field = 42;
    class Inner {
        public void show() {
            //есть доступ к состоянию внешнего класса!
            System.out.println(field);
            //печатает 42
        }
    }
    void initInner(){
        //инициализация вложенного класса внутри
        new Inner();
    }
}

    //инициализация вложенного класса извне
    Outer.Inner in = new Outer().new Inner();
```

Inner классы не могут содержать статических методов.



Каждый экземпляр Inner имеет неявную ссылку на Outer

nexign

Локальные вложенные классы (method local)

```
class Outer {
    void outerMethod() {
        //final (или effectively final)
        final int x = 98;
        System.out.println("inside outerMethod");
        class Inner {
            void innerMethod() {
                System.out.println("x = " + x);
            }
        }
        Inner y = new Inner();
        y.innerMethod();
    }
}
```

- Объект локального класса не может создаваться за пределами метода или блока, в котором его объявили.
- Не может использовать локальные переменные внешнего метода до тех пор, пока они не будут объявлены как **final**
- Не может быть помечен как **private**, **protected**, **static**, но может быть помечен как **abstract** и **final**, но не оба одновременно

nexign

Анонимные классы (Anonymous)



- анонимные классы не имеют названия
- декларируются внутри методов основного класса и могут быть использованы только внутри этих методов
- должен наследовать существующий класс или реализовывать существующий интерфейс
- не могут содержать определение статических полей, методов и классов (кроме констант), но могут их наследовать

nexign

Применение анонимных классов

- Тело класса является очень коротким;
- Нужен только один экземпляр класса;
- Класс используется в месте его создания или сразу после него;
- Имя класса не важно и не облегчает понимание кода.

```
. . .
button.onMouseClick(new EventListener(){
void onClick(Event e) {
//здесь у нас доступ ко всем внешним полям
//и effectively final-переменным
}
});
```


nexign

Статические вложенные классы (static nested)

- Статические вложенные классы могут получить доступ ко всем статическим членам внешнего класса, включая приватные
- Спецификация Java не позволяет объявлять класс верхнего уровня статическим. Только классы внутри других классов могут быть статическими

nexign

Статические вложенные классы (static nested)

```
class Outer {
    private static void outerMethod() {
        System.out.println("inside outerMethod");
    }
    static class Inner {
        public static void main(String[] args) {
            System.out.println("inside inner class Method");
            outerMethod();
        }
    }
}
. . .
        Outer.Inner x = new Outer.Inner();
```


nexign

Пример реализации – паттерн «singleton»

```
public class Singleton  {
    private Singleton() {}

    private static class SingletonHolder {
        private static final Singleton INSTANCE = new Singleton();
    }

    public static Singleton getInstance() {
        return SingletonHolder.INSTANCE;
    }
}
}
```

Singleton

это порождающий паттерн, который гарантирует существование только одного объекта определённого класса, а также позволяет достучаться до этого объекта из любого места программы

nexign

nexign

Абстрактные классы



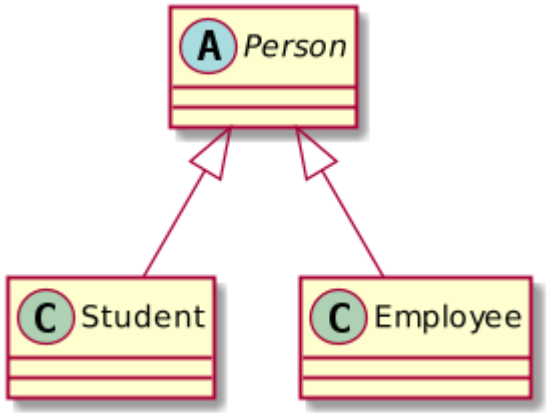
Абстрактные классы

- Класс, в котором хотя бы один из методов не реализован, должен быть помечен как **abstract**
- Нереализованные методы в классе возникают двумя способами:
  - явно объявлены как **abstract**
  - унаследованы из абстрактных классов или интерфейсов и не переопределены.
- Абстрактные классы нельзя инстанцировать через **new**.
  - **new Person("John Doe");** — ошибка компиляции: 'Person is abstract, cannot be instantiated'.

nexign

Абстрактные классы

```
public abstract class Person
{
    public Person(String name) {
        this.name = name;
    }
    public String getName() {
        return name;
    }
    public abstract String getDescription();
}
```



nexign

nexign

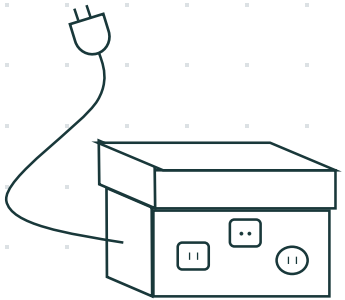
Интерфейсы



Интерфейсы

- **Интерфейс** – это набор абстрактных методов и статических констант
- Синтаксически определение интерфейса похоже на определение **полностью** абстрактного класса, содержащего заголовки методов и **static final** поля
- Для реализации интерфейса класс должен создать полный набор методов, определенных интерфейсом. Однако каждый класс волен сам определять детали своей реализации.
- Создавать объект типа «интерфейс» нельзя, но можно использовать в наследовании
- Как только интерфейс определен, то реализовать его может любое число классов. И наоборот: один класс может реализовать любое число интерфейсов.





nexign

Интерфейсы

```
//его нельзя инстацировать!
public interface Prism
{
    //это --- final-переменная!
    double PI = 3.14;

    //это --- публичные абстрактные методы!
    double getArea();
    double getHeight();

    //этот метод может вызывать другие и читать
final-поля
    default double getVolume() {
        return getArea() * getHeight();
    }
}
```


nexign

Реализация интерфейса

Если какой-то из методов интерфейса не будет переопределён, класс нужно пометить как abstract.

```
puplic class Parallelepiped implements Prism {
    private double a;
    private double b;
    private double h;
    @Override
    public double getArea() {
        return a * b;
    }

    @Override
    public double getHeight() {
        return h;
    }
}
```


nexign

Абстрактный класс

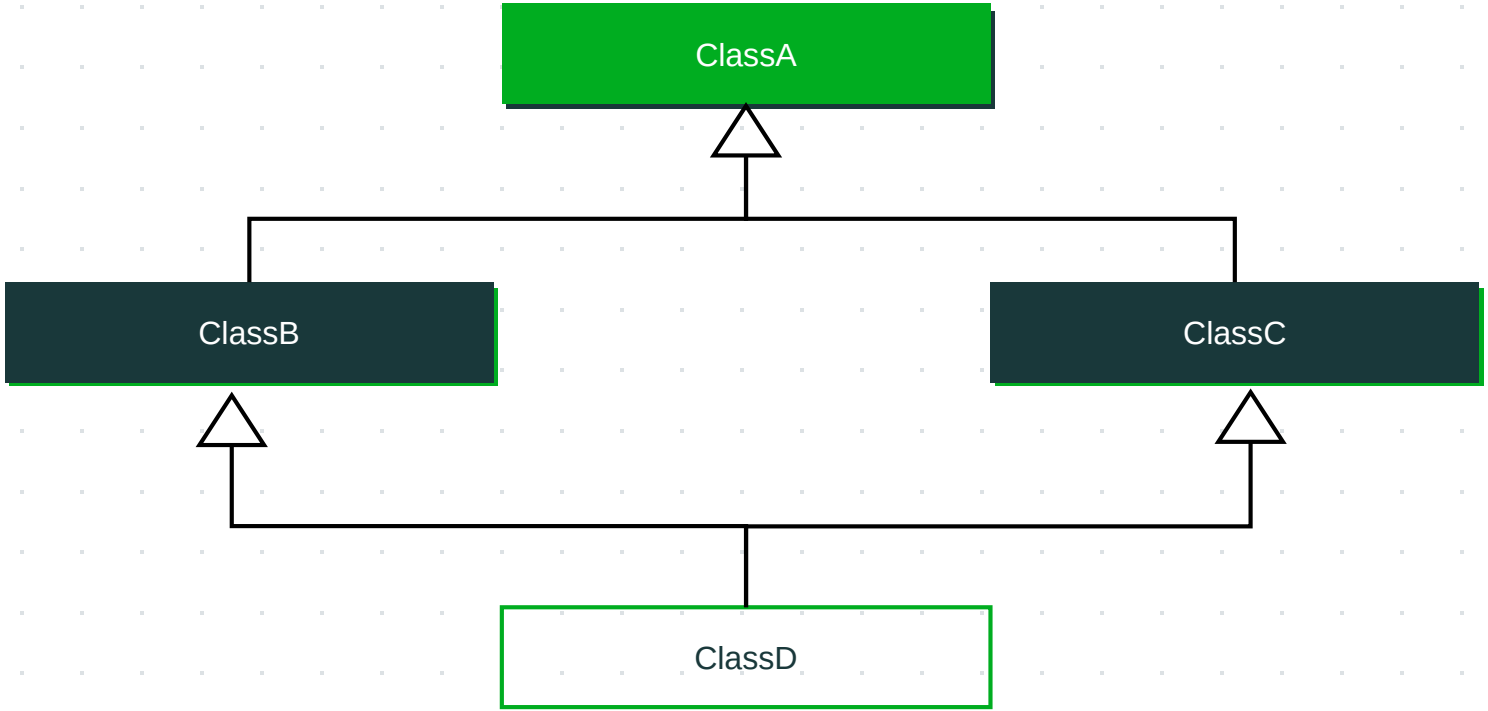
- Описывает концептуальный объект
- Abstract
- Extends
- Наследуется только один класс
- Есть конструктор
- Переменные с любыми модификаторами
- Любые модификаторы методов
- все методы кроме абстрактных должны иметь реализацию
- Может наследовать обычные и абстрактные классы, а так же реализовывать интерфейсы

Интерфейс

- Описывает поведение
- Interface
- Implements
- Реализуется множество интерфейсов
- -
- Все переменные неявно public static final
- Для методов только public и private(Java 9+ *)
- Default –методы должны иметь реализацию в интерфейсе и могут быть переопределены в имплементации
- Может наследовать множество других интерфейсов

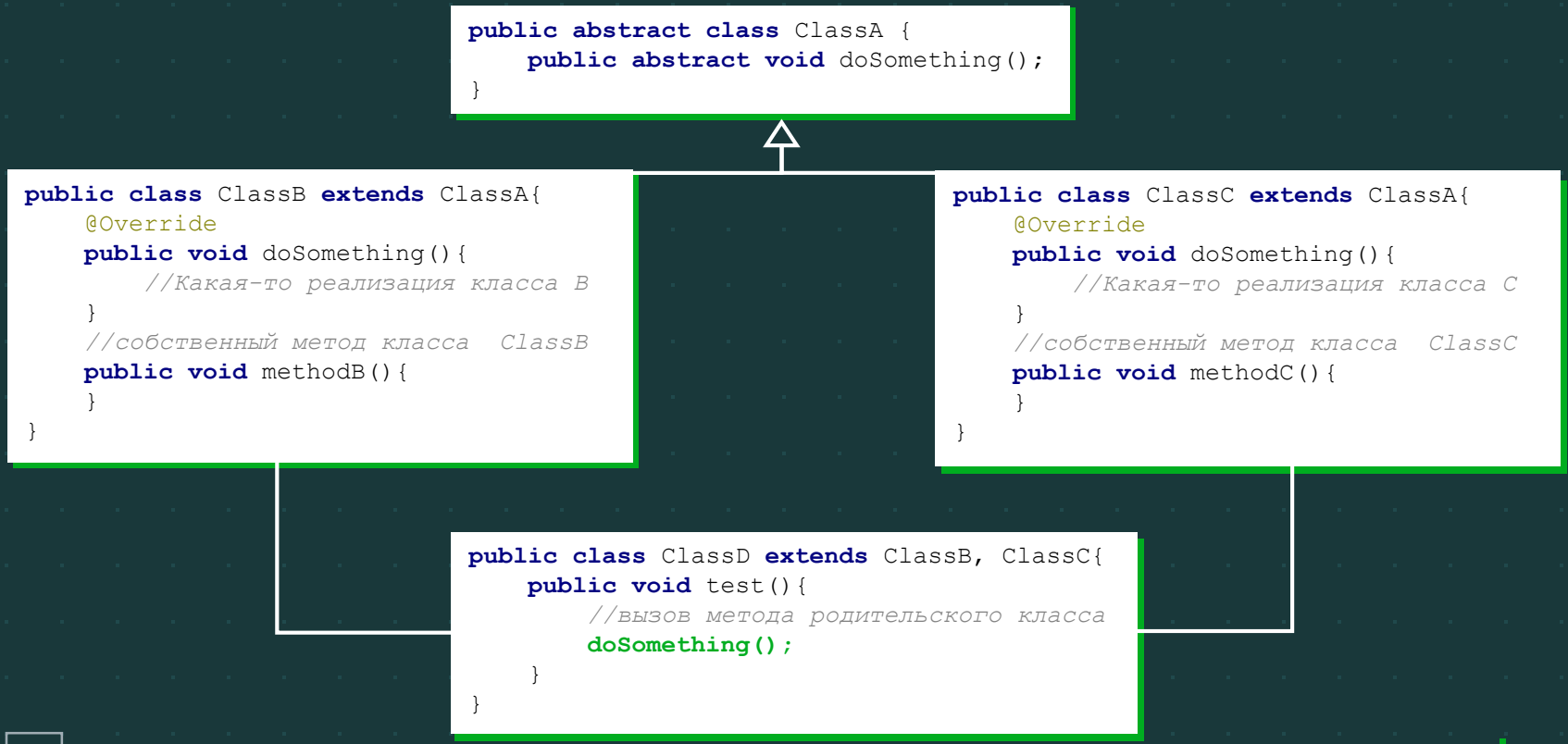
nexign

Множественное наследование



nexign

Множественное наследование



nexign

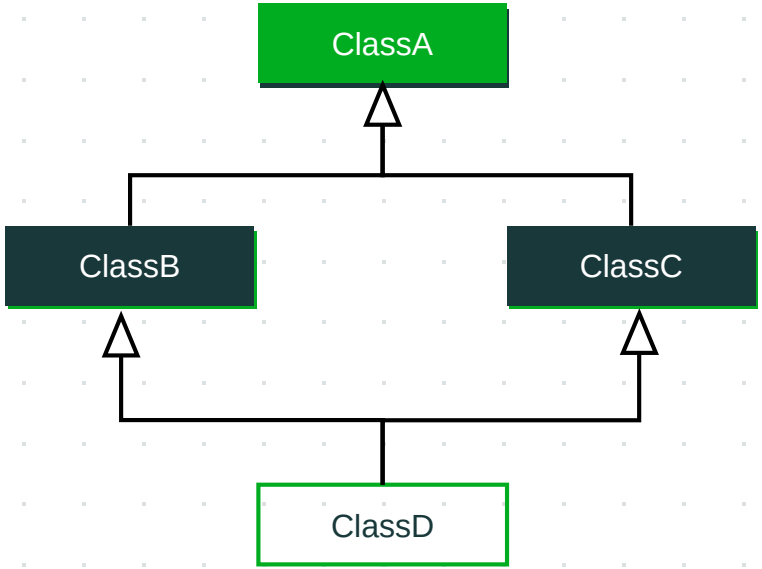
Ромбовидное наследование

#1

```
interface ClassA { . . . }
interface ClassB extends ClassA { . . . }
class ClassC implements ClassA { . . . }
class ClassD extends ClassC implements ClassB
{ . . . }
```

#2

```
interface ClassA { . . . }
interface ClassB extends ClassA { . . . }
interface ClassC extends ClassA { . . . }
class ClassD implements ClassB, ClassC { . . . }
```



nexign

nexign

Перечисления



Перечисления (Enum)



- Перечислимый тип – это список именованных констант
- Метод **values()** возвращает список констант перечислимого типа.
- Метод **valueOf()** возвращает константу перечислимого типа по значению строки-параметра.
- **ENUM_VALUE.name()** – возвращает имя **(=toString())**
- **ENUM_VALUE.ordinal()** – возвращает порядковый номер
- Можно определять конструкторы, добавлять поля и методы, реализовывать интерфейсы.

Q&A



nexign

IO Classes

Exceptions

Logging



nexign

nexign

Классы ввода-вывода



Типы потоков данных

В **Java** существует 2 типа потоков данных:

- Символьные потоки (**text-streams**, последовательности 16-битовых символов **Unicode**), содержащие символы.



- Байтовые потоки (**binary-streams**), содержащие восьми битную информацию.



Классы разделяются также по направлению потоков:

- Потоки ввода (*input*)
- Потоки вывода (*output*)

nexign

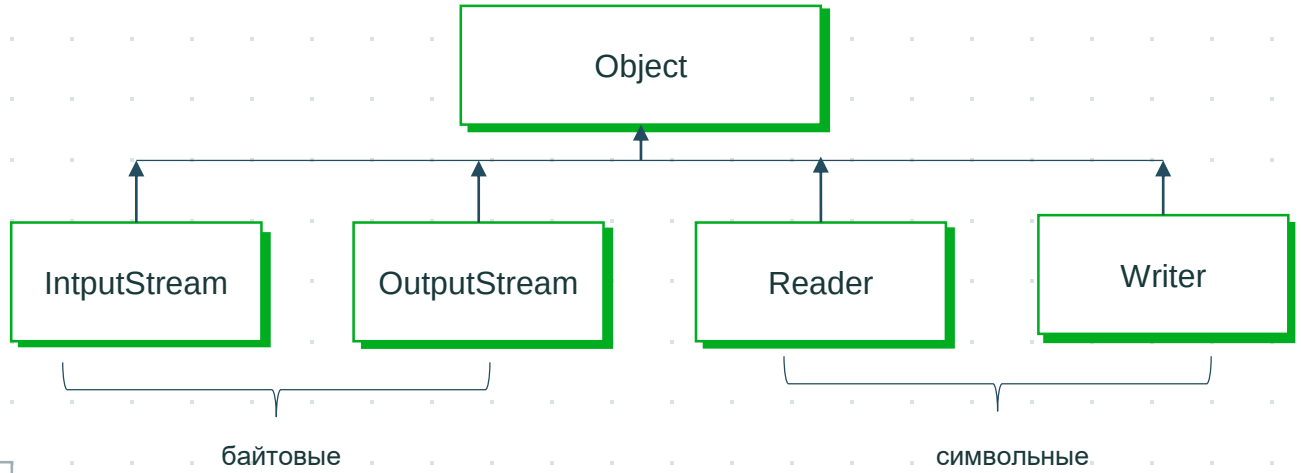
IO Classes

Общая схема работы с потоками

1.Создать потоковый объект и ассоциировать его с файлом на диске.

2.Пока есть информация, читать/писать очередные данные в/из потока.

3.Закрыть поток.



nexign

Высвобождение ресурсов

```
InputStream f =
    new FileInputStream("/etc/passwd");
int b = f.read();
f.close();
```

```
InputStream f =
    new FileInputStream("/etc/passwd");
try {
    int b = f.read();
}
finally {
    f.close();
}
```


nexign

Высвобождение ресурсов

```
InputStream in =
     new FileInputStream("/etc/passwd");
OutputStream out =
     new FileOutputStream("/etc/passwd.bak");
try {
    in.transferTo(out);
}
finally {
    out.close();
    in.close();
}
```

```
InputStream in =
      new FileInputStream("/etc/passwd");
try {
    OutputStream out =
          new
FileOutputStream("/etc/passwd.bak");
    try {
        in.transferTo(out);
    } finally {
        out.close();
    }
}
finally {
    in.close();
}
```


nexign

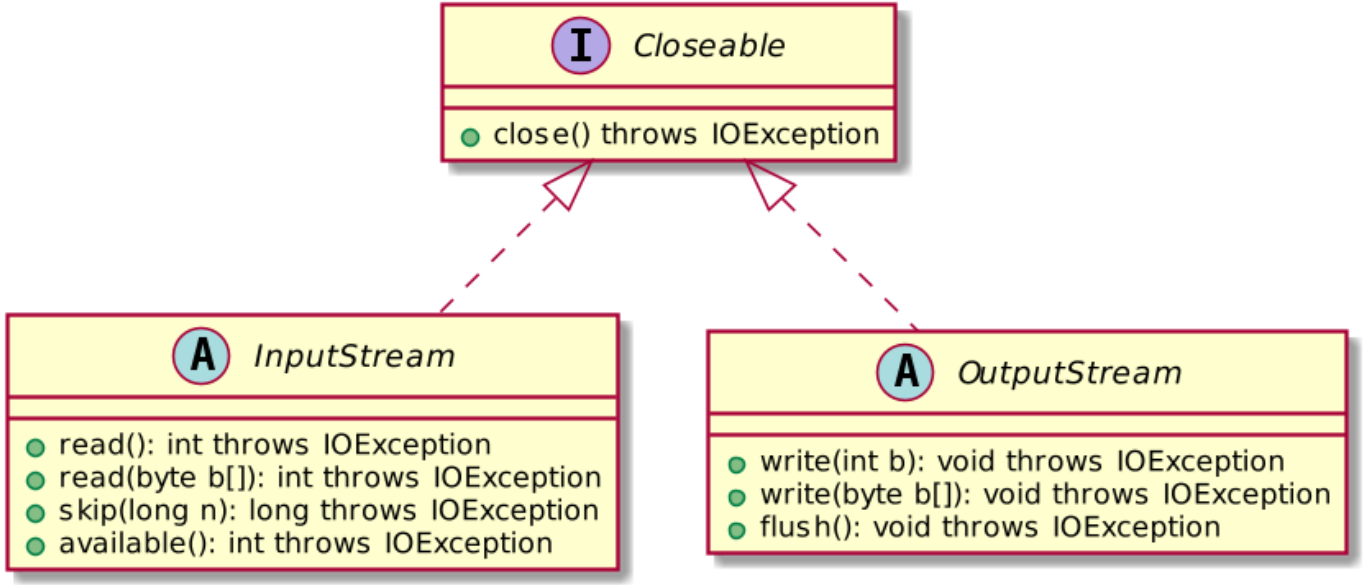
try-with-resources

```
public class TryWithResourcesClose {
    public static void main(String[] args) throws IOException {
        try (InputStream in = new FileInputStream("input-file.txt");
             OutputStream out = new FileOutputStream("output-file.txt")) {
            in.transferTo(out);
        }
    }
}
```

- Метод **close()** будет вызван автоматически, как в **finally**
- Можно перечислить сразу несколько ресурсов
- Ресурсы должны реализовать интерфейс **java.lang.AutoCloseable**
- Добавлен в **Java 7**

nexign

Byte streams



nexign

Реализации Input Stream

- **BufferedInputStream** – класс, который описывает буферизованные поток ввода;
- **ByteArrayInputStream** – класс, который описывает поток ввода, читающий байты из массива;
- **DataInputStream** –класс, который реализует методы для чтения данных стандартных типов, определенных в Java (int, double, float и т.д.);
- **FileInputStream** – класс, который реализует поток ввода, который читает данные из файла;
- **ObjectInputStream** – класс, реализующий поток ввода объектов;

nexign

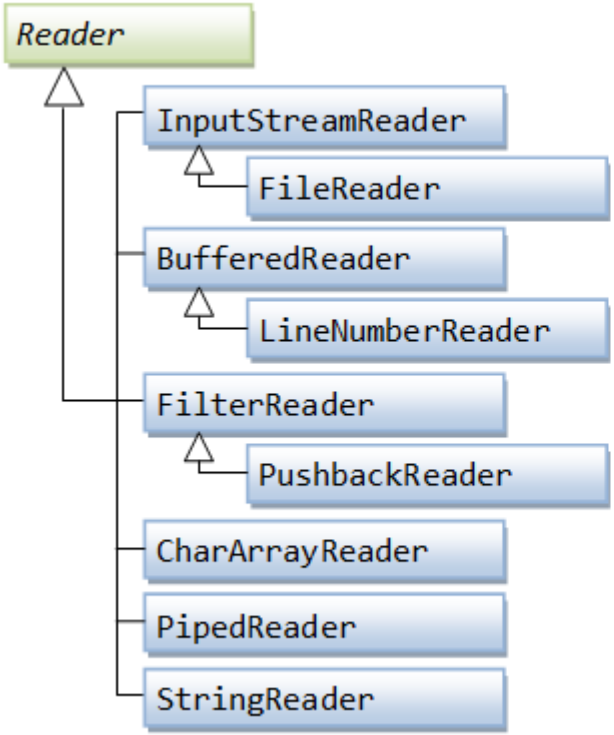
Реализации Output Stream

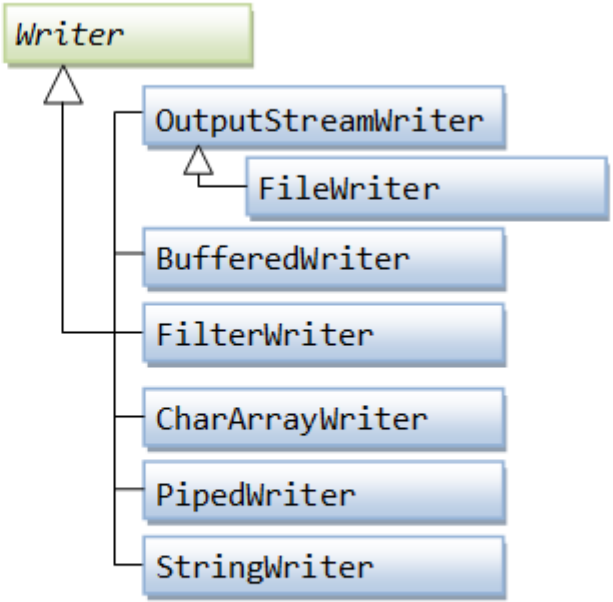


- **BufferedOutputStream** – класс, который имплементирует буферизованный поток вывода;
- **ByteArrayOutputStream** – класс, который реализует поток вывода, записывающий байты в массив;
- **DataOutputStream** – класс, который реализует поток вывода, содержащий методы для чтения данных стандартных типов, определенных в **Java (int, float, double и т.п.);**
- **FileOutputStream** – класс, который соответствует потоку вывода записывающему данные в файл;
- **ObjectOutputStream** – класс, соответствующий потоку вывода объектов;
- **PrintStream** – класс, который представляет собой поток вывода, содержащий методы **print()** и **println().**

nexign

Работа с текстом





nexign

Реализации Reader

- **BufferedReader** - буферизированный входной символьный поток; увеличивает производительность за счёт буферизации ввода;
- **CharArrayReader** - входной поток чтения символьного массива;
- **FileReader** - входной поток чтения содержимого файла; в конструкторе класса нужно указать либо путь к файлу, либо объект типа File (!до Java 11 использовался по стандартной кодировке системы);
- **InputStreamReader** - входной поток, транслирующий байты в символы;
- **StringReader** - входной поток, читающий из строки.

nexign

BufferedReader

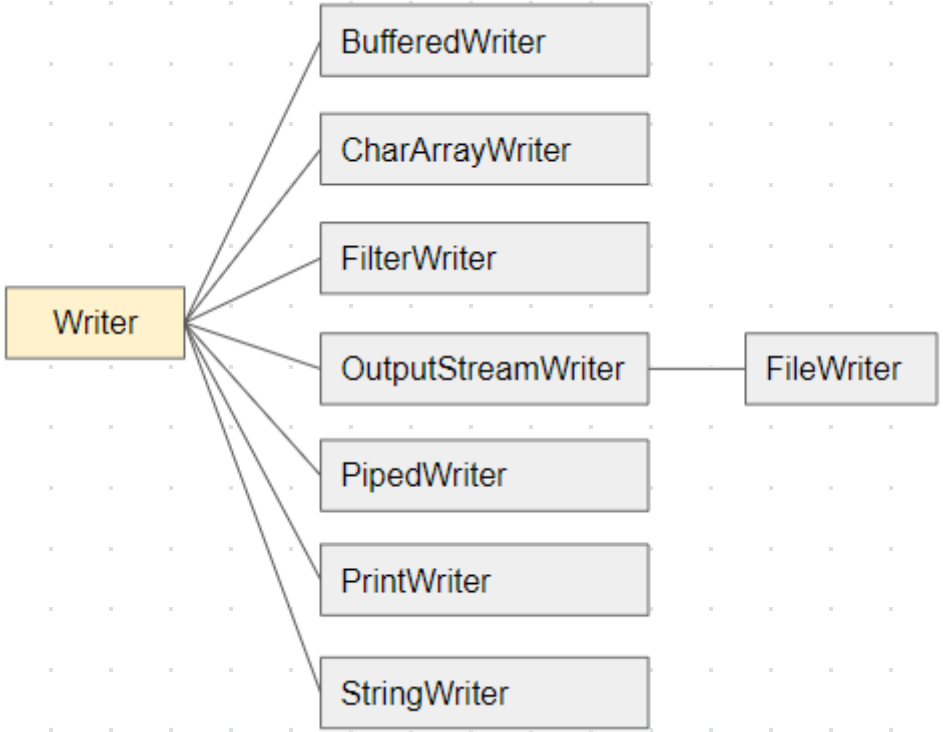


- **readline()** –считываение до перевода строки
- **lines()** – набор строк

```
    Reader reader = new FileReader("input-file.txt");
    BufferedReader buf = new BufferedReader(reader);
    String line = null;
    while((line = buf.readLine())!= null) {
        System.out.println(line);
    }
    buf.close();
}
```


nexign

Writer



nexign

Реализации Writer

- **BufferedWriter** - буферизированный выходной символьный поток; позволяет повысить производительность за счёт снижения количества операций физической записи в выходное устройство;
- **CharArrayWriter** - выходной поток, который пишет в символьный массив;
- **FileWriter** - выходной поток, пишущий в файл; в конструкторе можно определить вывод в конец файла. Создание объекта не зависит от наличия файла, он будет создан в случае необходимости. Если файл существует и он доступен только для чтения, то передаётся исключение IOException
- **OutputStreamWriter** -выходной поток, транслирующий байты в символы
- **PrintWriter** - выходной поток, включающий методы print() и println()
- **StringWriter** - выходной поток, пишущий в строку

nexign

PrintWriter



```
Path path = Paths.get("input-file.txt");
try (PrintWriter printWriter = new PrintWriter(
        Files.newBufferedWriter(
                path, StandardCharsets.UTF_8))) {
    printWriter.println("Hello");
}
```

- println() - вывод строковой информации с переводом строки.
- print() - вывод строковой информации без перевода строки.
- printf() - форматированный вывод.
- format() - форматированный вывод.

nexign

java.util.Scanner

- **next()**: считывает введенную строку до первого пробела
- **nextLine()**: считывает всю введенную строку
- **nextInt()**: считывает введенное число int
- **nextDouble()**: считывает введенное число double
- **nextBoolean()**: считывает значение boolean
- **nextByte()**: считывает введенное число byte
- **nextFloat()**: считывает введенное число float
- **nextShort()**: считывает введенное число short

```
Scanner in = new Scanner(System.in);
System.out.print("Input a number: ");
int num = in.nextInt();
System.out.printf("number: %d \n",
num);
in.close();
```


nexign

BufferedReader vs Scanner

Key	Scanner Class	BufferReader Class
Синхронизация	Не синхронизирован	Синхронизирован (потокобезопасен - thread-safe).
Объём буфера	Фиксированный размер 1 Кб	8 Кб и может быть расширен
Скорость выполнения	Медленнее, т.к. выполняет анализ входных данных	Быстрее, т.к. считывает данные без их синтаксического анализа
Методы	nextInt(), nextShort() и т.д.	parseInt(), parseShort() и т.д.

nexign

nexign

Файловая система



Файловая система



java.io.File



java.nio.files.*

nexign

java.nio.files.*

- Новый **API** для работы с файловой системой
- Добавлен в **Java 7**
- Покрывает всю функциональность **java.io.File**
- Более стройный **API**, а также более широкие возможности (работа со ссылками, с атрибутами файлов, отслеживание изменений)
- Центральная сущность — **java.nio.file.Path** — представляет путь в файловой системе
- Доступ к файловой системе обеспечивает класс **java.nio.file.Files**

nexign

Path

Paths - класс с единственным статическим методом **get()**.

Его создали исключительно для того, чтобы из переданной строки или URI получить объект типа **Path**.

- **getFileName()** — возвращает имя файла из пути;
- **getParent()** — возвращает «родительскую» директорию по отношению к текущему пути (то есть ту директорию, которая находится выше по дереву каталогов);
- **getRoot()** — возвращает «корневую» директорию; то есть ту, которая находится на вершине дерева каталогов;
- **startsWith()**, **endsWith()** — проверяют, начинается/заканчивается ли путь с переданного пути

nexign

Работа с файлами



Files — это утилитный класс, куда были вынесены статические методы из класса File.

```
Path java = Paths.get("/usr/bin/java ");
Files.exists(java); // true
Files.isRegularFile(java); // true
Files.isReadable(java); // true
Files.size(java); // 1536
Files.getLastModifiedTime(java)
        .toMillis(); // 1231914805000n
```

- Если файла нет, size() и getLastModifiedTime() бросают NoSuchFileException
- Stream<Path> list = Files.list(path); - содержимое папки (без захода в глубину)
- Stream<Path> Files.walk(Path start, int maxDepth, FileVisitOption... options) - метод для обхода дерева папок

nexign

nexign

DateTime API



Java 8 New Date/Time API



- **LocalDate –** дата без времени и временных зон;
- **LocalTime –** время без даты и временных зон;
- **LocalDateTime –** дата и время без временных зон;
- **ZonedDateTime –** дата и время с временной зоной;
- **DateTimeFormatter –** форматирует даты в строки и наоборот, только для классов **java.time;**
- **Instant –** колличество секунд с **Unix epoch time** (полночь 1 января 1970 UTC);
- **Duration –** продолжительность в секундах и наносекундах;
- **Period –** период времени в годах, месяцах и днях;
- **TemporalAdjuster –** корректировщик дат (к примеру, можно получить дату следующего понедельника);

Q&A



nexign

Обработка исключений

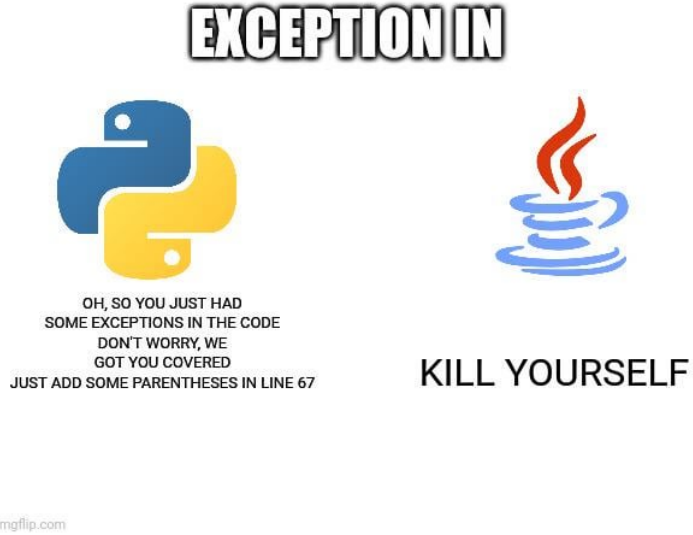


Исключение

Исключение (exception)
событие, возникающее в процессе работы программы и прерывающее её нормальное исполнение

- Исключение в Java − объект, описывающий исключительную (т.е. ошибочную) ситуацию в определенной части программного кода.

- В "ошибочном" методе генерируется объект, т.е. в определенный момент исключение перехватывается и обрабатывается.



nexign

Типы исключений

Error



Ситуации, при которых восстановление дальнейшей нормальной работы программы невозможно

Checked



Исключения, это те, которые должны обрабатываться или описываться в сигнатуре метода

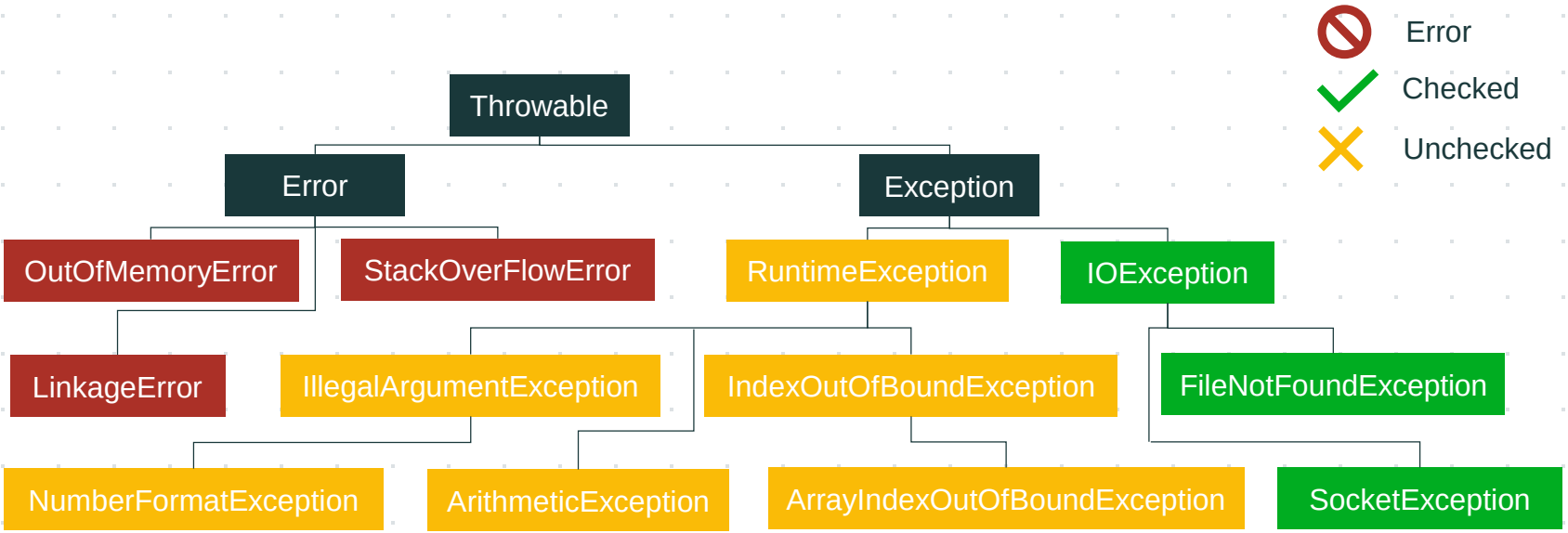
Unchecked



Могут не обрабатываться и не быть описанными

nexign

Иерархия исключений



nexign

Декларирование исключений

```
class MyAnimation {
. . .
    public Image loadImage(String s)
            throws FileNotFoundException, EOFException {
      . . .
    }
}
```

throws

В этом методе могут быть такие () исключения

- Можно перечислить для метода исключения в его поведении
- Перечисляются типы исключений, которые метод может генерировать
- Исключения кроме тех, которые относятся к классам **Error** и **RuntimeException** или любым их подклассам
- Не надо декларировать исключения, если одно — подтип другого

nexign

Выброс исключений

```
public double calculate(String expr) throws Exception{
    if (expr == null) {
        throw new Exception("expr is null");
    }
```

throw

прерывает нормальное исполнение программы и

запускает поиск обработчика исключения

- Если мы выбрасываем **checked exception**, компилятор позволит это сделать либо если оно задекларировано
- Либо если оно обрабатывается в самом методе





Перехват исключения: try-catch

```
try {
    //code that might throw exceptions
} catch (FileNotFoundException e) {
    //emergency action for missing files
} catch (UnknownHostException e) {
    // emergency action for unknown hosts
} catch (IOException e) {
    // emergency action for all other I/O problems
}
```

```
try {
        //code that might throw exceptions
    } catch (FileNotFoundException | UnknownHostException e) {
}
```


nexign

Объявление нового типа исключения

```
public class CalculatorException extends RuntimeException {
    public CalculatorException (String message) {
        super(message);
    }
    public CalculatorException (String message, Throwable cause) {
        super(message, cause);
    }
}
```

Throw early, catch late

- Не откладывайте выбрасывание исключения, как только стала ясна причина ошибки.
- Не спешите обрабатывать исключение, пока вам не стал полностью ясен механизм обработки.

nexign

Методы исключений

- String getMessage() – сообщение
- StackTraceElement[] getStackTrace() – стек вызовов
- Throwable getCause() / initCause()– причина
- Throwable[] getSuppressed()/addSupposed() – подавленные исключение

```
public static void main(String[] args) {
    try {
        ArithmeticException e = new ArithmeticException("Apparent cause");
        e.initCause(new NullPointerException("Actual cause"));
        throw e;
    } catch(ArithmeticException e) {
        System.err.println(e.getCause());
    }
```


nexign

nexign

Логирование



Логирование

Логирование — это процесс записи информации о работе приложения в структурированном виде для последующего анализа.

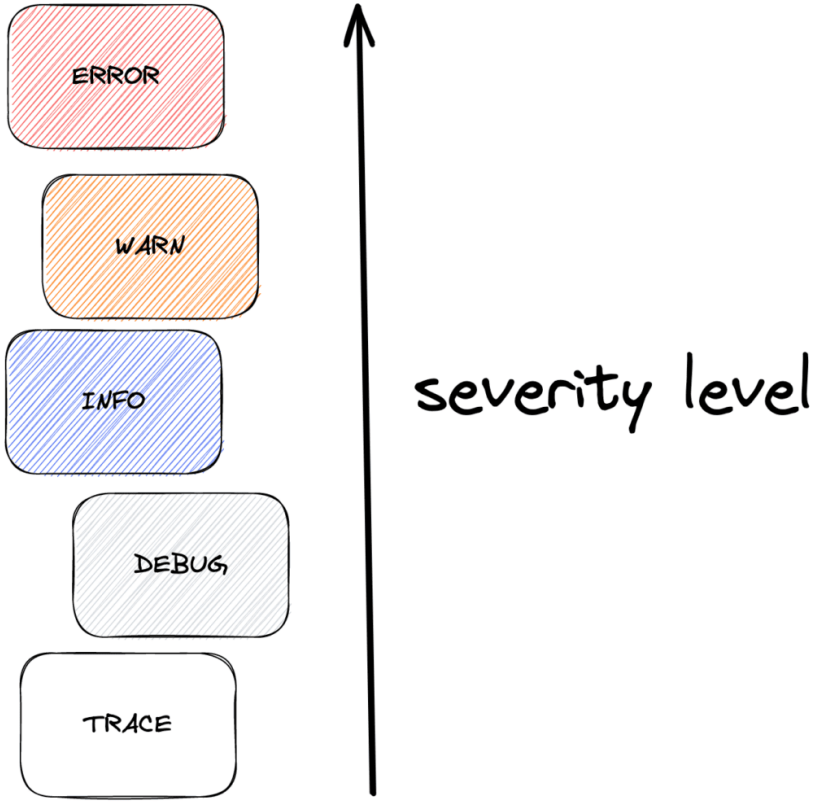
Основные цели:

- Отладка — поиск и исправление ошибок на этапе разработки
- Мониторинг — отслеживание состояния системы в production
- Аудит — фиксация важных событий (действия пользователей, изменения данных)
- Диагностика — анализ причин сбоев и аномалий

nexign

Уровни логирования

Критическая ошибка	Приложение не может выполнить операцию, требуется вмешательство
Предупреждение	Потенциальная проблема, но работа продолжается
Информация	Нормальные события жизненного цикла (запуск, остановка, завершение операций)
Отладка	Детальная информация для разработчиков (только на dev-среде)
Трассировка	Максимально детальный уровень (вход/выход из методов, значения переменных)



nexign

Инструменты



nexign

JUL Пример

```
public class DemoClass {
    private static final Logger logger = Logger.getLogger(DemoClass.class.getName());

    public static void main(String[] args) {
        try {
            FileHandler fh = new FileHandler("app.log", true);
            fh.setFormatter(new Formatter() {
                @Override
                public String format(LogRecord record) {
                    return "[" + record.getLevel() + "] " + record.getMessage() + "\n";
                }
            });
            fh.setLevel(Level.WARNING);
            logger.addHandler(fh);

        } catch (IOException e) {
            logger.log(Level.SEVERE, "Can't write logs to file", e);
        }
        logger.info("App started successfully");
        logger.warning("This is a warning message");
        logger.severe("This is an error message");
    }
```

Создаем экземпляр логгера по имени класса

Стандартный ConsoleHandler останется, но к нему добавиться FileHandler

Если хотим отключить то можно

logger.setUseParentHandlers(false);

Уровень вывода работает для этого хендлера

nexign

Logging.properties

DemoClass.java

```
try (InputStream config = DemoClass.class.getClassLoader().getResourceAsStream("logginig.properties")) {
    LogManager.getLogManager().readConfiguration(config);
} catch (IOException e) {
    e.printStackTrace();
}
```

Динамическая загрузка logging.properties из resources в classpath

Иначе загрузка идет из $JAVA_HOME

Logginig.properties

```
# logging.properties
handlers = java.util.logging.FileHandler

java.util.logging.FileHandler.pattern = logs/myapp.%g.log
java.util.logging.FileHandler.limit = 10485760    # 10 MB
java.util.logging.FileHandler.count = 10
java.util.logging.FileHandler.formatter =
java.util.logging.SimpleFormatter
java.util.logging.FileHandler.append = true
```

Ротация логов: пишем 10 файлов по 10 мб, потом перезатираем с самого старого

nexign

Q&A



nexign

Generics



Проблема



```
Manager ceo = ...;
Manager cto = ...;
Employee cleaner = ...;
List managers = new ArrayList();
managers.add(ceo);
managers.add(cto);

//bug!!
managers.add(cleaner);

//typecast with runtime exception -- too late!
Manager m = (Manager) managers.get(2);
```


nexign

Решение

```
Manager ceo = ...;
Manager cto = ...;
Employee cleaner = ...;
List<Manager> managers = new ArrayList<>();

managers.add(ceo);
managers.add(cto);
// не скомпилируется!
managers.add(cleaner);

// приводить тип не нужно!
Manager m = managers.get(1);
```



nexign

Обобщения (Generics)

- Позволяют указать «условный тип», с которым работает код
- Используются, когда один и тот же код применим к большому числу типов
- Позволяют повысить безопасность, за счет контроля типов на стадии компиляции программы, а не выполнения
- Все интерфейсы и классы коллекций – параметризованные
- Параметр типа может использоваться внутри нестатических членов класса:
  - Тип поля, параметра, переменной
  - Возвращаемый тип метода
  - Подставляться параметрам другого типа

nexign

Параметризированный класс



```
public class Pair<T> {
    private T first;
    private T second;
    public Pair() { first = null; second = null; }
    public Pair(T first, T second) {
        this.first = first;
        this.second = second;
    }
    public T getFirst() { return first; }
    public T getSecond() { return second; }
    public void setFirst(T newValue) { first = newValue; }
    public void setSecond(T newValue) { second = newValue; }
}
```

Параметризированный метод



```
public static <T, V> boolean isIn(T x, V[] array) {
    for (V element : array) {
        if (x.equals(element)) {
            return true;
        }
    }
    return false;
}
```


nexign

Особенности параметризированных типов

- Использовать примитивные типы в качестве параметров-типов нельзя
- Если одинаковые настраиваемые типы имеют различные аргументы, то это различные типы
- Статические члены класса не могут использовать его типы-параметры
- В обобщенном коде информация о параметрах-типах обобщения **стирается при компиляции**

nexign

Сырые типы

Generic Type (source)

```
class Pair<T> {
    private T first;
    Pair(T first)
    {this.first = first}
    T getFirst()
    {return first; }
    void setFirst(T newValue)
}
```

Raw Type (compiled)

```
class Pair {
    private Object first;
    Pair(Object first)
    {this.first = first}
    Object getFirst()
    {return first; }
    void setFirst(Object newValue)
    {first = newValue;}
}
```


nexign

Ограниченные типы



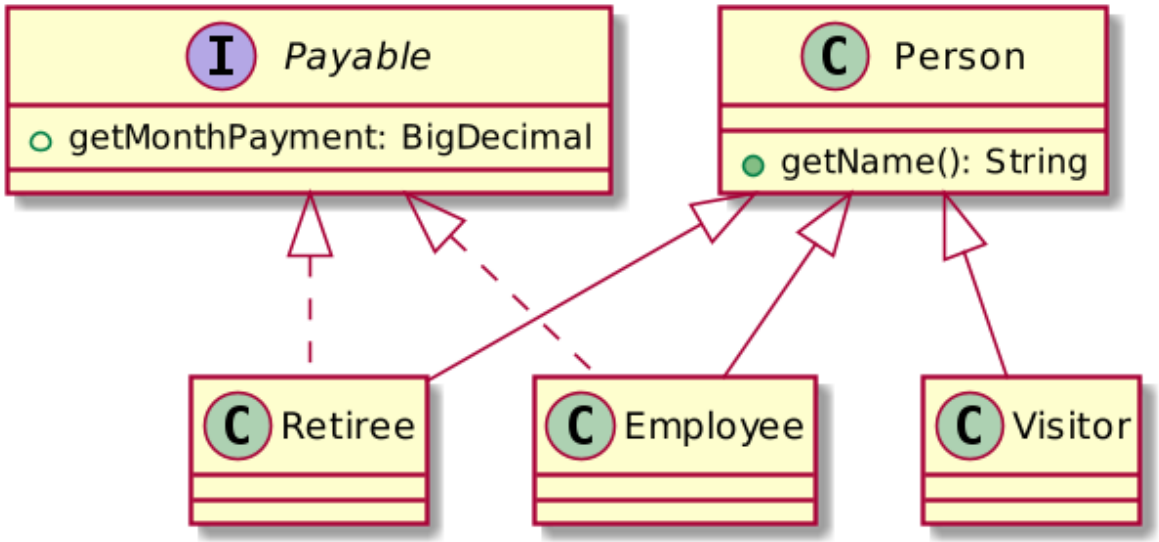
```
public <T extends Person> String getRandomPersonName(List<T> items) {
    Person result = //можно написать и T result =
            items.get(ThreadLocalRandom.current().nextInt(items.size()));

    return result.getName();
}
```

- Ограничение типа позволяет использовать у ссылок методы и поля, доступные в типе-ограничителе
- Типы, не наследующие от указанного, не могут быть использованы при создании объектов

nexign

Пересечение типов



nexign

Сырые типы

Generic Type (source)

```
class Pair<T extends Employee> {
    private T first;
    Pair(T first)
    {this.first = first}
    T getFirst()
    {return first; }
    void setFirst(T newValue)
    {first = newValue;}
}
```

Raw Type (compiled)

```
class Pair {
    private Employee first;
    Pair(Employee first)
    {this.first = first
    Employee getFirst()
    {return first; }
    void setFirst(Employee newValue)
    {first = newValue;}
}
```


nexign

Ограничения

```
if (obj instanceof T)
// compilation error
```

Во время исполнения информация о generic-параметрах недоступна

```
T obj = new T (); // compilation error
T [] arr = new T [0]; // compilation error
```

По имени параметра нельзя создать экземпляр или массив

Вы не можете создать class generic который является наследником Throwable

```
public class MyException<E>
          extends Exception {...}
// compilation error
```


nexign

nexign

Wildcards



Метасимвольные аргументы (Wildcards)

```
class Generic<T> {
    //...
    boolean compare(Generic<T> o) {…}
}
```

```
Generic<Integer> gi = new Generic<Integer>();
isEqual = gi.compare(new Generic<Integer>());
isEqual = gi.compare(new Generic<Runnable>()); //error
isEqual = gi.compare(new Generic<StringBuilder>()); //error
isEqual = gi.compare(new Generic());
```

В качестве параметра методу **compare** может быть передан тип **Generic**, параметризированный только конкретным типом

nexign

Метасимвольные аргументы (Wildcards)

```
class Generic<?> {
    //...
    boolean compare(Generic<T> o) {…}
}
```

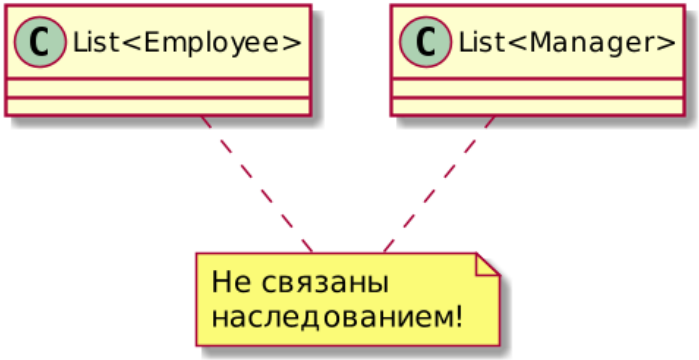
```
Generic<Integer> gi = new Generic<Integer>();
isEqual = gi.compare(new Generic<Integer>());
isEqual = gi.compare(new Generic<Runnable>()); //ok
isEqual = gi.compare(new Generic<StringBuilder>()); //ok
isEqual = gi.compare(new Generic());
```

В качестве параметра методу compare может быть передан тип Generic, параметризированный **любым (а не конкретным)** типом

nexign

Инвариативность

```
List<Manager> managers = ...
Employee lowlyEmployee = ...
/*Тупо не скомпилируется*/
List<Employee> e = managers;
```



List<T> - List принимает T и его наследников, и возвращает объекты T.

В Java обобщённые типы инвариативны

```
ArrayList<String> strings = new ArrayList<Object>();
ArrayList<Object> objects = new ArrayList<String>();
```


nexign

Ковариативность

```
List<? extends Person> list = ...

Person e1 = list.get(...);

Person e2 = ...;
//не скомпилируется! ПОЧЕМУ??
list.add(e2);
//скомпилируется. ПОЧЕМУ??
list.add(null);
```



Upper Bounded Wildcards

`<? extends T>`

List<? extends T> - List **возвращает объекты** T, так как содержит объекты T либо наследников T:
вы можете привести **A<C>** к **A<B>,** где **C** -подкласс **B**, если **A** производит общие значения (возвращает в результате функции).
Ковариация связана с производителями (**producers**).

nexign

Контрвариативность

```
List<? super Person> list = ...

Person e1 = ...;
//скомпилируется!
list.add(e1);
list.add(null);
//Только Object. ПОЧЕМУ??
Object e2 =  list.get(...);
```



Lower Bounded Wildcards

`<? super T>`

List<? extends T> - List **возвращает** объекты T, так как содержит объекты T либо наследников T:
вы можете привести **A<B>** к **A<C>,** где **C** -подкласс **B**, если **A** производит общие значения (возвращает в результате функции).
Ковариация связана с потребителями (**consumers**).

nexign

Выводы

- Параметризированный тип **Gen<T>** называют ковариантным типу **T**, если из
  T extends K => Gen<T> extends Gen<K>

- Параметризированный тип **Gen<T>** называют контравариантным типу **T**, если из
  T extends K => Gen<T> super Gen<K>

- Параметризированный тип **Gen<T>** называется инвариантным типу **T**, если отношение наследования исходных типов **T**,**K** не переносится на производные **Gen<T>, Gen<K>**.
- **List<?>** — это то же, что **List<? extends Object>**

nexign

PECS

> Если метод читает данные из аргумента, то этот аргумент — *производитель (producer)*, а если метод передаёт данные в аргумент, то аргумент является *потребителем (consumer)*.

producers – **use** **extends** если нужно получить состояние объекта

consumers – **use** **super** если нужно изменить состояние объекта

nexign

nexign

Коллекции



Коллекции

Коллекция – это набор объектов, упорядоченный по определенным правилам

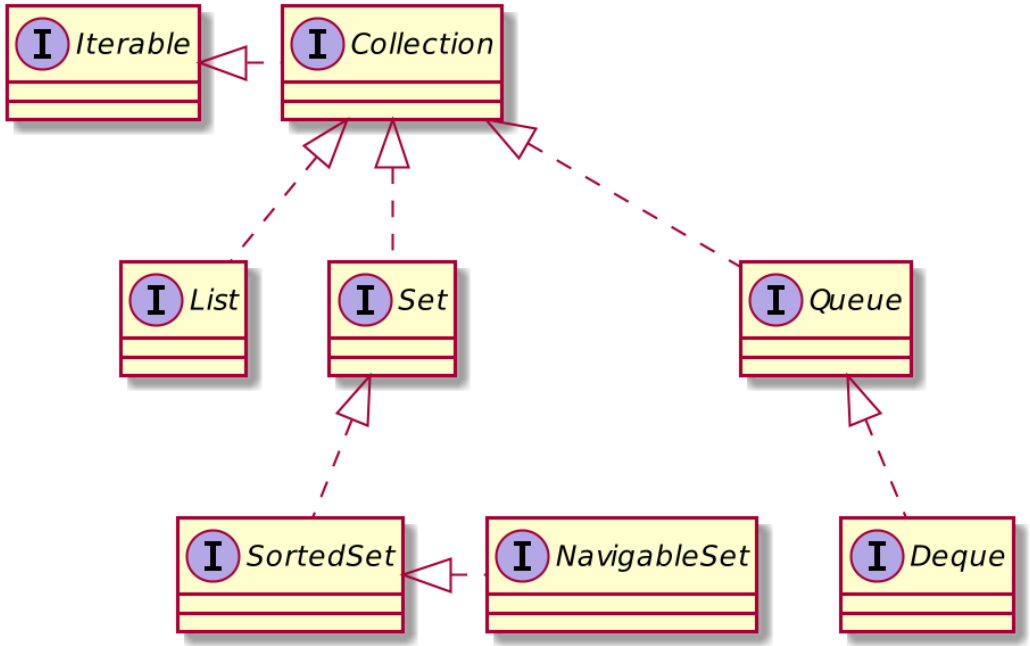
API коллекций содержит набор интерфейсов, описывающих разные способы упорядочивания элементов:

- добавление/удаление элементов
- read-only коллекции
- операции поиска, объединения, вычитания
- поддержка разнообразных специальных случаев

- В отличие от массивов, могут хранить только объекты.
- Для примитивных типов — классы-обертки и autoboxing
- Пакет **java.util**

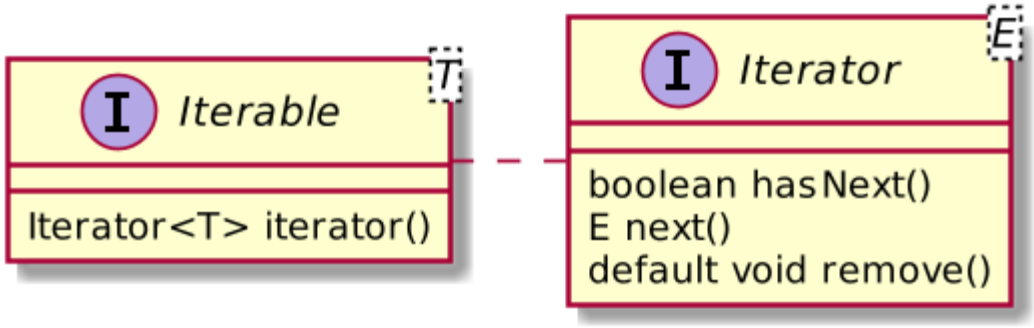
nexign

Иерархия интерфейсов



nexign

Итератор




```
Iterable<T> collection = ...
        for (T e: collection) {
        ...
}
```

```
Iterable<T> collection = ...
Iterator<T> i =
collection.iterator();
while (i.hasNext()) {
    T e = i.next();
    if (e...)
    i.remove();
}
```


nexign

Collection

Методы:

- Object[] toArray();
- <T> T[] toArray(T[] a);
- int size();
- boolean isEmpty();
- boolean contains(Object o);
- boolean add(E e) ;
- boolean remove(Object o);
- void clear();

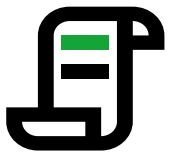
Суррогатные коллекции:

Пустые	Из одного элемента
Collections.emptyList();	Collections.singletonList(o);
Collections.emptySet();	Collections.singleton(o);
Collections.emptyMap();	Collections.singletonMap(k,v);

```
//Из заданных элементов
//Java 9+ feauture
List.of(a, b, c...);
Set.of(a, b, c...);
Map.of(k1, v1, k2, v2...);
```

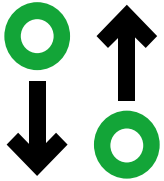

nexign

Разновидности коллекций



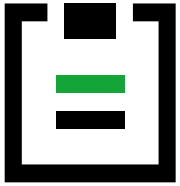
List

список (фиксированный порядок, доступ к элементам по индексу)



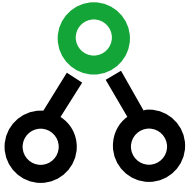
Queue/Deque

очередь и дек доступ к элементам с начала и с конца)



Set

множество (каждый элемент встречается не более одного раза)



Map

ассоциативный массив (набор пар «ключ–значение»)

nexign

nexign

Lists



List

Методы:

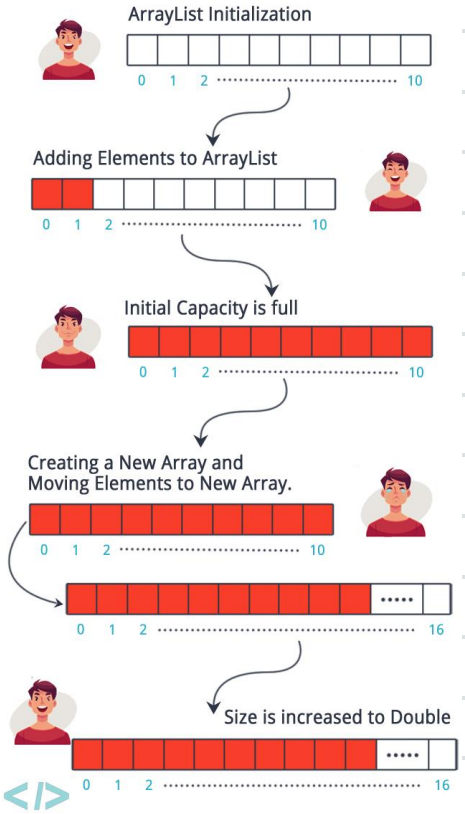
- **boolean addAll(int index, Collection<? extends E> c)** – вставляет все элементы в вызывающий список, начиная с позиции, переданной в index
- **E get(int index)** - возвращает элемент коллекции по заданному индексу.
- **E set(int index, E element)** - присваивает obj элементу, находящемуся в списке в позиции index.
- **void add(int index, E element) –** добавить элемент в позицию. Остальные сдвигаются
- **E remove(int index)** - удаляет элемент из вызывающего списка в позиции index и возвращает удаленный элемент. Остальные сдвигаются
- **int indexOf(Object o)** – индекс первого вхождения
- **int lastIndexOf(Object o)** – индекс последнего вхождения
- **List<E> subList(int fromIndex, int toIndex)** – фрагмент листа

nexign

ArrayList

ArrayList
является одной из реализаций *List* , созданных поверх массива, которая может динамически увеличиваться и уменьшаться при добавлении/удалении элементов.

- Реализация списка на основе массива
- Специфические операции:
  - void ensureCapacity(int capacity)
  - void trimToSize()
- Эффективный доступ к элементу по индексу
- Вставка/удаление по индексу имеет линейную трудоемкость



nexign

Сложность алгоритмов в ArrayList

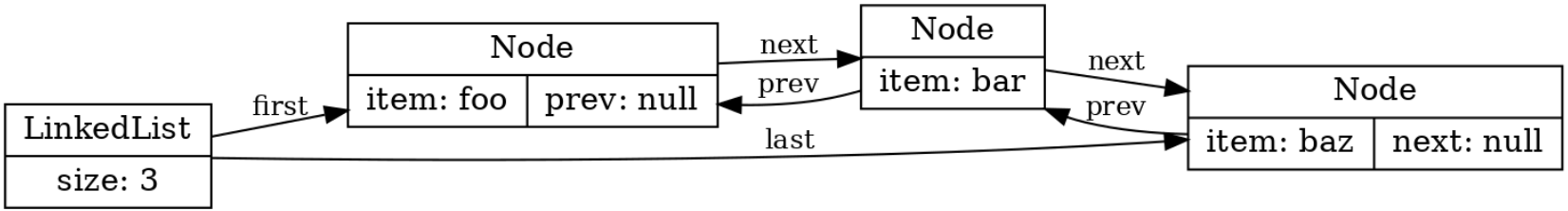
- get(int index) **O(1)**
- add(E element) **O(1)** / **O(n)**
- add(int index, E element) **O(n)**
- remove(int index) is **O(n)**
- Iterator.remove() is **O(n)**
- ListIterator.add(E element) is **O(n)**

```
List<String> list = new ArrayList<>();
list.add("one");
list.add("two");
list.add(1,"three");
list.remove(0);
list.sort(Comparator.reverseOrder());
System.out.println(list.toString());
```

O(1) — константная сложность
O(n) — линейная сложность
O(log n) — логарифмическая сложность
O(n^2) — квадратичная сложность

nexign

LinkedList



- Реализация списка на основе двусвязного списка
- Эффективные вставка и удаление элемента в начале и в конце списка
- Доступ к элементу по индексу имеет линейную трудоемкость **O(n)**

nexign

LinkedList

LinkedList содержит все те методы, которые определены в интерфейсах List, Queue, Deque. Некоторые из них:

- **addFirst() / offerFirst()**: добавляет элемент в начало списка
- **addLast() / offerLast()**: добавляет элемент в конец списка
- **removeFirst() / pollFirst()**: удаляет первый элемент из начала списка
- **removeLast() / pollLast()**: удаляет последний элемент из конца списка
- **getFirst() / peekFirst()**: получает первый элемент
- **getLast() / peekLast()**: получает последний элемент

nexign

nexign

Queue



Queue

First In — First Out (FIFO)

- **E peek()**: **/ E element()**: возвращает, но не удаляет, элемент из начала очереди
- **E add() / boolean offer(E obj)**: пытается добавить элемент obj в конец очереди.
- **E remove()**: **/ E poll()**: возвращает с удалением элемент из начала очереди.

Очереди не могут хранить значения *null*.

nexign

PriorityQueue



- ***PriorityQueue()*** - создает очередь с приоритетами начальной емкостью 11, размещающую элементы согласно естественному порядку сортировки (*Comparable*);
- ***PriorityQueue(Collection<? extends E> c);***
- ***PriorityQueue(int initialCapacity);***
- ***PriorityQueue(int initialCapacity, Comparator<? super E> comparator);***
- ***PriorityQueue(PriorityQueue<? extends E> c);***
- ***PriorityQueue(SortedSet<? extends E> c).***

nexign

Deque

Расширяет Queue и описывает поведение двунаправленной очереди.
Двунаправленная очередь может функционировать как стандартная очередь FIFO либо как стек LIFO

nexign

ArrayDeque



Операции:

- **boolean offerFirst(E e) /boolean offerLast(E e)** - пытается добавить *obj* в голову/хвост двунаправленной очереди
- **E peekFirst() /E peekLast()** - возвращает элемент, находящийся в голове/хвосте двунаправленной очереди
- **E pollFirst() /E pollLast() -** - возвращает элемент, находящийся в голове/хвосте, одновременно удаляя его из очереди
- **void addFirst(E obj) / void addLast(E obj)** - добавляет *obj* в голову/хвост двунаправленной очереди
- **E getFirst() / E getLast()** - возвращает первый/последний элемент двунаправленной очереди
- **E pop()** - возвращает элемент, находящийся в голове двунаправленной очереди, одновременно удаляя его из очереди.
- **void push(E obj)** - добавляет элемент в голову двунаправленной очереди.

nexign

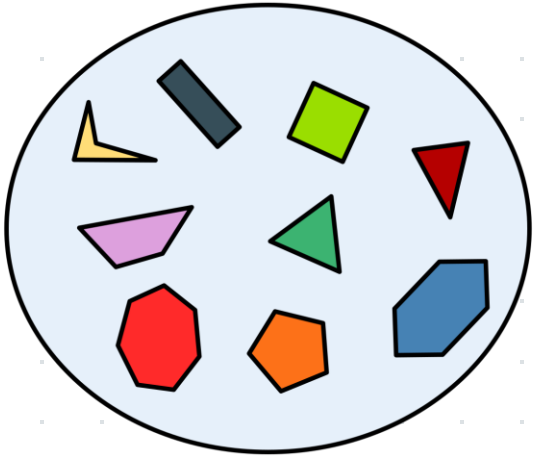
nexign

Set



Множества (Sets)

- Каждый элемент встречается не более одного раза
- Гарантирует, что при добавлении элементов дубликаты не появятся
- Множества равны, если содержат одинаковые элементы



nexign

Реализации Set

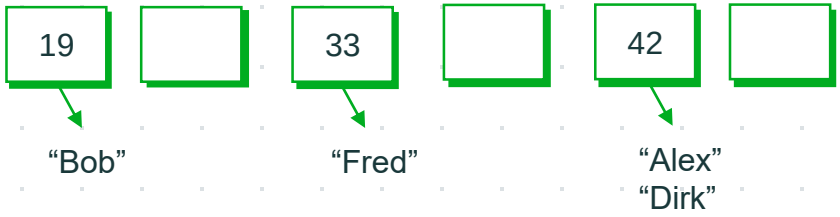
- **HashSet** – изменяемое неупорядоченное множество общего назначения
- **LinkedHashSet** – изменяемое упорядоченное (ordered) множество общего назначения
- **TreeSet –** изменяемое сортированное (sorted) множество
- **EnumSet –** изменяемое множество enum
- **Collection.emptySet()** – неизменяемое пустое множество
- **Collection.singleton(x) –** неизменяемое множество из одного элемента
- **Set.of()** – Java 9+ - неупорядоченное неизменяемое множество заданных элементов без null и повторов)
- **Set.copyOf()** – Java 10+ - неизменяемая копия

nexign

Хэш-таблица

Элементы хеш-таблицы хранятся в виде пар ключ-значение.
Ключ определяет ячейку (или бакет) для хранения значения.
Содержимое ключа служит для определения однозначного значения, называемого хеш-кодом.

Ключ	Алгоритм хеш-кода	Хеш-код
Alex	A(1) + l(12) + e(5) + x(24)	**42**
Bob	B(2) + o(15) + b(2)	19
Dirk	D(4) + i(9) + r(18) + k(11)	**42**
Fred	F(6) + r(18) + e(5) + d(4)	33



nexign

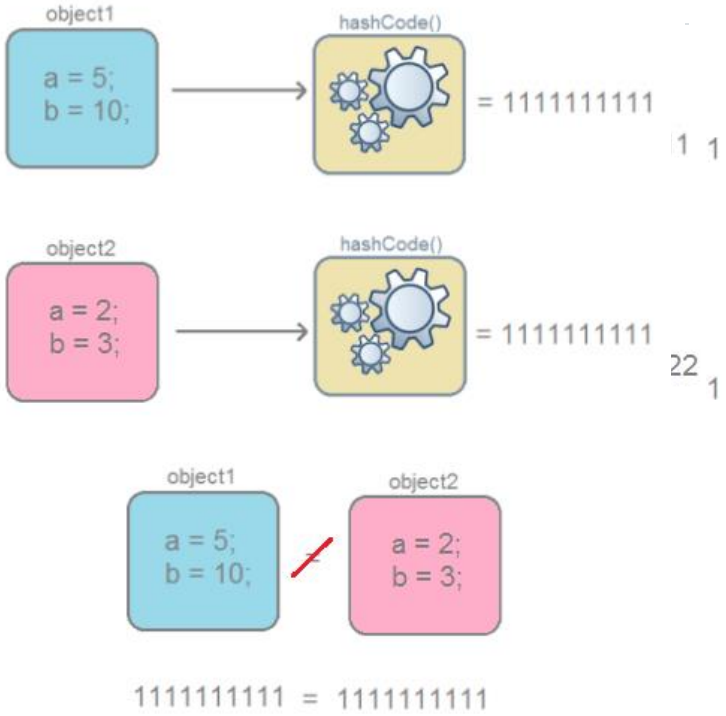
Контракт hashCode() и equals()

1.Для одного и того же объекта, хеш-код всегда будет одинаковым.

2.Если объекты одинаковые, то и хеш-коды одинаковые (но не наоборот).

3.Если хеш-коды равны, то входные объекты не всегда равны.

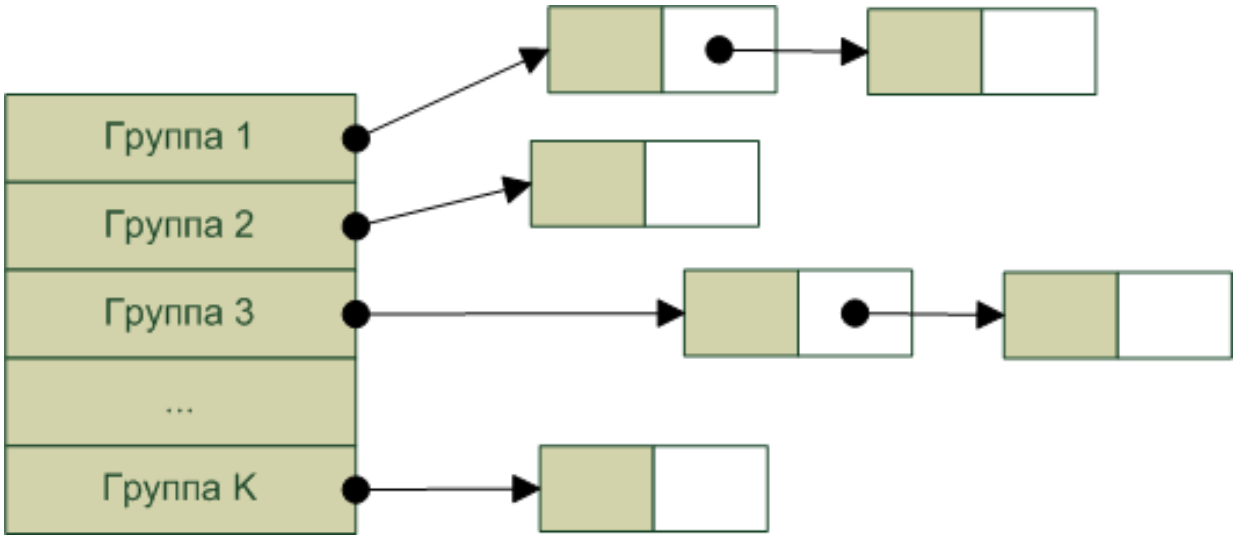
4.Если хеш-коды разные, то и объекты гарантированно будут разные.



nexign

Разрешение коллизий

В java хэш-таблицы реализованы в виде связанных списков



nexign

HashSet

- Реализация множества на основе хеш-таблицы
- Порядок обхода элементов непредсказуем
- При коллизии формирует связанный список
- Если коллизий много, а элементы сравнимы (comparable), то растит RB-дерево

```
Set<String> words = new HashSet<>();
words.add("one");
words.add("one");
words.add("two");
words.add("two");
```

LinkedHashSet

- Порядок обхода элементов определяется порядком вставки
- Работает дольше чем класс HashSet.

Хэш-таблица с прямой адресацией обеспечивает высокую скорость операций:

- insert $O(1)$
- search $O(1)$
- remove $O(1)$

nexign

TreeSet

Порядок элементов определяется:

- объектом типа **java.util.Comparator с методом int compare(T o1, T o2)**
- методом элементов **int compareTo(T o)** (элементы должны реализовать интерфейс **java.lang.Comparable**)

- **first(), last() -** возвращает первый/последний элемент набора
- **headset(), trailSet(), subset()** - возвращает объект SortedSet, который содержит все элементы первичного набора до элемента end/start/между ними
- **floor(E), ceiling(E) -** ищет в наборе наименьший/больший элемент e, который больше/меньше obj
- **higher(E), lower(E)** - ищет в наборе наименьший/больший элемент e, который больше/меньше элемента
- **E pollFirst()/pollLast()** - возвращает первый/последний элемент и удаляет его из набора

nexign

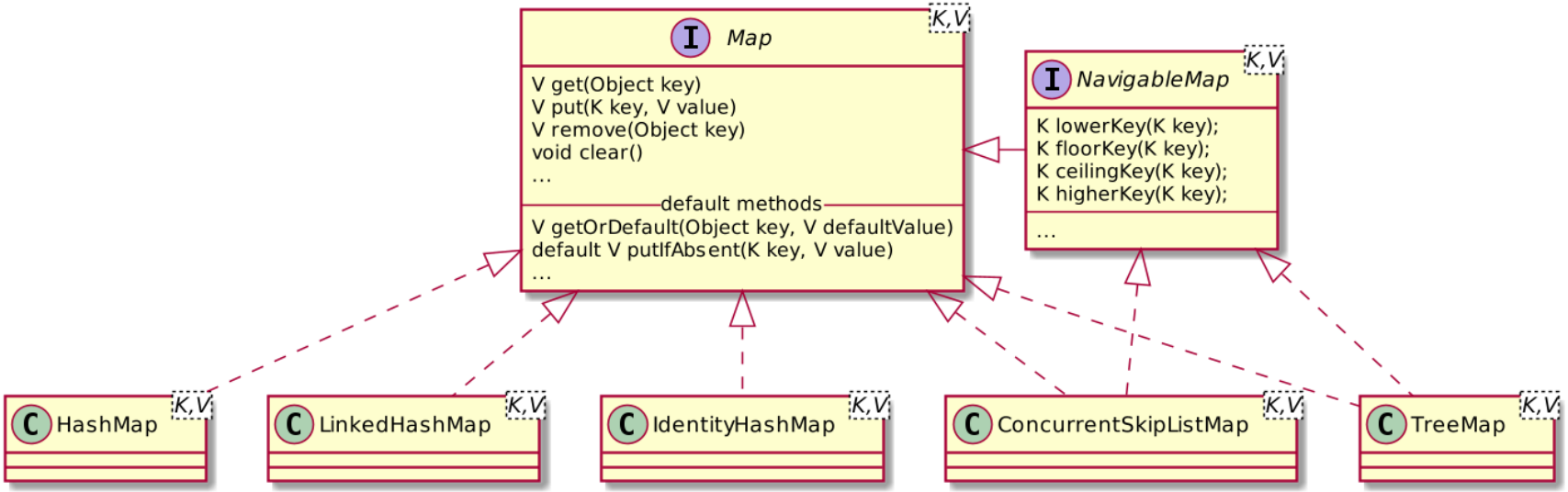
nexign

Map



Map

Карта (Map) – это набор пар ключ-значение



nexign

Map



- Не наследует java.util.Collection

int size()	возвращает число пар "ключ-значение" в карте.
V get(Object key)	возвращает значение, ассоциированное с ключом k. Возвращает null, если ключ не найден.
V put(K key, V value)	помещает элемент в вызывающую карту, перезаписывая любое предшествующее значение, ассоциированное с ключом. Возвращает null, если ключ ранее не существовал. В противном случае возвращается предыдущее значение, связанное с ключом.
V remove(Object key)	удаляет элемент, чей ключ равен k.
boolean containsKey(Object key)	возвращает true, если вызывающая карта содержит ключ k
boolean containsValue(Object value)	возвращает true, если вызывающая карта содержит значение v
Set<K> keySet()	возвращает Set, который содержит ключи вызывающей карты. Этот метод представляет ключи вызывающей карты в виде набора.
Collection<V> values()	возвращает коллекцию, содержащую значения карты. Этот метод представляет значения, содержащихся в карте, в виде коллекции.
Set<Map.Entry<K, V>> entrySet()	возвращает Set, содержащий все значения карты. Набор содержит объекты типа Map.Entry. То есть этот метод представляет карту в виде набора.
boolean isEmpty()	возвращает true, если вызывающая карта пуста
void putAll(Map<? extends K, ? extends V> m)	помещает все значения из m в карту.

Реализации Map

- **HashMap**– изменяемый неупорядоченный Map общего назначения
- **LinkedHashMap** – изменяемый упорядоченный Map общего назначения
- **TreeMap**–сортированный Map
- **EnumMap**– Map, где ключи –и элементы enum
- **IdentityHashMap** -ключи сравниваются по ==, а не по equals().
- **WeakHashMap** - значения могут быть собраны сборщиком мусора, если не используются где-то ещё.
- **Collection.emptyMap()** – неизменяемый пустой Map
- **Collection.singletonMap(x)** – неизменяемый Map из одного элемента

nexign

HashMap

```
Map<String, String> dictionary = new HashMap<>();
dictionary.put("foo", "bar");
dictionary.put("bar", "baz");
dictionary.remove("bar");
```

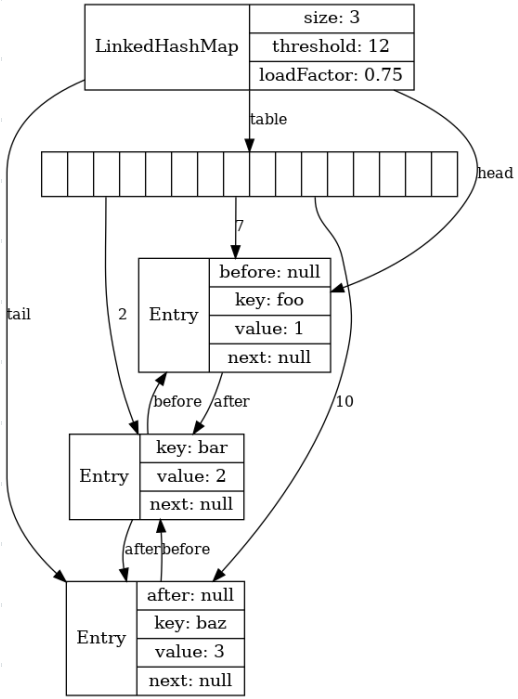
- Реализация ассоциативного массива на основе хеш-таблицы
- Порядок обхода элементов непредсказуем
- Есть **java.util.LinkedHashMap**

nexign

LinkedHashMap

```
Map<String, Integer> map = new LinkedHashMap<>();
map.put("foo", 1);
map.put("bar", 2);
map.put("baz", 3);
```

- Помнит порядок вставки элементов.
- Хорош для хранения настроек вида «ключ-значение», задаваемых текстом.



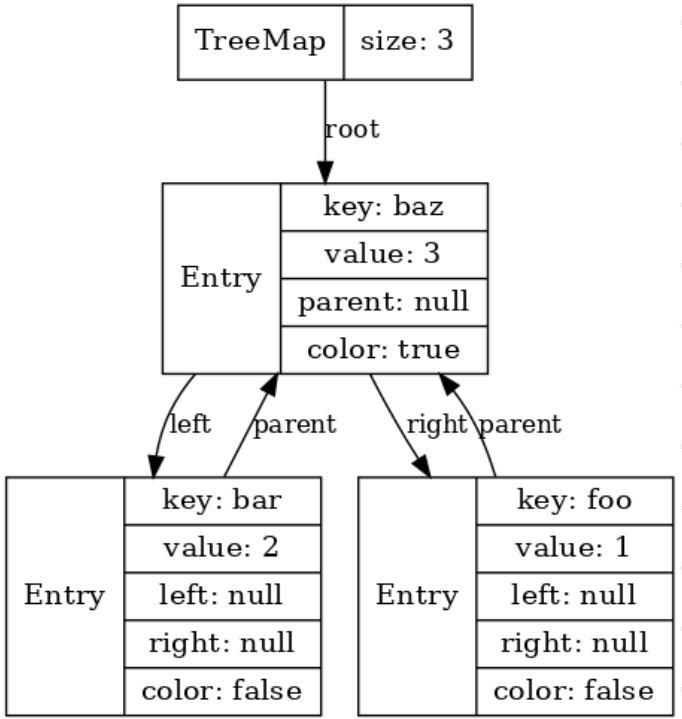
nexign

TreeMap

```
Map<String, Integer> map = new TreeMap<>();
map.put("foo", 1);
map.put("bar", 2);
map.put("baz", 3);
```

- Красно-чёрное дерево, ключи сравниваются по Comparable или Comparator.
- Как обычный Map уступает HashMap-у,

.



nexign

Обход Map

```
void iterateOverMap(Map<String, Integer> map) {
    for (String key: map.keySet()) {
        Integer value = map.get(key);
        System.out.println(key + " --> " + value);
    }
}
```



```
void iterateOverMap(Map<String, Integer> map) {
    for (Map.Entry<String, Integer> entry: map.entrySet()){
        String key = entry.getKey();
        Integer value = entry.getValue();
        System.out.println(key + " --> " + value);
    }
}
```



nexign

Только значения

```
void iterateOverValues(Map<String, Integer> map) {
    for (Integer value: map.values()) {
        System.out.println("Value: " + value);
    }
}
```


nexign

Обход в функциональном стиле

```
void iterateOverMap(Map<String, Integer> map) {
    map.forEach(new BiConsumer<String, Integer>() {
        @Override
        public void accept(String key, Integer value) {
            System.out.println(key + " --> " + value);
        }
    });
}
```

```
void iterateOverMap(Map<String, Integer> map) {
    map.forEach((key, value) ->
                System.out.println(key + " --> " + value));
}
```


nexign

Модификация

```
void trimAllValues(Map<String, String> map) {
    for (String key: map.keySet()) {
        String value = map.get(key);
        map.put(key, value.trim());
    }
}
```



```
void trimAllValues(Map<String, String> map) {
    for (Map.Entry<String, String> entry : map.entrySet()) {
        entry.setValue(entry.getValue().trim());
    }
}
```



nexign

Модификация

```
void trimAllValues(Map<String, String> map) {
   map.replaceAll((key, value) -> value.trim());
}
```



nexign

Удаление элементов

```
void removeUnwantedValues (Map<String, String> map) {
   for (String key : map.keySet()) {
       String value = map.get(key);
       if (value.equals("foo")
          || (value.equals("bar")
          || (value.equals("buz")) {
           map.remove(key);}}
}
```



```
void removeUnwantedValues (Map<String, String> map) {
    Iterator<Map.Entry<String, String>> iterator = map.entrySet().iterator();
    while (iterator.hasNext()) {
        Map.Entry<String, String> entry = iterator.next();
        if (entry.getKey().equals("bar")
            ||(entry.getKey().equals("foo")
            ||(entry.getKey().equals("buz")) {
            iterator.remove();}}}
```



nexign

Удаление элементов

```
void removeUnwantedValues (Map<String, String> map) {
   map.values().removeIf(value -> value.equals("foo") ||
           value.equals("bar") ||
           value.equals("buz"));
}
```



```
public static final List<String>UNWANTED_VALUES  = Arrays.asList("foo","bar", "baz");

void removeUnwantedValues (Map<String, String> map) {
   map.values().removeAll(UNWANTED_VALUES);
}
```



nexign

Устаревшие коллекции

- java.util.Vector
- java.util.Stack
- java.util.Dictionary
- java.util.Hashtable

nexign

nexign

Элементы функционального программирования

Lambdas

Stream API



nexign

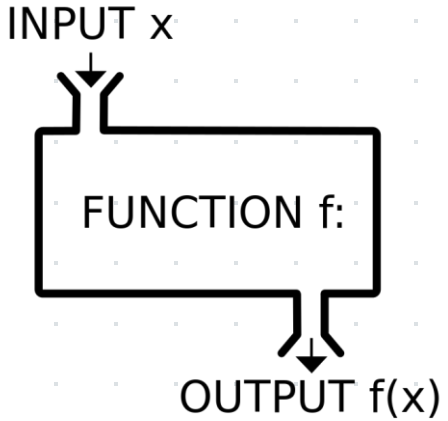
nexign

Функциональный подход в Java



Функциональное программирование

> **Функциона́льное программи́рование**
>
> парадигма программирования, в которой процесс вычисления трактуется как вычисление значений функций в математическом понимании последних (в отличие от функций как подпрограмм в процедурном программировании).



- LISP + диалекты
- Erlang
- F#
- Haskell
- и другие

nexign

Функциональный интерфейс

- Функциональным называется такой интерфейс, в котором определяется единственный абстрактный метод
- Помечается аннотацией **@FunctionalInterface**
- Метод интерфейса считается абстрактным лишь в том случае, если у него отсутствует реализация (**default**).

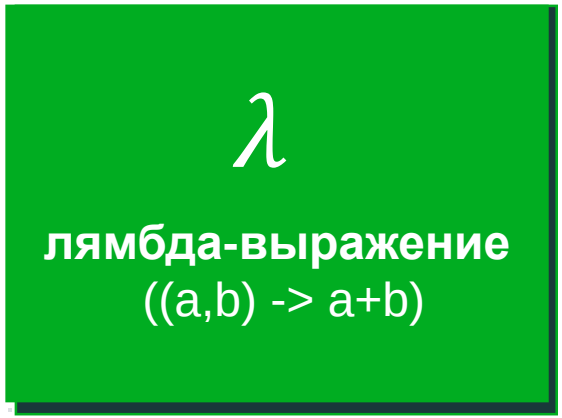
```
@FunctionalInterface

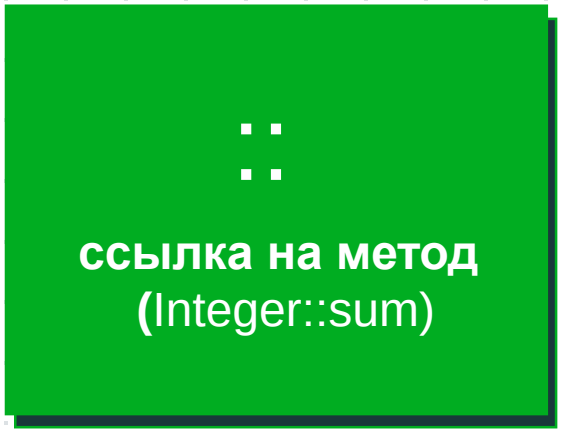
interface Operationable{
    int calculate(int x, int y);
}
```


nexign

Функциональные выражения

- Не имеют состояния
- Конкретный тип устанавливается по контексту
- Образует реализацию метода, определенного в функциональном интерфейсе





nexign

nexign

Лямбда выражения



Лямбда-выражения

Лямбда-выражение − является анонимным (т.е. безымянным) методом.

Метод служит для реализации метода, определяемого в функциональном интерфейсе.

Структура:

Аргументы

(Type1 name1, Type2 name2)

(name1, name2)

Name

Стрелка ->

Тело:

Выражение System.out.println()

Блок {System.out.println()}

```
(int a, int b) -> {  return a + b; };

() -> System.out.println("Hello World");

(String s) -> { System.out.println(s);
};

() -> 42;

() -> { return 3.1415 };
```


nexign

Особенности лямбда- выражений

Являются в некотором роде сокращенной формой внутренних анонимных классов

Чтобы объявить и использовать лямбда-выражение, основная программа разбивается на ряд этапов:

1.Определение ссылки на функциональный интерфейс //Operationable operation;

2.Создание лямбда-выражения //operation = (x,y)->x+y;

3. Использование лямбда-выражения в виде вызова метода интерфейса // int result = operation.calculate(10, 20)

- Параметры лямбда-выражения должны соответствовать по типу параметрам метода из функционального интерфейса.

  При написании самого лямбда-выражения тип параметров писать необязательно;
- В лямбда-выражении использование обобщений не допускается

nexign

Захват значений

Closure (замыкания):

- Локальная переменная – должна быть effectively final → производится захват значения
- Поле – захват this и чтение при выполнении
- Static - не захватывает, читает при выполнении

```
int start = ...
        for (int i = 0; i < count; i++) {
    ActionListener linstener = event -> {
        start--; //ERROR: Can't mutate captured value

        //ERROR: Cannot refer to changing i
        System.out.println(i);
    }
}
```


nexign

Применение

Iterable.forEach()

```
List <User> users = getAllUsers();
list.forEach ((u) ->
        System.out.println (u.getName()));
```

Collection.removeIf()

```
void removeUserByName (String userName) {
    List <User> users = getAllUsers();
    users.removeIf ((u) ->
            u.getName().equals(userName));
}
```

List.replaceAll()

```
List <Strin> list = getList();
    list.replaceAll((s) -> {
    StringBuilder sb = new StringBuilder(s);
    return sb.reverse().toString();
});
```

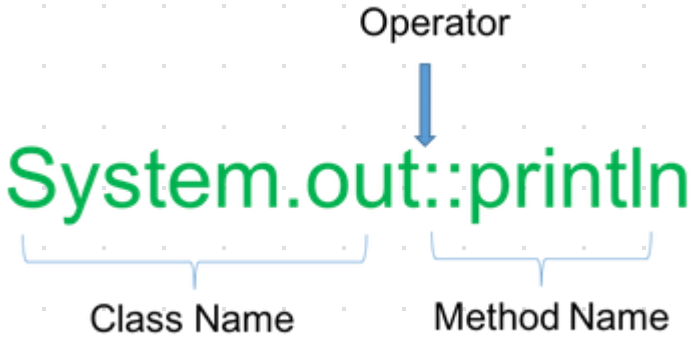

nexign

nexign

Ссылки на метод



Ссылка на метод (method reference)



Lambda	Method reference
event -> System.out.println(event)	System.out::println
(s1,s2)->s1.compareToIgnoreCase(s2)	String::compareToIgnoreCase
(x, y) -> Math.pow(x,y)	Math::pow

nexign

Типы ссылок на метод

Они могут использоваться только для замены лямбда-выражения с одним методом.

Lambda	Method reference	Type
(x, y, z…) → object.method(x, y, z…)	object::method	Ссылка на нестатический метод конкретного объекта
(x, y, z…) → x.instanceMethod(y, z…)	Class::instanceMethod	Ссылка на нестатический метод любого объекта конкретного типа
(x, y, z…) → Class.staticMethod(x, y, z…)	Class::staticMethod	Ссылка на статический метод
(x, y, z…) → new Class(x, y, z…)	Class::new	Ссылка на конструктор

nexign

nexign

Функциональные интерфейсы



Функциональные интерфейсы

Стандартная библиотека содержит пакет java.util.function

- **Функции** – принимают параметр и возвращает значение другого типа
- **Поставщики (Suppliers)** – не принимают ничего, возвращают (поставляют) значение.
- **Потребители (Consumers)** – принимают (потребляют) значение, ничего не возвращают
- **Предикаты** – принимают параметр, возвращают boolean
- **Операторы** – просто функция с одинаковым типом параметров и результата

nexign

Function

Function(функция) используется для преобразования входного параметра или в двух параметров (**для BiFunction)** в какое-либо значение, тип значение может не совпадать с типом входных параметров.

```
@FunctionalInterface
public interface Function<T, R> {
    R apply(T t);
}

@FunctionalInterface
public interface BiFunction<T, U, R> {
    R apply(T t, U u);
}
...
Function<String, Integer> func = s -> s.length();
    System.out.println(func.apply("Функциональные интерфейсы в Java");
BiFunction<String, String, String> funcBi = (s1, s2) -> s1.concat(s2);
    System.out.println(funcBi.apply("Функциональные интерфейсы ", " в Java"));
```


nexign

Supplier

(поставщик) используется для создание какого-либо объекта без использования входных параметров

```
package java.util.function;
@FunctionalInterface
public interface Supplier <T> {
    T get();
    // ..
}
Supplier<String> sup = () -> "Функциональные интерфейсы в Java";
System.out.println(sup.get());
```


nexign

Consumer

(потребитель) используется в том случае, если нам нужно применить какое-то действие или операцию к параметру (или к двум параметрам для **BiConsumer**) и при этом в возвращаемом значении нет необходимости

```
@FunctionalInterface
public interface Consumer<T> {
    void accept(T t);
}

@FunctionalInterface
public interface BiConsumer<T, U> {
    void accept(T t, U u);
}
...
Consumer<String> con = s -> System.out.println(s);
    con.accept("Функциональные интерфейсы в Java");
BiConsumer<String, String> con = (s1, s2) -> System.out.println(s1 + s2);
    con.accept("Функциональные интерфейсы в Java - ", "BiConsumer");
```


nexign

Predicate

(утверждение) наиболее часто применяется в фильтрах и сравнении

```
package java.util.function;
@FunctionalInterface
public interface Predicate <T> {
    boolean test (T t);
    default Predicate <T> negate () {
    // ...
    }
    // and , or
}
```


nexign

Unary operator

Используется в случае, если аргумент и возвращаемое значение одного типа.

```
@FunctionalInterface
public interface UnaryOperator<T> extends Function<T, T>
{
    …
}
```


nexign

Comparable & Comparator

Comarable<T>

```
public int compareTo(T o);
```

- < 0 → this < o
- == 0 → this == o
- > 0 → this > 0

Делает объекты «сравнимыми»

natural comparison method

Comparator<T>

```
@FunctionalInterface
...
int compare (T o1, T o2);
```

- < 0 → o1 < o2
- == 0 → o1 == o2
- > 0 → o1 > o2

Задаёт правила/ порядок сравнения

nexign

Comparator

```
@FunctionalInterface
public interface Comparator<T> {
    int compare(T o1, T o2);
    boolean equals(Object obj);
    default Comparator<T> reversed() {
        return Collections.reverseOrder(this);
    }
    ...
}
```

```
String[] friends = {"Peter", "Paul", "Mary"};

Arrays.sort(friends,
        new Comparator<String>() {
    @Override
    public int compare(String o1, String o2) {
        return o1.length() - o2.length();
    }
});
```


nexign

Применение функциональных интерфейсов

- find(Predicate<T> predicate)
- sort(Comparator<T> order)
- forEach(Consumer<T> action)
- map(Function<T, R> func)

nexign

nexign

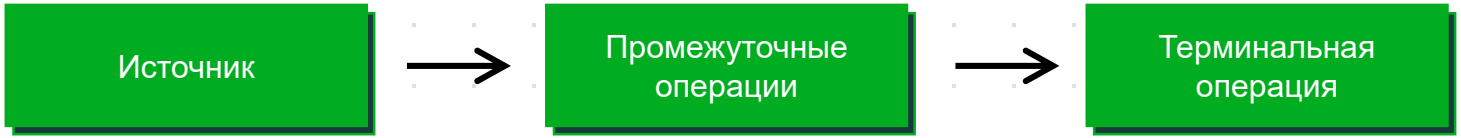
StreamAPI



Stream API

Stream
последовательность элементов (возможно, бесконечная) с поддержкой различных преобразований

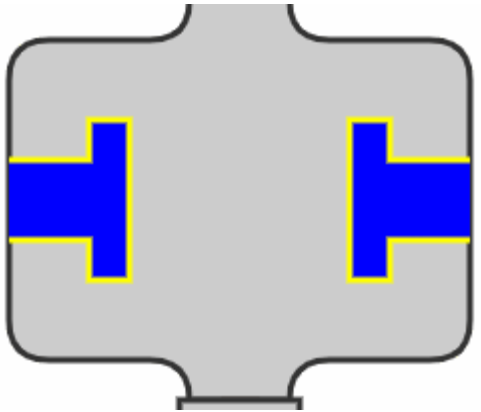
- java.util.stream
- Появились в Java8, вместе с lambdas & method references
- Декларативный подход к обработке данных: описываем что хотим получить, а не как мы это получим
- Может потребить не все элементы
- Нельзя переиспользовать



nexign

Создание источника

blockStream



squashedStream

```
List<Block> blocks = ...;

Stream<Block> blocksStream = blocks.stream();

Stream<SquashedBlock> squashedStream =
        blocksStream.map(Block::squash);
```

Автор анимаций — Тагир Валеев :
https://stackoverflow.com/questions/35150231/java-streams-lazy-vs-fusion-vs-short-circuiting

nexign

Создание источника

```
//Пустого
Stream<Foo> stream0 = Stream.empty();

//Перечислением элементов
Stream<String> stream1 =
        Stream.of("gently", "down", "the", "stream");
//Из массива
Stream<String> stream2 =
        Arrays.stream("gently down the stream".split(" "));
//Из коллекции
List<String> strings = ...
Stream<String> stream3 = strings.stream();

//Из API
Path path = Paths.get(...);
Stream<Path> stream4 = Files.list(path);
```

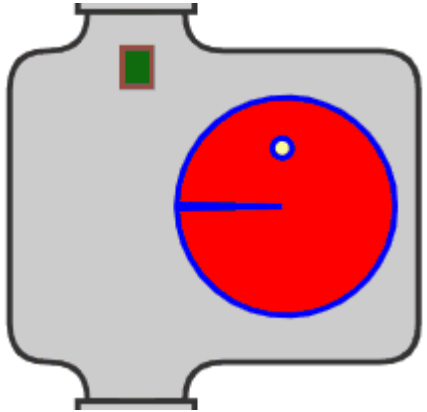
Специализации

- java.util.stream.Stream
- java.util.stream.IntStream
- java.util.stream.LongStream
- java.util.stream.DoubleStream

nexign

Промежуточные операции

squashedStream



filtredStream

```
Stream<SquashedBlock> filteredStream =
        squashedStream.filter(block ->
                block.getColor() != YELLOW);
```


nexign

Промежуточные операции

Метод stream	Описание	Пример
filter	Отфильтровывает записи, возвращает только записи, соответствующие условию	collection.stream().filter("a1"::equals).count();
skip	Позволяет пропустить N первых элементов	collection.stream().skip(collection.size() — 1).findFirst().orElse("1");
distinct	Возвращает стрим без дубликатов (для метода equals)	collection.stream().distinct().collect(Collectors.toList())
map	Преобразует каждый элемент стрима	collection.stream().map((s) -> s + "_1").collect(Collectors.toList())
peek	Возвращает тот же стрим, но применяет функцию к каждому элементу стрима	collection.stream().map(String::toUpperCase).peek((e) -> System.out.print("," + e)).collect(Collectors.toList())
limit	Позволяет ограничить выборку определенным количеством первых элементов	collection.stream().limit(2).collect(Collectors.toList())
sorted	Позволяет сортировать значения либо в натуральном порядке, либо задавая Comparator	collection.stream().sorted().collect(Collectors.toList())
mapToInt, mapToDouble, mapToLong	Аналог map, но возвращает числовой стрим (то есть стрим из числовых примитивов)	collection.stream().mapToInt((s) -> Integer.parseInt(s)).toArray()
flatMap	Похоже на map, но может создавать из одного элемента несколько	collection.stream().flatMap((p) -> Arrays.asList(p.split(",")).stream()).toArray(String[]::new)

nexign

Терминальная операция

filtredStream



```
filteredStream
  .forEach(System.out::println);
```

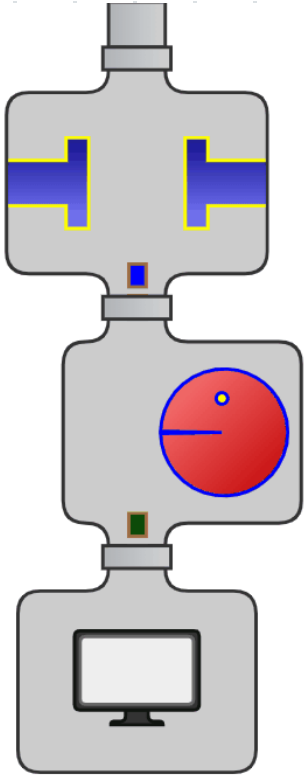

nexign

Терминальная операция

Метод stream	Описание	Пример
findFirst	Возвращает первый элемент из стрима (Optional)	collection.stream().findFirst().orElse("1")
findAny	Возвращает любой подходящий элемент из стрима (Optional)	collection.stream().findAny().orElse("1")
collect	Представление результатов в виде коллекций и других структур данных	collection.stream().filter((s) -> s.contains("1")).collect(Collectors.toList())
count	Возвращает количество элементов в стриме	collection.stream().filter("a1"::equals).count()
anyMatch	Возвращает true, если условие выполняется хотя бы для одного элемента	collection.stream().anyMatch("a1"::equals)
noneMatch	Возвращает true, если условие не выполняется ни для одного элемента	collection.stream().noneMatch("a8"::equals)
allMatch	Возвращает true, если условие выполняется для всех элементов	collection.stream().allMatch((s) -> s.contains("1"))
min	Возвращает минимальный элемент (используется компаратор)	collection.stream().min(String::compareTo).get()
max	Возвращает максимальный элемент (используется компаратор)	collection.stream().max(String::compareTo).get()
forEach	Применяет функцию к каждому объекту стрима	set.stream().forEach((p) -> p.append("_1"));
forEachOrdered	Применяет функцию к каждому объекту стрима, сохранение порядка элементов гарантирует	list.stream().forEachOrdered((p) -> p.append("_new"));
toArray	Возвращает массив значений стрима	collection.stream().map(String::toUpperCase).toArray(String[]::new);
reduce	Позволяет выполнять агрегатные функции над всей коллекцией и возвращать один результат	collection.stream().reduce((s1, s2) -> s1 + s2).orElse(0)

nexign

Stream inline



```
blocks.stream()
        .map(Block::squash)
        .filter(block -> block.getColor() !=YELLOW)
        .forEach(System.out::println);
```


nexign

Практика

- Создать классы:
- Клиент
  - id, email, List<Order>
- Заказ
  - name, cost, date
- Создать несколько экземпляров клиентов со связью «один ко многим» по отношению к экземплярам заказов
- При помощи StreamsAPI
  - найти всех клиентов у которых >2 заказов
  - отобразить последние 2 заказа



nexign

Q&A



nexign

Многопоточность

Concurrency collections



nexign

nexign

Класс Thread и интерфейс Runnable



Когда использовать?

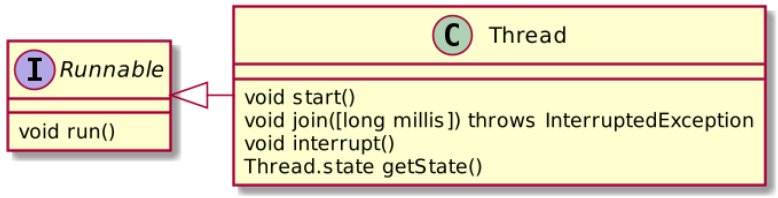
- Много CPU-ядер, вычислительная задача хорошо разделяется на подзадачи.
- Подзадачи блокируются на вводе/выводе, можно параллельно ждать других или делать что-то полезное.
- Нужно быстро отреагировать на запрос, а потом дать детальный ответ (пользовательский интерфейс).
- Многопользовательский сервис (каждый запрос в своём треде).

nexign

Интерфейс Runnable

- Нечто, что можно выполнить
- Содержит единственный метод void run();
- Может использоваться для описания действий, которые должны быть выполнены в отдельном потоке
- Вместе с тем, напрямую с потоками не связан и может быть использован и сам по себе

```
Runnable runnable = new Runnable() {
    @Override
    public void run() {
    // do some work
    }
};
Thread thread = new Thread(runnable);
```



nexign

Создание потока: Thread

- Выполняемый поток
- Реализует Runnable: class Thread implements Runnable
- Метод run() по умолчанию не делает ничего и должен быть переопределен в производном классе
- Для запуска потока на исполнение служит метод start()

```
Thread thread = new Thread () {
    @Override
    public void run () {
    // do some work
    }
}
```


nexign

Методы

- **join()**: ожидает завершение потока
- **run()**: определяет точку входа в поток
- **sleep()**: приостанавливает поток на заданное количество миллисекунд
- **start()**: запускает поток, вызывая его метод run()
- **getName()**: возвращает имя потока
- **setName(String name)**: устанавливает имя потока

getPriority(): возвращает приоритет потока

setPriority(int proirity): устанавливает приоритет потока.. В этот метод в качестве параметра передается числовое значение приоритета - от 1 до 10. По умолчанию главному потоку выставляется средний приоритет - 5.

isAlive(): возвращает true, если поток активен

isInterrupted(): возвращает true, если поток был прерван

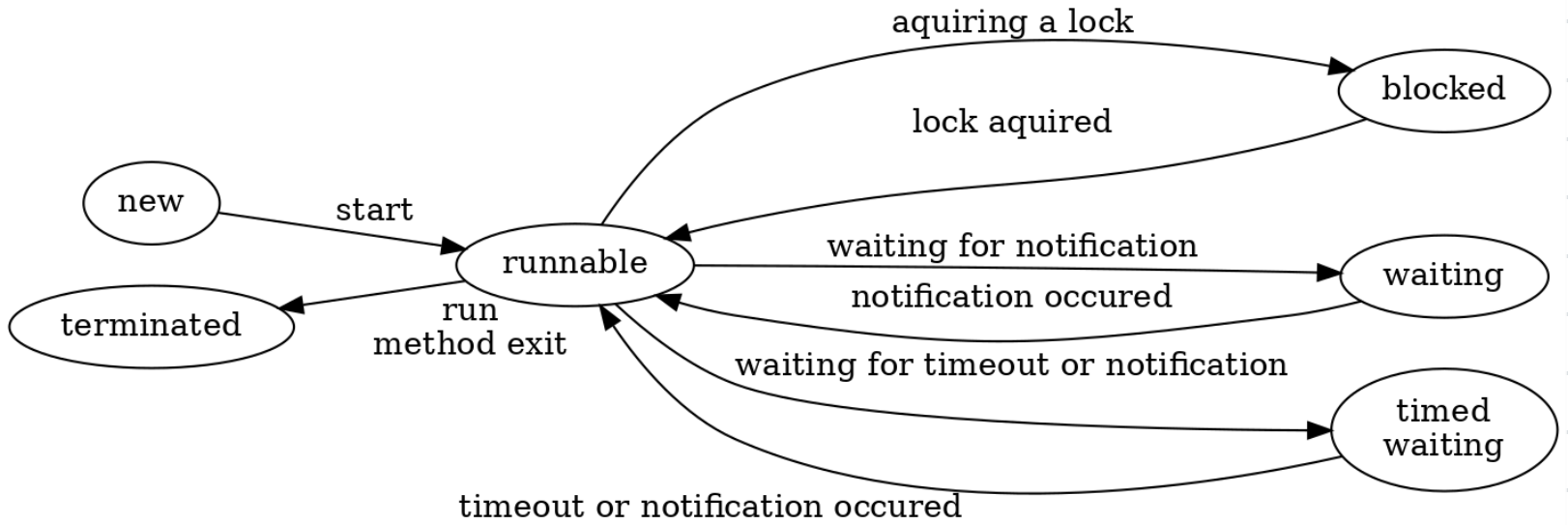
nexign

Стандартные потоки

- Главный поток
  - Создаётся при запуске программы
  - Выполняет функцию main. Может создавать другие потоки
  - Они, в свою очередь, могут создавать ещё потоки
- Графические библиотеки
- Поток обработки сообщений (обычно создаётся внутри библиотеки)
- Дополнительные потоки сборщика мусора (Garbage Collector Threads)

nexign

Состояния треда



nexign

Прерывание потока

- thread.interrupt()
- Если поток находится в ожидании (sleep, join, wait), то ожидание прерывается исключением InterruptedException
- Иначе у потока просто устанавливается флаг interrupted
  - флаг проверяется методами interrupted() и isInterrupted()
  - проверять флаг (isInterrupted) и завершать поток надо самостоятельно
- thread.join()

nexign

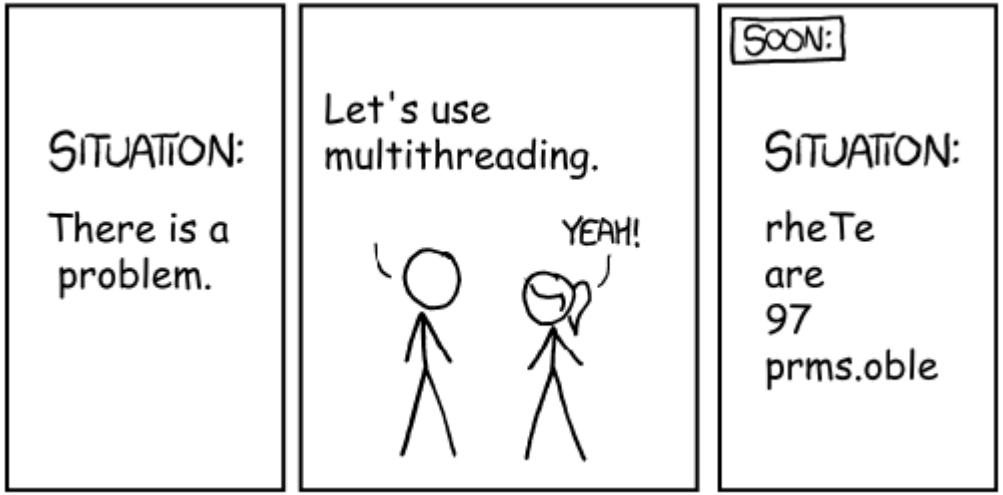
nexign

Разделённый ресурс



Проблемы shared state

- Race condition
- Stale values
- Reordering



nexign

Race Condition

```
class DumbCounter {
    int count;
    void increment(){
        count++;
    }
}

DumbCounter c1 = new DumbCounter();
IntStream.range(0, 1000000).forEach(i->c1.increment());

DumbCounter c2 = new DumbCounter();
IntStream.range(0, 1000000).parallel().forEach(i->c2.increment());

System.out.printf("%d, %d%n", c1.count, c2.count);

//1000000,??????
```


nexign

State Values

```
class DumbWayToFallAsleep implements
Runnable {
    private boolean asleep;

    public void setAsleep(boolean
asleep){
        this.asleep = asleep;
    }

    @Override
    public void run() {
        while (!asleep){
            //countSomeSheep
            //УСНЁМ ЛИ МЫ?
        }
    }
}
```


nexign

Java Memory Model

Часть спецификации языка Java

Описывает взаимодействие приложения с памятью

Не зависит от реализации JVM, операционной системы и железа

отвечает на вопрос: «при каких условиях поток, читающий переменную увидит значение

nexign

volatile

- Переменные классов могут быть определены с ключевым словом volatile.
- **Запись в volatile-переменную *happens-before* чтения из этой переменной в другом потоке.**
- Это автоматом делает видимыми изменения в других переменных. Полагаться на это не рекомендуется: это работает, но делает код хрупким. В процессе рефакторинга можно поменять порядок доступа к переменным и тем самым незаметно поломать программу.

nexign

Solve stale value

```
class NotSoDumbWayToFallAsleep implements
Runnable {
    private volatile boolean asleep;

    public void setAsleep(boolean asleep){
        this.asleep = asleep;
    }

    @Override
    public void run() {
        while (!asleep){
            //countSomeSheep
            //...
        }
    }
}
```


nexign

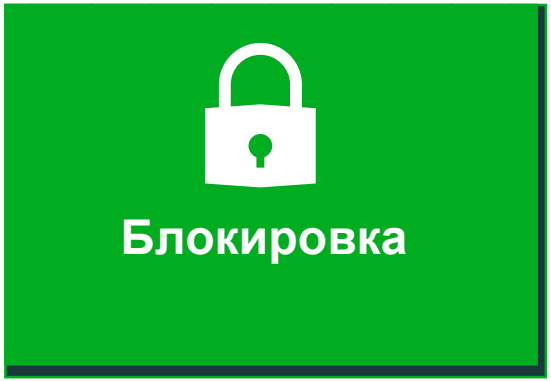
Доступ к разделяемому ресурсу

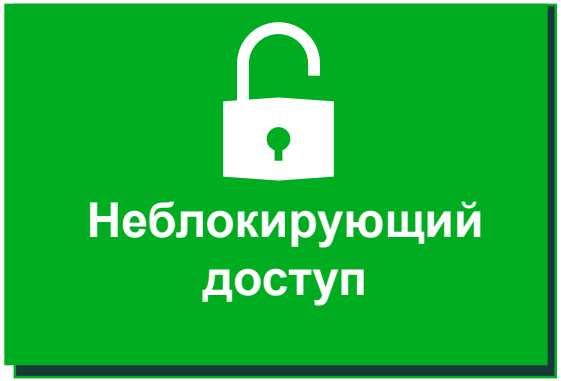
Блокировка - взаимное исключение (пока один поток что-то делает, другие не могут ему помешать)

- Поток ожидает уведомление от других

Неблокирующий доступ

- Lock-free: гарантируется общий прогресс
- Wait-free: гарантируется прогресс в каждом потоке исполнения





nexign

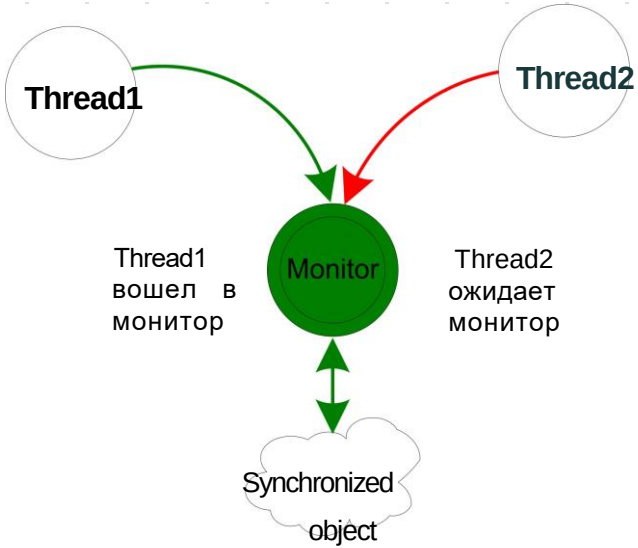
Синхронизация

- **Синхронизация по ресурсам (управление ресурсами)**
  - Объекты совместно используют ресурсы данных
  - Ресурсы сами следят за корректностью работы
  - Java: synchronized + volatile
- **Синхронизация по событиям (управление временем)**
  - Для совместной работы потоки уступают процессорное время друг другу и/или уведомляют друг друга о возможности продолжения работы
  - Java: Object.wait(), notify(), notifyAll()

nexign

Монитор





nexign

synchronized

- **Синхронизация блоков** — по монитору указанного объекта
- **Синхронизация методов** — по монитору текущего объекта (this)
- **Синхронизация статических методов** — по монитору класса

Синхронизованный метод

```
public synchronized void doSomething () {
    // ...
}
```

Синхронизованный блок внутри метода

```
public void doSomething () {
    synchronized ( obj ) {
        // ...
    }
}
```


nexign

Immutable state

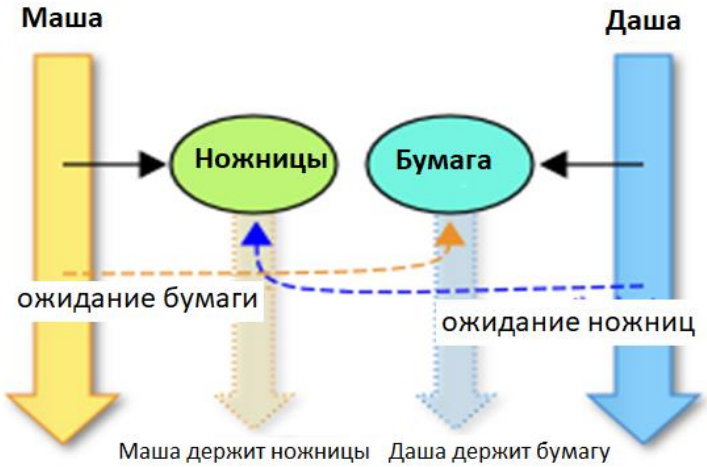
> Если поток увидел ссылку на объект, и она не утекала из конструктора, то он гарантированно увидит все final-поля, записанные в конструкторе
>
> Неизменяемые объекты – друзья многопоточности!

nexign

Deadlock

Как происходит блокировка?

- Два потока работают с общими ресурсами.
- Поток 1 захватывает Ресурс 1 и начинает операции с ним.
- Поток 2 последовательно захватывает Ресурс 2 и Ресурс 1.
- Поток 2 не получает доступа к Ресурсу 1 и в ступоре ждёт, когда тот освободится.
- Поток 1 не завершил работу с Ресурсом 1, но пытается захватить Ресурс 2 и тоже впадает в ступор.



nexign

Ожидание и уведомления

Допустимо только внутри synchronized

- **void wait()** – уступить монитор и перейти в режим ожидания
- **void notify()** – пробудить один из потоков, вызвавший wait() на данном объекте
- **void notifyAll()** –пробудить все потоки, вызвавшие wait() на данном объекте

nexign

nexign

java.util.concurrent.*



ReentrantLock

- Класс java.util.concurrent.locks.ReentrantLock
- Обеспечивает взаимное исключение потоков, аналогичное synchronized-блокам
- Операции:
  - lock()
  - unlock()

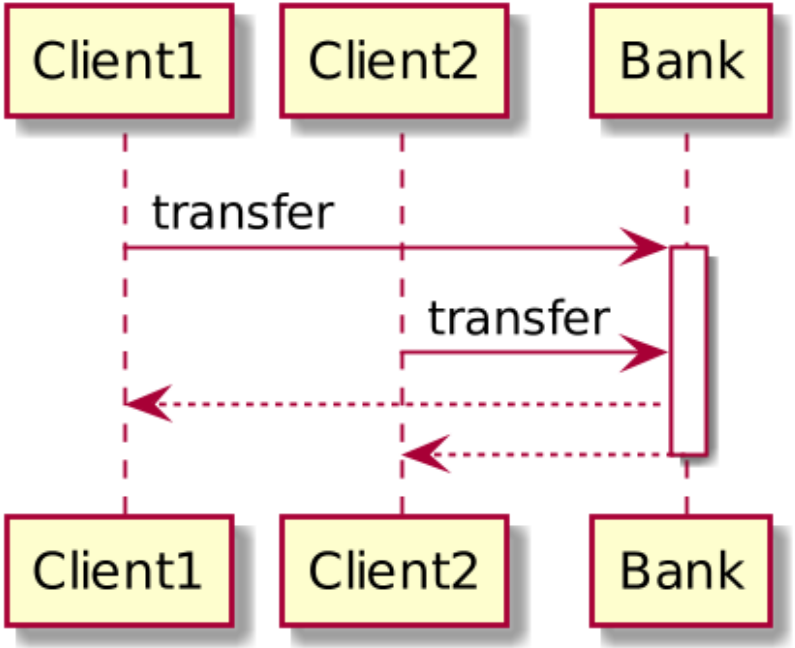
```
Lock lock = new ReentrantLock();
lock.lock();
try {
    doSomething();
} finally {
    lock.unlock();
}
```


nexign

Не синхронизованное выполнение

```
void dumbMoneyTransfer(
        int from, int to, int amount){

    accounts[from]-=amount;
    accounts[to]+=amount;
}
```

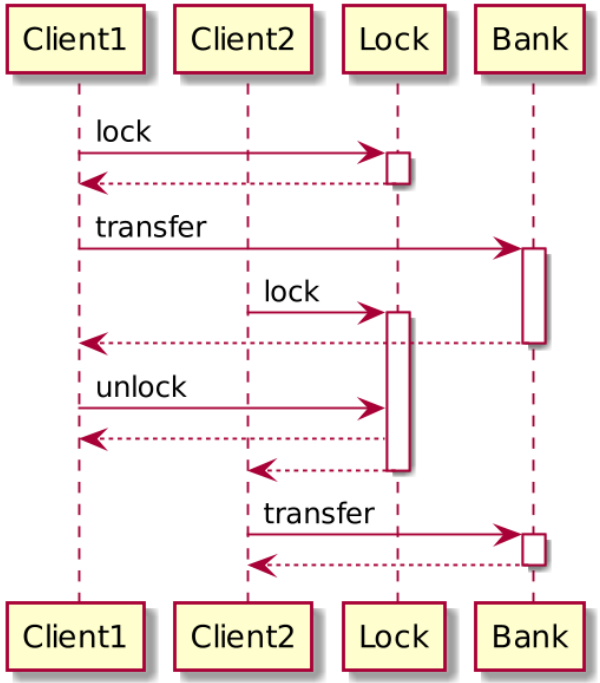


nexign

Синхронизованное выполнение

```
private ReentrantLock bankLock = new
ReentrantLock();

void moneyTransfer(int from, int to, int amount) {
    bankLock.lock();
    try {
        accounts[from]-=amount;
        accounts[to]+=amount;
    } finally {
        bankLock.unlock();
    }
}
```



nexign

Condition

- Класс java.util.concurrent.locks.Condition
- Аналог wait/notify
- Привязан к Lock’у
- У одного Lock’а может быть много Condition’ов

```
Lock lock = new ReentrantLock();
Condition condition = lock.newCondition ();
lock.lock();
try {
    while (!conditionSatisfied()) {
        condition.await ();
    }
} finally {lock.unlock(); }
    // somewhere else in our program
lock.lock ();
try {
    condition.signal();
} finally { lock.unlock();}
```


nexign

Потокобезопасные структуры

java.util.concurrent

- Атомарные переменные
- Неблокирующие коллекции
- Блокирующие коллекции
- Примитивы синхронизации
- Пулы потоков
- CompletableFuture

nexign

Atomics

java.util.concurrent.atomic

- AtomicBoolean
- AtomicInteger
- AtomicLong
- AtomicReference<V>

Операции

- getAndSet(newValue)
- compareAndSet(expect, update)
- incrementAndGet() /decrementAndGet()
- getAndIncrement() /getAndDecrement()
- getAndAdd(delta)
- addAndGet(delta)
- getAndUpdate(updateFunction)
- updateAndGet(updateFunction)

nexign

Примитивы синхронизации

Синхронизаторы

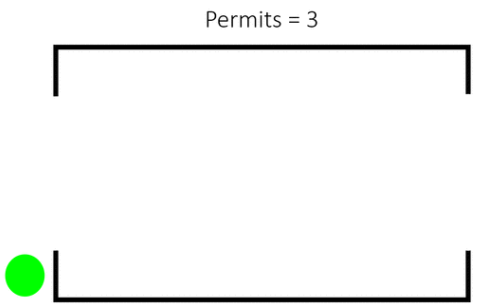
вспомогательные утилиты для синхронизации потоков, которые дают возможность разработчику регулировать и/или ограничивать работу потоков и предоставляют более высокий уровень абстракции, чем основные примитивы языка (мониторы).

- Semaphore
- CountDownLatch
- CyclicBarrier
- Exchanger
- Phaser

nexign

Semaphore

- Класс java.util.concurrent.Semaphore
- Ограничивает одновременный доступ к ресурсу
- В отличие от synchronized-блока, одновременно могут работать несколько потоков (но не более заданного N)
- **Операции:**
  - void acquire()
  - void release()



nexign

Semaphore

```
Semaphore semaphore = new Semaphore(10);
semaphore.acquire();
try {
    // up to 10 threads may
    // execute this code concurrently
} finally {
    semaphore.release();
}
```


nexign

Executors

- Тред — дорогой ресурс, поэтому мы хотим:
  - ограничивать количество наших тредов, чтобы не устроить Out of Memory,
  - переиспользовать имеющиеся треды, подавая им новые задачи после завершения старых,
- но если какой-то тред «вылетел» — автоматически создавать новый.
- В стандартной библиотеке для этого есть Thread Pools, не надо ничего делать самостоятельно.
- Никто не использует Thread API напрямую

nexign

ExecutorService

- Класс java.util.concurrent.Executors
  - **ExecutorService newSingleThreadExecutor() –** пул одного потока
  - **ExecutorService newFixedThreadPool(int nThreads**) - фиксированный размер пула
  - **ExecutorService newCachedThreadPool()** - пул растёт по необходимости, держит неактивный тред 60 секунд

nexign

Future

Абстракция результата "in progress", который можно ждать, а можно и отменить

```
public interface Future<V> {
    V get() throws InterruptedException, ExecutionException;
    V get(long timeout, TimeUnit unit)
            throws InterruptedException, ExecutionException,
TimeoutException;
    boolean cancel(boolean mayInterruptIfRunning);
    boolean isDone();
    boolean isCancelled();
}
```


nexign

Обработка задач

Запускаем и ждём, пока все выполнятся

```
<T> List<Future<T>> invokeAll(
            Collection<? extends Callable<T>> tasks)
    throws InterruptedException
```

Запускаем, возвращаем первый успешный результат, отменяем остальные

```
  <T> T invokeAny(Collection<? extends Callable<T>> tasks)
          throws InterruptedException, ExecutionException;
```


nexign

CompletableFuture

- Появились в Java 8, расширяют Future.
- Позволяют явно задать результат (отсюда 'Completable') и собрать цепочку асинхронных вычислений.

```
CompletableFuture<Integer> f = new CompletableFuture<>();
executor.execute(() -> { int n = workHard(arg);
    f.complete(n);});

executor.execute(() -> {int n = workSmart(arg);
    f.complete(n);});

executor.execute(() -> {Throwable t = ...;
    f.completeExceptionally(t); });
```


nexign

Concurrent collections

Неблокирующие коллекции

- ConcurrentLinkedQueue (LinkedList)
- ConcurrentLinkedDeque (LinkedList/ArrayDeque)
- CopyOnWriteArrayList (ArrayList)
- CopyOnWriteArraySet
- ConcurrentHashMap (+ newKeySet()) (HashMap/HashSet)
- ConcurrentSkipListMap (TreeMap)
- ConcurrentSkipListSet (TreeSet)

- Простые операции атомарны
- Пакетные операции (addAll, removeAll) могут быть неатомарны!
- Как правило, длина не хранится (isEmpty()!)
- Не кидает ConcurrentModificationException
- могут быть видны или не видны

nexign

Concurrent collections

Блокирующие коллекции

- SynchronousQueue (без памяти, одна вставка = одно удаление)
- ArrayBlockingQueue (фиксированный размер)
- LinkedBlockingDeque (размер может быть не фиксирован)
- PriorityBlockingQueue (PriorityQueue x ArrayBlockingQueue)

nexign

Q&A



nexign

Аннотации

Reflection API



nexign

Аннотации



Аннотации

Аннотация

способ разметки кода на Java, через добавление метаинформации к различным элементам программы на Java.

Аннотация выполняет следующие функции :

- предоставляет необходимую информацию для компилятора;
- предоставляет информацию различным инструментам для генерации другого кода, конфигураций и т. д.;
- может быть использована во время работы кода.

Работают приблизительно как модификаторы (public, static), но существует развитый механизм создания собственных аннотаций.

- Могут быть обработаны на трёх этапах:
  - на этапе кодогенерации перед компиляцией (annotation processing, Language Model API),
  - на этапе инструментации байт-кода (Instrumentation API),
  - на этапе выполнения кода (через Reflection API).

nexign

Стандартные аннотации

Annotation interface	Применимость	Назначение
@Deprecated	Все элементы API	Данный элемент API deprecated — т. е. сохранён для обратной совместимости, запрещён к использованию в новых проектах, и возможно будет удалён в будущем.
@SuppressWarnings	Все, кроме пакетов и аннотаций	Подавляет предупреждения заданного типа от компилятора и линтеров.
@Generated	Все элементы API	Помечает код, как автоматически сгенерированный (как следствие, этот код не следует модифицировать руками).
@Override	Методы	Метод переопределяет метод в базовом классе. Ошибка компиляции, если это не так (см. лекцию про классы).
@FunctionalInterface	Интерфейсы	Помечает интерфейс как функциональный (имеющий единственный абстрактный метод). Ошибка компиляции, если это не так (см. лекцию про lambdas & method refs).
@SafeVarargs	Методы и конструкторы	Декларирует безопасное использование параметризованного varargs (см. лекцию про Generics)

nexign

Синтаксис определения аннотаций

```
modifiers @interface AnnotationName {
    type elementName();
     ...
    type elementName default value;
}
```

Пример

```
public @interface BugReport {
    String assignedTo() default "";
    int severity();
}
```

annotation-интерфейсы — это специальный вид интерфейсов

Наследуются от java.lang.annotation.Annotation.

Нельзя расширять наследованием, нельзя параметризовать через generics.

Методы не могут иметь параметров.

nexign

Пример определения аннотации

```
public @interface BugReport {
    //вложенные типы допустимы!
    enum Status {UNCONFIRMED, CONFIRMED, FIXED, NOTABUG}
    boolean showStopper() default false;
    String assignedTo() default "";

    //метакласс
    Class<?> testCase() default Void.class;
    Status status() default Status.UNCONFIRMED;

    //Тип аннотации. При конструировании используется синтаксис
    //аннотации
    Reference ref() default @Reference(id = "");
    String[] reportedBy();
}
```


nexign

Использование аннотаций

```
@AnnotationName(elementName1=value1, elementName2=value2,...)

@AnnotationName(singleValue)

@AnnotationName //no values, no parens
```

```
@BugReport(assignedTo="Harry", severity=10)
//порядок не имеет значения
@BugReport(severity=10, assignedTo="Harry")
//если заданы значения по умолчанию, то можно пропускать
@BugReport(severity=10)
//если параметров нет или все имеют умолчания,
//то можно не ставить скобки
@BugReport
```


nexign

Доступность (retention)

SOURCE – только в исходном тексте

CLASS – в исходном тексте и в байткоде

RUNTIME – в исходном тексте, в байткоде и во время исполнения через reflection

nexign

Управление аннотациями

Какие элементы кода могут быть аннотированы

```
@Target({ElementType.TYPE, ElementType.METHOD})
```

На каком уровне сохраняются эти аннотации

```
@Retention(RetentionPolicy.Source)
    public @interface BugReport {
        ...
    }
```

ElementType	Применимость аннотаций
ANNOTATION_TYPE	Интерфейсы аннотаций
PACKAGE	Пакеты
TYPE	Классы (в т. ч. enums) и интерфейсы (в т. ч. @interface)
METHOD	Методы
CONSTRUCTOR	Конструкторы
FIELD	Поля
PARAMETER	Параметры методов или конструкторов
LOCAL_VARIABLE	Определения локальных переменных
TYPE_PARAMETER	Типовые параметры
TYPE_USE	Использования типов

nexign

Аннотации в декларации

```
@Entity
public class User {

    @Caption("user")
    public User getUser(@Param("id") String userId) {

        @SuppressWarnings("unchecked")
        List<User> users = ....
    }
}


class Cache<@Immutable V> {
    ...
}
```

- Packages
- Classes (включая enum)
- Interfaces (включая annotation interfaces)
- Methods
- Constructors
- Instance fields (включая константы enum)
- Local variables' declarations
- Parameter variables' declarations
- Type parameters

nexign

Аннотации в типах

```
//аннотированный параметр, утверждает, что userId --
не `null`
User getUser(@NonNull String userId) { ... }

//Аннотированные использования типа: каждая строчка не
`null`
List<@NonNull String> list = ...
Comparator.<@NonNull String>reverseOrder()
class Warning extends @Localized Message { ... }

List<@Localized ? extends Message> = ...

        (@Localized String) text

if (text instanceof @Localized String) ...

//и т. д., список примеров не полный.
// Аннотации могут быть почти везде!
```


nexign

nexign

ReflectionAPI



Reflection API

Reflection
информация о программе из программы

Классы

Методы

Поля

Типы

Аннотации

Пакеты

Модули (Java 9)

nexign

Использование reflection

- Динамическая загрузка (плагины, расширения)

```
MyInterface iface = new URLClassLoader(new URL[]
{url}).loadClass(className).asSubClass(MyInterface.class).getConstructor()
.newInstance();
```

- Поддержка нескольких версий зависимости в runtime
- Dependency injection
- Сериализация

nexign

Reflection API

```
Employee e = ... //присваиваем Employee
Employee m = ... //присваиваем Manager

//Получаем информацию в Runtime
Class<? extends Employee> ce = e.getClass();
Class<? extends Employee> cm = m.getClass();

System.out.printf("%s, %s%n", ce.getName(), cm.getName());
//edu.phystech.hello.Employee, edu.phystech.hello.Manager

System.out.printf("%s, %s%n", ce.getSimpleName(), cm.getSimpleName());
//Employee, Manager
```


nexign

Reflection API

```
Em em = new Em();
Class<? extends Em> e = em.getClass();
Em newEm = e.getDeclaredConstructor().newInstance();
```

getXXX — все публичные элементы (поля, конструкторы, методы)

getDeclaredXXX — все элементы, задекларированные в данном классе (без унаследованных)

nexign

Class literals

Литералы

строки в коде, порождающие объекты или значения. Например, "foo" — литерал, порождающий объект String с содержимым "foo".

Литералы классов

специальные выражения в языке, порождающие объекты с типом Class<…>.

```
Class<String> c1 = String.class;
Class<Integer> c2 = Integer.class;

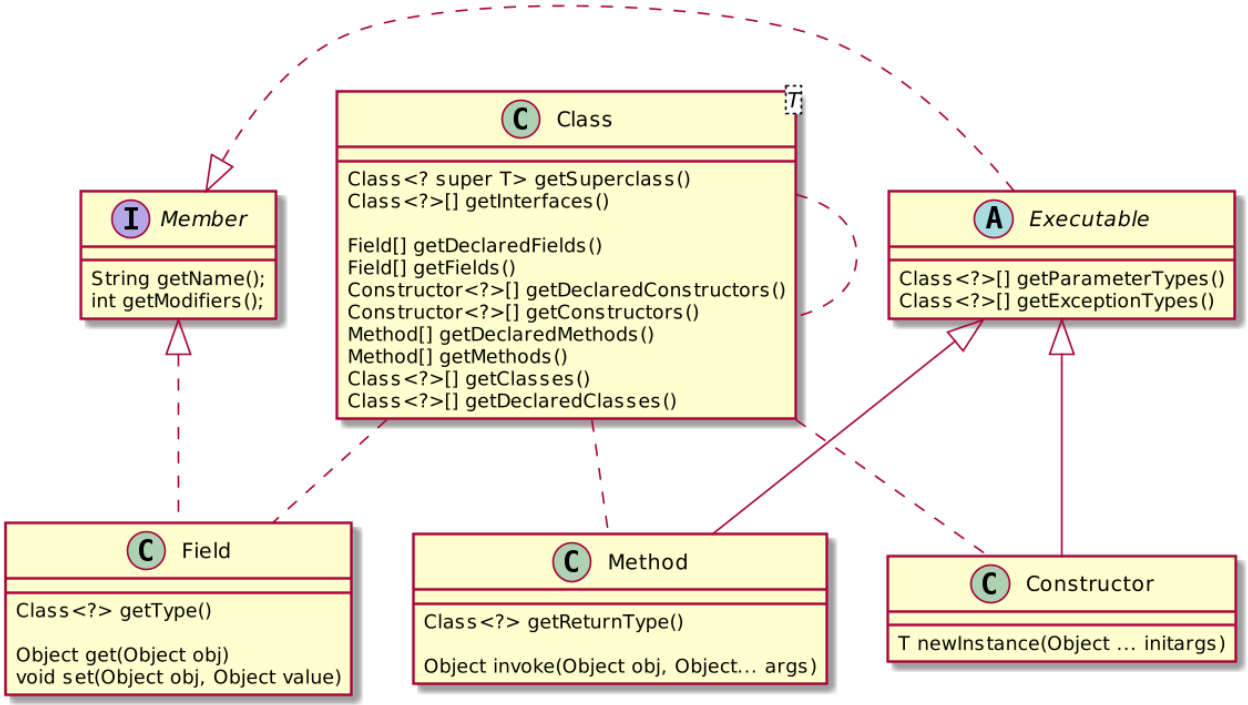
Class<Integer> c3 = int.class;
//А ещё double.class, boolean.class и т. д.

//При этом c2 и c3 -- существенно РАЗНЫЕ объекты
//(например, c3.getConstructors() возвращает пустой массив).

//Тут ничего неожиданного нет, массивами можно параметризовать:
Class<int[]> c4 = int[].class;
```


nexign

Анализ возможностей класса и его элементов



nexign

Доступ к private полям

```
Car car = new Car(500, "1233");
Class<? extends Car> carClass = car.getClass();
Field horsepowerField = carClass.getDeclaredField("horsepower");

horsepowerField.setAccessible(true);
System.out.println("Before change:" + horsepowerField.getInt(car));
horsepowerField.setInt(car, 751);
System.out.println("After change:" + horsepowerField.getInt(car));
```

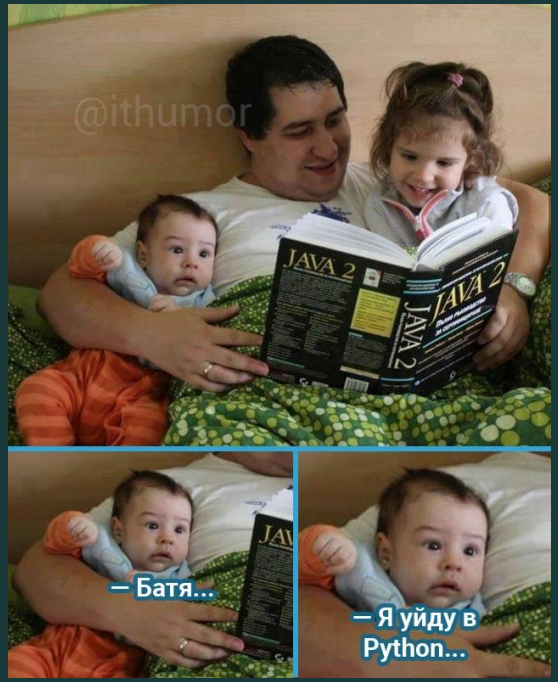

nexign

Q&A



Итоги

- У Java мощный API
- Многие классы не убираются из-за обратной совместимости
- Исключения необходимо обрабатывать
- Дженерики расширяют возможности в написании обобщенного кода
- Все коллекции параметризованны
- Реализаций коллекций много, но чаще встречаются
    - ArrayList
    - HashMap



Спасибо!


Aleksandr.Mityasov


@mityasov


Aleksandr.Mityasov@nexign.com

В настоящем документе содержится не предназначенная для публикации конфиденциальная информация и материалы, принадлежащие АО «Нэксайн» или его аффилированным лицам, охраняемые российским и международным законодательством об авторском, патентном праве, праве на секрет производства (ноу-хау), о товарных знаках и иных средствах индивидуализации.

Программное обеспечение АО «Нэксайн» и связанная с ним документация могут распространяться и использоваться только на основании отдельного лицензионного соглашения с АО «Нэксайн».

Любое использование, копирование, воспроизведение, публикация, обнародование, передача, изменение, переработка или перевод настоящего документа и содержащихся в нем информации и материалов, полностью или частично, в любой форме и любыми средствами без предварительного письменного разрешения АО «Нэксайн» строго запрещено.

Если настоящий документ предоставляется с программным обеспечением, лицензированным АО «Нэксайн», информация в нем предоставляется в соответствии с условиями гарантии, предоставляемой с лицензией на программное обеспечение.

Если настоящий документ не предоставляется с лицензионным программным обеспечением, информация в нем предоставляется «как есть» без каких-либо гарантий.

В обоих случаях АО «Нэксайн» не предоставляет никаких иных гарантий в отношении настоящего документа и содержащихся в нем информации и материалов, явных или подразумеваемых, включая, помимо прочего, подразумеваемые гарантии в отношении их качества, коммерческой пригодности, пригодности для определенной цели, ненарушения прав третьих лиц.

АО «Нэксайн» или его аффилированные лица ни при каких обстоятельствах не несут ответственность за любые прямые или косвенные убытки, случайные расходы или штрафы, возникшие в результате использования или невозможности использования материалов и/или информации, содержащихся в настоящем документе.

АО «Нэксайн» принимает надлежащие меры для обеспечения качества информации и материалов, содержащихся в настоящем документе. Однако документ может содержать технические неточности или опечатки, отсутствие ошибок не гарантируется.

Настоящий документ может быть изменен или дополнен без какого-либо уведомления. АО «Нэксайн» не обязуется обновлять настоящий документ на основе изменений в продуктах или услугах, как собственных, так и третьих лиц. АО «Нэксайн» может вносить улучшения или изменения в продукты или услуги, описанные в настоящем документе, в любое время без предварительного уведомления.